7.动态链接

1._dl_start

8. 转入 _dl_start_final

4.调用 _dl_sysdep_start，进入平台相关启动逻辑

Q1：为什么要自重定位

Q2：bootstrap_map 是什么

2.依赖库装载阶段

2.1 主程序 link_map 补全阶段(_rtld_setup_main_map)

2.2 dl_map_object

3.重定位 _dl_relocate_object

3.1关键控制参数：

3.2两种绑定方式

3.3 判断代码段（text）是否有需要修改的重定位

3.4 核心：真正的重定位执行

3.4.1 架构相关初始化（PLT/GOT）

3.4.2 处理 RELR（压缩相对重定位）

3.4.3处理 REL / RELA

3.4.4 elf_dynamic_do_Rel

2. RTLD_BOOTSTRAP 分支

elf_machine_rel

1. 获取重定位地址和重定位类型

2. 快速处理 RELATIVE / RELATIVE64 / NONE

3. 解析符号所在对象和符号运行时地址

resolve_map()

_dl_lookup_symbol_x

图示：

do_lookup_x()

4. 处理 IFUNC

5.常见重定位类型分析

6.TLS 相关重定位总结

3.Lazy Binding 分支

elf_machine_lazy_rel

4.非 lazy 分支

1.RELATIVE 重定位优化

2.符号版本处理

3.audit机制

5.elf_dynamic_do_Rel整体执行流程总结：

3.5 PLT profiling / audit 支持

3.6 标记完成

3.7 恢复 text 段权限

4._dl_init

4.1 调用 INITFIRST 动态库

4.1.1 call_init

1. 防止代理对象重复初始化

2.确认对象已经重定位完成

3.防止重复执行构造函数

4.跳过主可执行文件

5.调用 DT_INIT

6.调用 DT_INIT_ARRAY

4.2 调用主程序的 .preinit_array

4.3 调用共享库构造函数

5._start

5.1 _start 如何准备 __libc_start_main 参数

6.__libc_start_main

6.1动态链接程序

6.2静态链接程序

6.1 静态链接路径详解：#ifndef SHARED

6.1.1 根据 argv 找到环境变量 envp

6.1.2 保存栈边界地址

6.1.3 解析辅助向量 auxv

6.1.4 初始化 glibc tunables

6.1.5 初始化 CPU features

6.1.6 static PIE 自重定位

6.1.7 IFUNC / IREL 重定位

6.1.8 设置 stack canary

6.1.9 初始化 pthread 最小环境 + 设置 pointer guard

6.2 注册动态链接器析构函数 rtld_fini

6.3 静态链接下的 libc early init

6.4 动态链接场景下初始化主程序

6.5 最终调用 main

7.延迟绑定

8.do_lookup_x

8.1关键数据结构

8.1.1scope

8.1.2 link_map

8.1.3 result

8.2读取查找作用域长度

8.3 遍历当前 scope 中的每个对象

8.4 获取符号表和字符串表

8.5 GNU hash 查找路径

8.5.1 Bloom filter 快速过滤

8.5.2 bucket 定位

8.5.3 遍历 GNU hash chain

8.6 SysV hash 查找路径

举例：

1.SysV Hash匹配示例

2.GNU Hash匹配示例

8.7 版本符号兼容逻辑

8.8 找到符号后的处理逻辑

8.8.1hidden/internal 符号不可被外部绑定

8.8.2 protected 符号检查

8.8.3 根据 binding 决定返回策略

```
136 #define RTLD_START asm ("\n\
137 .text\n\
138         .align 16\n\
139 .globl _start\n\
140 .globl _dl_start_user\n\
141 _start:\n\
142         mov %" RSP_LP ", %" RDI_LP "\n\
143         call _dl_start\n\
144 _dl_start_user:\n\
145         # Save the user entry point address in %r12.\n\
146         mov %" RAX_LP ", %" R12_LP "\n\
147         # Save %rsp value in %r13.\n\
148         mov %" RSP_LP ", % " R13_LP "\n\
149 "\
```

Plain Text

```
用户态，先执行动态链接器:
ld-linux 入口
-> _dl_start
  ->_dl_start_final
    -> _dl_sysdep_start          //进入平台相关启动逻辑
      -> dl_main
        -> _dl_map_object_deps       // 装载主程序依赖库
        -> _dl_relocate_object       // 做重定位
-> _dl_init                  // 跑初始化
-> 跳转到程序入口 _start
```

1._dl_start

_dl_start 是动态链接器 ld.so 的早期自举函数。但这时的 ld.so 只是“被装进内存了”

它运行时，用户程序尚未开始执行，而 ld.so 自身也还没有完成动态段解析和自重定位，因此还不具备完整运行能力。

1. 记录启动时间与准备附加信息

函数开头第一部分，处理的是启动时间和后续 _dl_start_final 需要的信息：

```c
//elf/rtld.c:518
static ElfW(Addr) __attribute_used__
    _dl_start (void *arg)
{
    #ifdef DONT_USE_BOOTSTRAP_MAP
    /* 记录动态链接器启动的最早时间点。 */
    rtld_timer_start (&start_time);
    #else
    /* 保存后续 _dl_start_final 需要的附加启动信息。 */
    struct dl_start_final_info info;

    /* 记录启动时间。此时仍处于自举早期阶段。 */
    rtld_timer_start (&info.start_time);
    #endif
.....
}
```

记录动态链接器最早的启动时间点

- 如果使用 bootstrap_map 路径，则额外构造 info，把后续 _dl_start_final 可能需要的附加信息先保存起来
- 这一段还不涉及重定位本身，但它说明 _dl_start 不只是“修地址”，还承担了启动上下文准备的职责。

2. 清理 bootstrap_map.l_info，为解析 .dynamic 做准备

```c
/* 部分清理 bootstrap_map 结构。
     这里只清理 l_info，因为后面要用它保存 .dynamic 的解析结果。

     不能直接调用 memset：
     1. 此时动态链接器自己还未完成重定位；
     2. 普通函数调用未必安全；
     3. 如果编译器支持内建 memset，则可安全展开为内联代码。 */
    #ifndef DONT_USE_BOOTSTRAP_MAP
    # ifdef HAVE_BUILTIN_MEMSET
    __builtin_memset (bootstrap_map.l_info, '\0', sizeof (bootstrap_map.l_info));
    # else
    for (size_t cnt = 0;
       cnt < sizeof (bootstrap_map.l_info) / sizeof (bootstrap_map.l_info[0]);
       ++cnt)
        bootstrap_map.l_info[cnt] = 0;
    # endif
    #endif
```

把 bootstrap_map 中用于保存动态段解析结果的 l_info[] 清零。

这里清的不是整个 bootstrap_map，而是只清理 l_info，原因是后面会调用 `elf_get_dynamic_info`，把 .dynamic 中的 DT_* 条目解析后存到这里。

这一步也体现了 _dl_start 所处阶段的特殊性：

此时 ld.so 自身还未完成重定位，普通函数调用不一定安全，所以连 memset 都要谨慎处理，只能依赖编译器内建或者手写循环，说明 _dl_start 运行在动态链接器最早期、最受限的自举阶段。

3. 计算 ld.so 自身的运行时装载地址

```c
 /* 计算动态链接器自身在运行时的装载基址。 */
   bootstrap_map.l_addr = elf_machine_load_address ();
```

它体现的是：计算动态链接器自身在当前进程中的实际装载基址。

这是 _dl_start 中极其关键的一步。因为 ld.so 虽然已经被内核映射进内存，但它的运行时地址不一定等于链接时地址。后面无论是找 .dynamic、找重定位表，还是修正 GOT、全局数据，全部都要以这个基

址为起点。

4. 定位 ld.so 自身的 .dynamic 段

```c
      /* 计算动态链接器自身 .dynamic 段在内存中的实际地址。 */
      bootstrap_map.l_ld = (void *) bootstrap_map.l_addr + elf_machine_dynami
  c ();

      /* 标记 .dynamic 段是否位于只读区域。 */
      bootstrap_map.l_ld_readonly = DL_RO_DYN_SECTION;
```

- 通过“装载基址 + .dynamic 相对位置”的方式，计算出 ld.so 自身 .dynamic 段在内存中的实际地址
- 标记 .dynamic 段是否位于只读区域

关键是第一句。

因为 .dynamic 段里保存着 DT_STRTAB、DT_SYMTAB、DT_RELA、DT_JMPREL 等运行时链接所需的核心信息，只有先找到 .dynamic，动态链接器才知道“我自己要修哪里、怎么修”。

_dl_start 先找到 ld.so 自己的动态段，为后续解析动态段和执行自重定位准备输入数据

5. 解析 .dynamic 段，提取 DT_* 信息

```c
   /* 解析自身的 .dynamic 段，填充 l_info 数组。
       后续自重定位会依赖这里得到的 DT_* 信息。 */
      elf_get_dynamic_info (&bootstrap_map, true, false);
```

这一调用的作用是：遍历 ld.so 自己的 .dynamic 段，把里面的各种 DT_* 条目解析出来，并填入 bootstrap_map.l_info[]。

- 解析自身 .dynamic
- 建立自重定位所需的数据索引
- 把原始动态段转换成 rtld 内部可直接访问的 l_info[]

例如这里会得到：

- 符号表位置 DT_SYMTAB
- 字符串表位置 DT_STRTAB

- 重定位表位置 DT_RELA / DT_REL
- PLT 重定位表位置 DT_JMPREL
- 各种大小字段如 DT_RELASZ、DT_PLTRELSZ
- 以及其它动态链接器后续需要的元数据

这一步完成后，bootstrap_map 才有了可以描述 ld.so 这个 ELF 对象的能力。也就是说，_dl_start 通过这一段代码，先把 ld.so 自己的动态信息整理成可直接索引使用的内部结构。

6. 对 ld.so 自身执行重定位(关键)

```c
    if (bootstrap_map.l_addr)
      {
          /* 对动态链接器自己执行重定位。
           完成后，普通函数调用、全局数据访问、GOT 访问才算真正安全。 */
          ELF_DYNAMIC_RELOCATE (&bootstrap_map, NULL, 0, 0, 0);
      }
```

使用刚刚从 .dynamic 中解析出来的信息，对动态链接器 ld.so 自身执行动态重定位。

这一步修正的对象主要包括：

- ld.so 自己的 GOT
- 某些全局数据引用
- 某些需要运行时修正的地址
- 架构相关的动态重定位项

```c
pwndbg> p/x $rcx
$5 = 0x7ffff7fc8cb8
pwndbg> p/x $rdi
$6 = 0x7ffff7fc8cd0
pwndbg> p/x $rdi - $rcx
$7 = 0x18
```

在这之前，ld.so 虽然已经执行起来了，但它还只是一个“被内核映射进内存但尚未完全修正”的 ELF 对象。

只有完成这一段之后：普通函数调用才真正可靠，对全局变量和 GOT 的访问才真正安全，动态链接器才具备继续执行复杂初始化逻辑的基础

这一段最能体现 _dl_start 的本质作用：利用 bootstrap_map 和 .dynamic 中的信息，让 ld.so 自己完成自重定位。

7. 标记自重定位完成

重定位完成后，函数立刻设置：

```c
/* 标记动态链接器自身已经完成重定位。 */
   bootstrap_map.l_relocated = 1;
/* 到这里，自举最危险的阶段已经结束：
    现在可以正常调用函数、访问全局数据、继续完整初始化流程。*/
```

这表示：

- 动态链接器自身已经完成重定位
- 最危险的 bootstrap 阶段已经结束
- 后续代码可以按“已可用的动态链接器”状态继续运行

在此之前，早期自举阶段，能力受限；之后，动态链接器已进入相对稳定、可继续展开完整初始化的阶段

8. 转入 _dl_start_final

函数最后：

```c
    /*故意把后续逻辑放到 _dl_start_final 里，
    是为了防止编译器在本函数前半段提前生成对 GOT 的访问代码。 */
   #ifdef DONT_USE_BOOTSTRAP_MAP
   return _dl_start_final (arg);
   #else
   return _dl_start_final (arg, &info);
   #endif
```

这表明 _dl_start 自己并不完成整个动态链接器启动过程，而是在完成最危险、最关键的早期自举后，把控制权交给 _dl_start_final。

_dl_start 的职责是“完成早期自举并让 ld.so 变得可用”，真正更完整的初始化流程会在 _dl_start_final 里继续展开

主要工作：

1.把前面 _dl_start 在 bootstrap 阶段计算出来的关于 ld.so 自身的信息，拷贝到动态链接器正式常驻的 link_map 结构 _dl_rtld_map 中。

```
469   /* Transfer data about ourselves to the permanent link_map structure.  */
470 #ifndef DONT_USE_BOOTSTRAP_MAP
471   _dl_rtld_map.l_addr = info->l.l_addr;
472   _dl_rtld_map.l_ld = info->l.l_ld;
473   _dl_rtld_map.l_ld_readonly = info->l.l_ld_readonly;
474   memcpy (_dl_rtld_map.l_info, info->l.l_info, sizeof _dl_rtld_map.l_info);
475   _dl_rtld_map.l_mach = info->l.l_mach;
476   _dl_rtld_map.l_relocated = 1;
477 #endif
```

- l_addr：ld.so 自身运行时装载基址
- l_ld：自身 .dynamic 段地址
- l_ld_readonly：动态段是否只读
- l_info[]：解析 .dynamic 后得到的 DT_* 索引表
- l_mach：机器相关状态
- l_relocated = 1：说明动态链接器自己已经完成重定位

前面的 _dl_start 用的是 bootstrap 阶段的临时对象，而从这里开始，动态链接器以后长期使用的是 _dl_rtld_map。

2.建立哈希访问结构并完善 _dl_rtld_map

```
478   _dl_setup_hash (&_dl_rtld_map);
479   _dl_rtld_map.l_real = &_dl_rtld_map;
480   _dl_rtld_map.l_map_start = (ElfW(Addr)) &__ehdr_start;
481   _dl_rtld_map.l_map_end = (ElfW(Addr)) _end;
```

_dl_setup_hash (&_dl_rtld_map);

- 根据 _dl_rtld_map 中已经存在的动态段信息，建立符号哈希相关状态。
- 也就是让 ld.so 自己作为一个 ELF 对象，具备后续符号查找所需的哈希结构。

_dl_rtld_map.l_real = &_dl_rtld_map;

- 设置“真实对象”指针指向自身。
- 这在 link_map 体系中表示：ld.so 这个 map 本身就是它自己的真实对象。

l_map_start 和 l_map_end

记录 ld.so 在内存中的映射起止范围：

- l_map_start = &__ehdr_start
- l_map_end = _end

这样后续调试、查找对象、异常展开、地址归属判断等机制就知道 ld.so 这个对象在内存里覆盖哪一段地址。

这一步使_dl_rtld_map 不再只是“有几个关键地址”，而是逐渐被补全成一个正式、完整的 link_map 对象

3. 设置栈底信息

```c
__libc_stack_end = __builtin_frame_address (0);
```

初始化 __libc_stack_end。

它记录当前栈的一个边界位置，供 libc / rtld 后续使用。

意义：

- 动态链接器即将进入更完整的系统相关初始化
- 需要先把进程启动时的栈状态记录下来

4.调用 _dl_sysdep_start，进入平台相关启动逻辑

```c
start_addr = _dl_sysdep_start (arg, &dl_main);
```

这个函数时是操作系统/平台相关的启动入口。

在这里会做一些依赖 OS 和 ABI 的准备工作，使动态链接器进入真正可执行“主逻辑”的状态。

```c
//sysdeps/unix/sysv/linux/dl-sysdep.c +100
ElfW(Addr)
_dl_sysdep_start (void **start_argptr,
                  void (*dl_main) (const ElfW(Phdr) *phdr,
                                   ElfW(Word) phnum,
                                   ElfW(Addr) *user_entry,
                                   ElfW(auxv_t) *auxv))
```

- 第一个参数 start_argptr 是内核交给 ld.so 的原始启动栈位置
- 第二个参数 dl_main 是一个函数指针

```c
_dl_sysdep_parse_arguments (start_argptr, &dl_main_args);
...
(*dl_main) (dl_main_args.phdr, dl_main_args.phnum,
            &dl_main_args.user_entry, GLRO(dl_auxv));
return dl_main_args.user_entry;
```

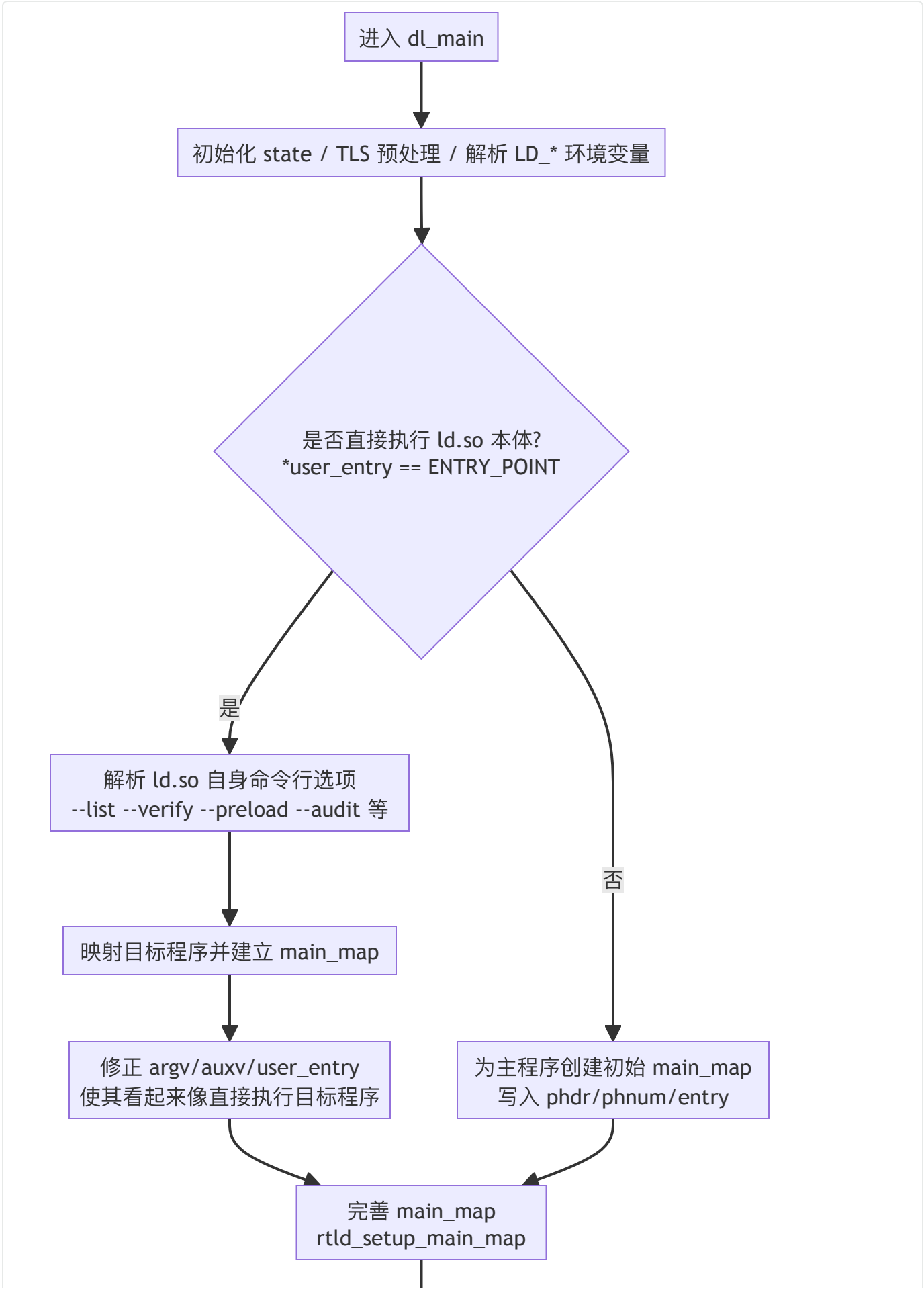
`&dl_main`

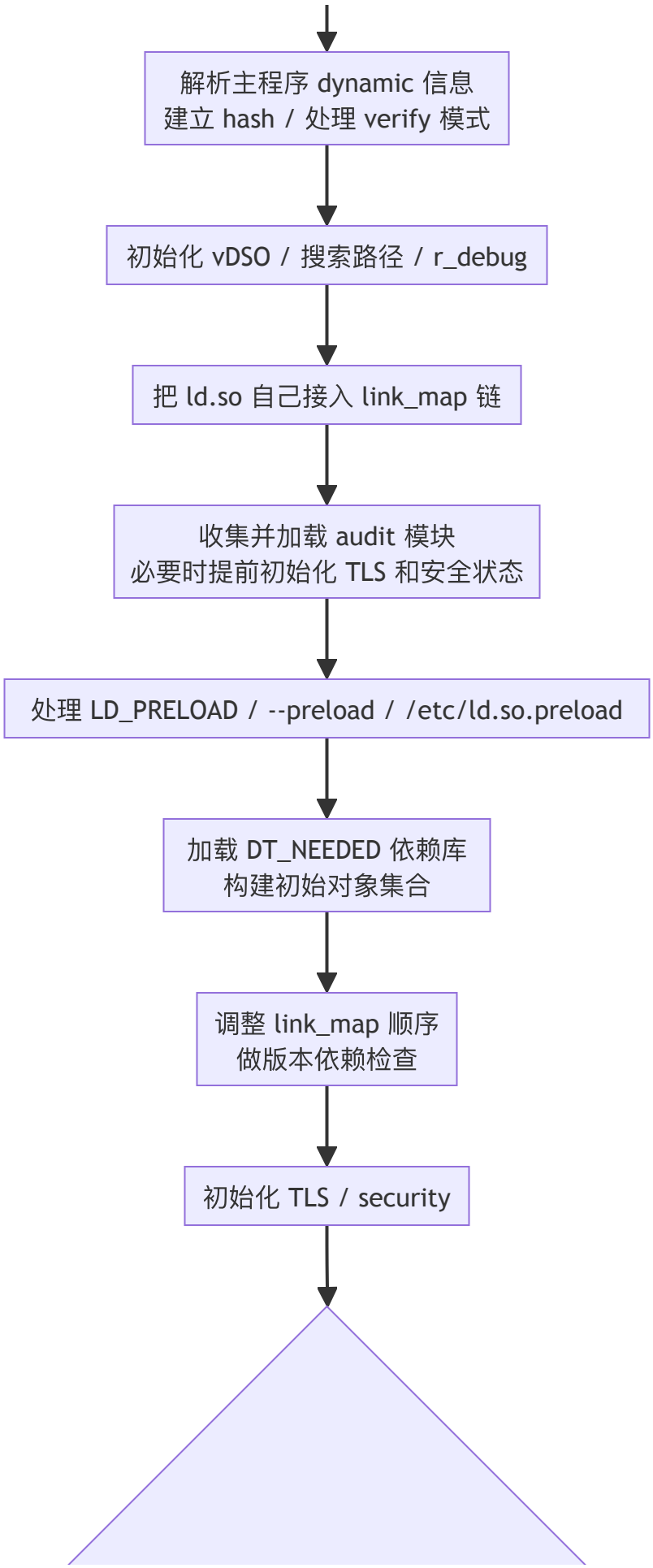
传进去的是 dl_main 的函数指针。也就是说，_dl_sysdep_start 后面会调用 dl_main，而 dl_main 才是动态链接器的主体工作函数，负责：

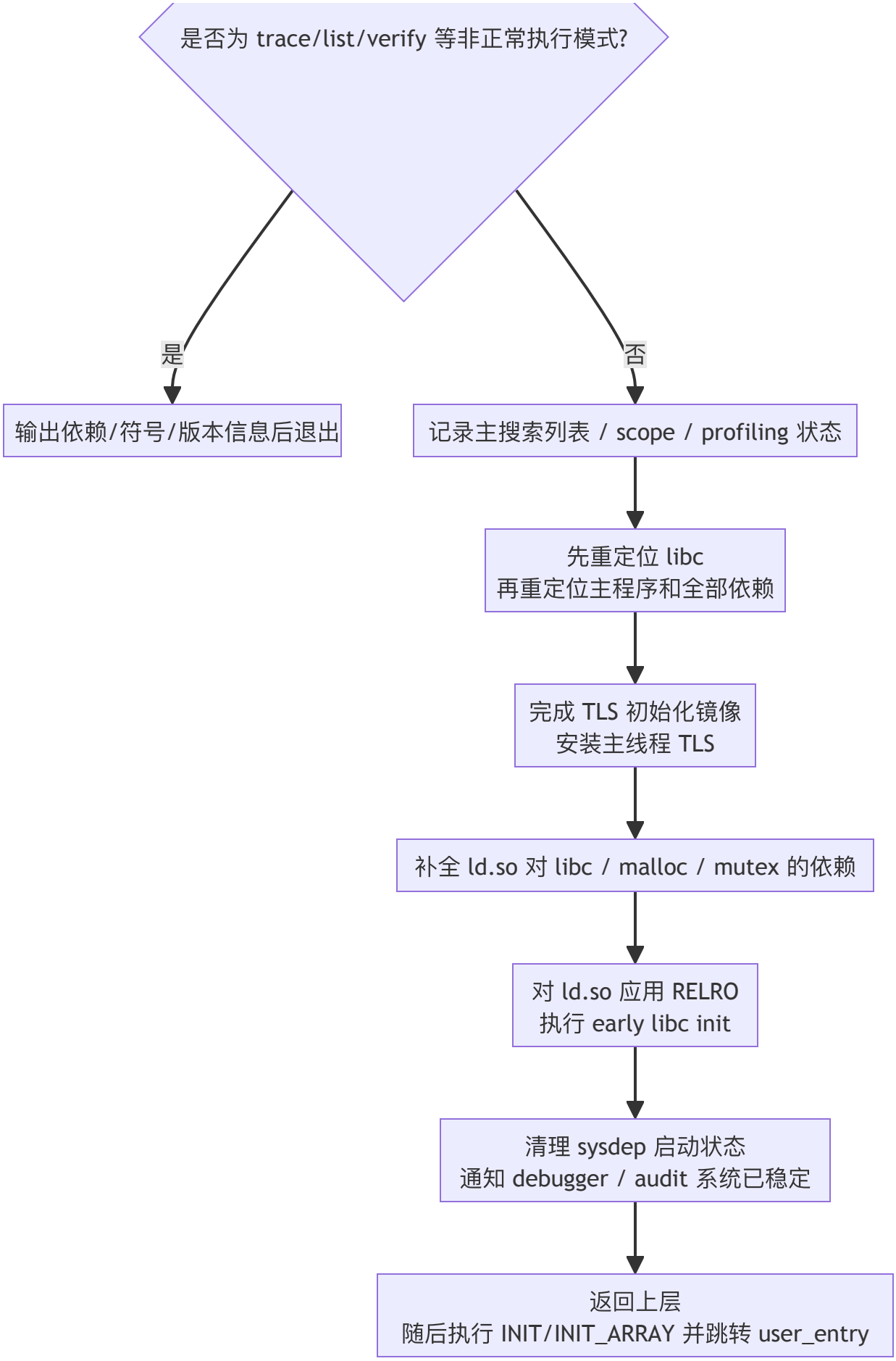
- 解析辅助向量
- 处理环境变量
- 建立命名空间
- 加载主程序依赖
- 做主程序和共享库重定位
- 调用初始化函数
- 最终确定跳转到用户程序入口

返回值 start_addr：返回的是最终要跳转到的程序入口地址。

总结：_dl_start_final 在这里把控制权交给平台相关的启动逻辑，由它驱动 dl_main 完成动态链接器的主体初始化，并最终返回用户程序入口地址。


进入 dl_main
初始化 state / TLS 预处理 / 解析 LD_* 环境变量
是否直接执行 ld.so 本体?
*user_entry == ENTRY_POINT
是
否
解析 ld.so 自身命令行选项
--list --verify --preload --audit 等
映射目标程序并建立 main_map
修正 argv/auxv/user_entry
使其看起来像直接执行目标程序
为主程序创建初始 main_map
写入 phdr/phnum/entry
完善 main_map
rtld_setup_main_map


解析主程序 dynamic 信息
建立 hash / 处理 verify 模式
初始化 vDSO / 搜索路径 / r_debug
把 ld.so 自己接入 link_map 链
收集并加载 audit 模块
必要时提前初始化 TLS 和安全状态
处理 LD_PRELOAD / --preload / /etc/ld.so.preload
加载 DT_NEEDED 依赖库
构建初始对象集合
调整 link_map 顺序
做版本依赖检查
初始化 TLS / security


是否为 trace/list/verify 等非正常执行模式?
是
否
输出依赖/符号/版本信息后退出
记录主搜索列表 / scope / profiling 状态
先重定位 libc
再重定位主程序和全部依赖
完成 TLS 初始化镜像
安装主线程 TLS
补全 ld.so 对 libc / malloc / mutex 的依赖
对 ld.so 应用 RELRO
执行 early libc init
清理 sysdep 启动状态
通知 debugger / audit 系统已稳定
返回上层
随后执行 INIT/INIT_ARRAY 并跳转 user_entry

简单来说：先确定“我要运行谁”，为主程序建立 main_map，把 ld.so 自己挂进 link map 链，然后加载 preload、audit、vDSO、DT_NEEDED 依赖，做版本检查、TLS 初始化、安全初始化，重定位主程序、依赖库、再补齐 ld.so 自己，通知调试器和 audit：系统已稳定，返回上层，准备执行构造函数和用户入口

Q1：为什么要自重定位

在 ELF 动态链接体系里，ld.so 自己通常也是一个 ELF 动态对象。
那就意味着它自己也带有：

GOT、动态重定位项、可能的全局数据引用、可能的函数入口跳转修正需求
如果这些东西还没处理完，就会出现一种悖论：动态链接器要靠自身代码去做重定位，但它自己的代码/数据访问本身又依赖重定位结果

所以 _dl_start 的写法非常克制，目的就是在最少依赖条件下先完成“自救”。

那它是如何做到“可以自重定位”的

- 先算出 ld.so 的运行时基址
- 用基址找到自己的 .dynamic
- 从 .dynamic 里找到重定位表、符号表、字符串表
- 用这些表去修正自己内部还没修好的地址
- 修完后，ld.so 才变成“真正可运行”的动态链接器

Q2：bootstrap_map 是什么

在 glibc 里，运行时装载的 ELF 对象通常都用 link_map 表示。这个结构会记录一个共享对象的关键信息，例如：

- 该对象的运行时装载基址 l_addr
- 动态段地址 l_ld
- 解析后的 DT_* 条目表 l_info[]
- TLS 信息
- 重定位状态
- 与其他对象的链接关系

而 bootstrap_map 本质上就是一个专门给 ld.so 自己在最早期启动时使用的 link_map 实例。原因很直接：

此时用户程序还没开始执行

- 正常的 link map 链表体系还没完全建立
- ld.so 自己也还没完成重定位
- 但它又必须先知道“我自己是谁、装到哪、动态段在哪、有哪些重定位要做”

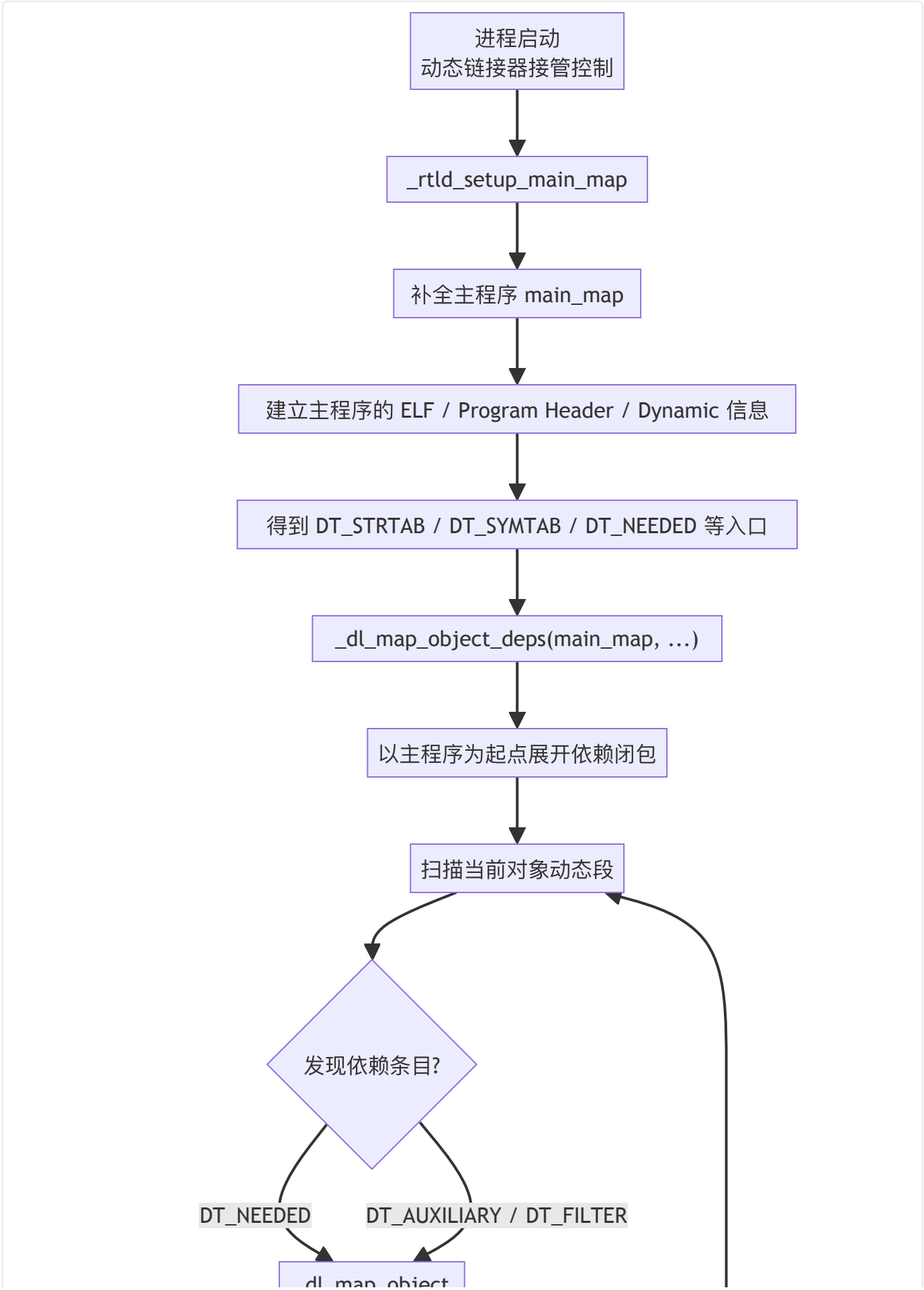
所以需要一个静态可用、无需依赖外部初始化的最小对象描述结构，这就是 bootstrap_map。

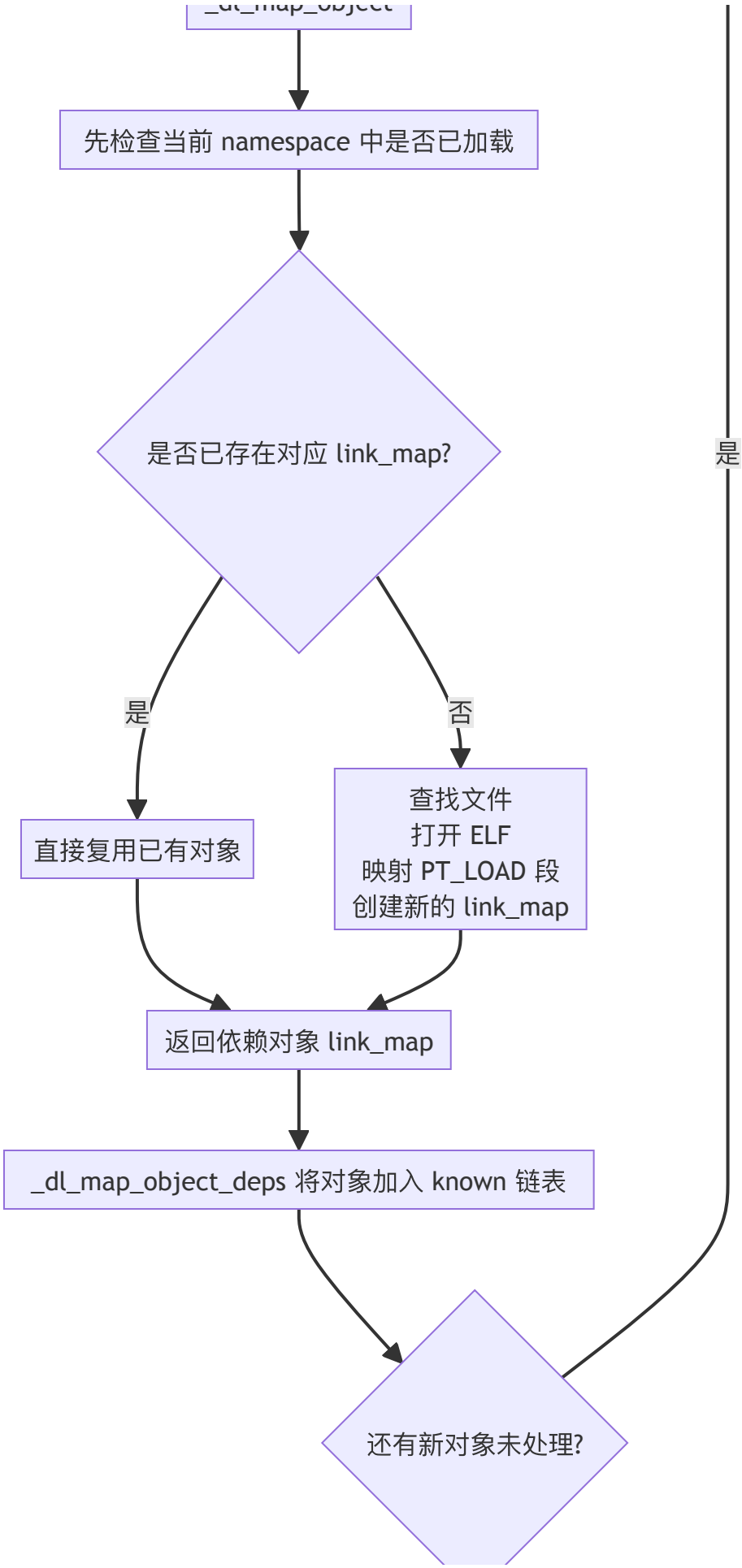
可以把它理解成：

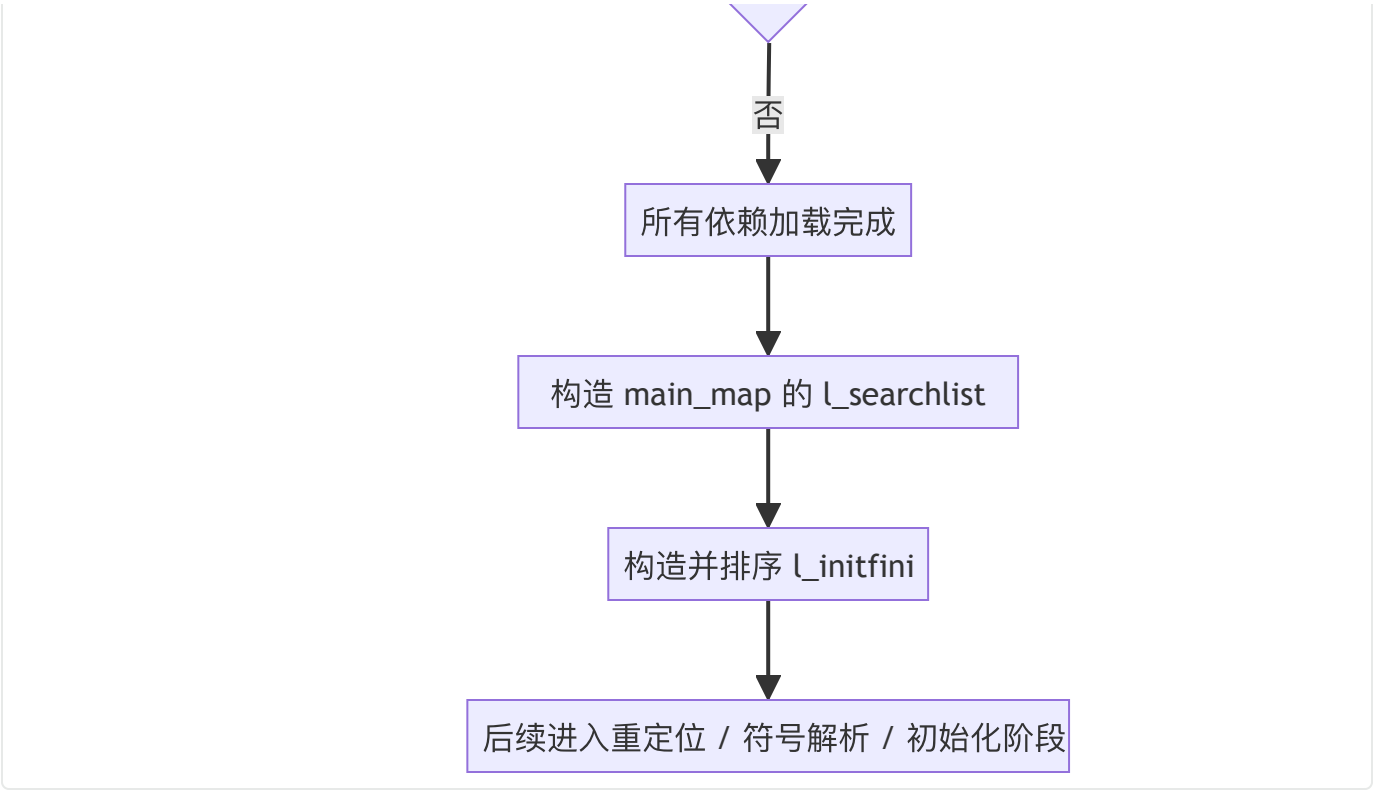
正常阶段：每个 ELF 共享对象都有自己的 link_map
自举阶段：ld.so 先用 bootstrap_map 临时描述“我自己”
等自重定位完成后，才逐步进入完整运行时链接器状态

2.依赖库装载阶段

`_rtld_setup_main_map` 先把主程序自己的 `link_map` 补完整，`_dl_map_object` 负责装载一个具体的共享对象，而 `_dl_map_object_deps` 则以主程序为起点反复调用 `_dl_map_object`，把整个依赖图展开并最终构造 `searchlist`。


进程启动
动态链接器接管控制
_rtld_setup_main_map
补全主程序 main_map
建立主程序的 ELF / Program Header / Dynamic 信息
得到 DT_STRTAB / DT_SYMTAB / DT_NEEDED 等入口
_dl_map_object_deps(main_map, ...)
以主程序为起点展开依赖闭包
扫描当前对象动态段
发现依赖条目?
DT_NEEDED
DT_AUXILIARY / DT_FILTER


先检查当前 namespace 中是否已加载
是否已存在对应 link_map?
是
否
直接复用已有对象
查找文件
打开 ELF
映射 PT_LOAD 段
创建新的 link_map
返回依赖对象 link_map
_dl_map_object_deps 将对象加入 known 链表
还有新对象未处理?
是



DT_STRTAB 提供字符串池基址、DT_SYMTAB 提供符号表基址、DT_NEEDED 提供依赖库名字在字符串表中的偏移

拿到 DT_STRTAB 后，动态链接器就能解释各种字符串名字、拿到 DT_SYMTAB 后，动态链接器就能访问动态符号表、拿到 DT_NEEDED 后，动态链接器就能知道还要继续装载哪些共享库

1.DT_NEEDED

表示“普通依赖库”。当前对象运行时必须依赖这个共享库，动态链接器需要把它装入，并把它纳入当前对象的依赖图中。在 .dynamic 里，DT_NEEDED 的值不是直接的字符串，而是一个偏移，指向动态字符串表里的库名，比如 libc.so.6。这也是最常见的一类依赖，通常来自链接阶段的 -lxxx。

在 _dl_map_object_deps 里，遇到 DT_NEEDED 时会：

- 取出库名
- 调用 _dl_map_object 去加载或复用该库
- 如果是第一次遇到，就把它追加到 known 链表尾部
- 所以 DT_NEEDED 对应的是“标准依赖边”，在流程上表现为广度优先扩展依赖图。

2.DT_AUXILIARY

表示“辅助过滤对象”或“辅助提供者”。它不是普通的硬依赖，而更像一种“可选的、优先参与符号解析的辅助库”。它的设计目的可以理解成：当前对象希望借助另一个对象来提供符号，或者在符号查找时优先考虑那个对象。但如果这个辅助对象不存在，通常可以继续运行。

这也是为什么在 _dl_map_object_deps 里：

- DT_AUXILIARY 加载失败时不会立刻报错退出
- 成功加载后，它不会像 DT_NEEDED 一样简单尾插
- 而是要插到当前对象之前，保证它在符号搜索顺序中更早被看到

所以它的关键特征是两点：

- 可选性更强
- 符号搜索优先级更高

3.DT_FILTER

表示“过滤对象”。可以把它理解成一种更强语义的“代理/转发关系”：当前对象本身像一个过滤层，真正的符号实现应该从另一个对象里找。

和 DT_AUXILIARY 相比，DT_FILTER 更强、更严格。它不是“有最好，没有也行”，而是“这个过滤目标必须存在”，否则对象语义就不成立。

因此在 _dl_map_object_deps 里：

- 遇到 DT_FILTER 也会去加载对应对象
- 也会把它放到更早的搜索位置
- 但如果加载失败，这是致命错误，函数会报错退出

所以 DT_FILTER 的特点是：

- 它也影响符号搜索顺序
- 但它是强约束，不允许缺失

总结

DT_NEEDED 表示普通必需依赖，按正常依赖关系展开；DT_AUXILIARY 和 DT_FILTER 都要求目标对象在符号查找中先于当前对象出现，其中 DT_AUXILIARY 通常是可选的，而 DT_FILTER 是强约束、缺失即报错。

---

l_searchlist：用于符号查找顺序

l_initfini：用于 初始化/析构的调用顺序

Q: 为什么还要再构造一份 map->l_initfini?

因为初始化/析构顺序不能简单等同于符号搜索顺序。

符号搜索更关心“谁先可见”；初始化关心“谁依赖谁，谁应该先初始化，谁应该后初始化”。

2.1 主程序 link_map 补全阶段(_rtld_setup_main_map)

在动态链接器接管控制权之前，内核其实已经完成了主程序的第一轮 ELF 解析。它会先读取主程序的 ELF Header 和 Program Header Table，按照其中的 PT_LOAD 描述把各个可装载段映射到进程地址空间；如果看到 PT_INTERP，还会继续把解释器，也就是动态链接器本身装入内存。

随后，内核通过初始栈把一组辅助向量 auxv 传给动态链接器，其中 AT_PHDR、AT_PHNUM、AT_PHENT 直接给出了主程序 Program Header Table 在内存中的地址、项数和项大小。因此，动态链接器后续处理主程序时，并不需要重新打开可执行文件去读取 Program Header，而是直接遍历内核已经准备好的那张表。

在此基础上，`rtld_setup_main_map` 做的工作，本质上就是把“内核已经映射好的主程序”转换成“动态链接器内部可操作的主程序对象”。它扫描 main_map->l_phdr 指向的 Program Header 数组：

- 通过 PT_PHDR 计算主程序装载基址 l_addr。
- 通过 PT_DYNAMIC 找到 .dynamic 段地址 l_ld。
- 通过所有 PT_LOAD 统计主程序整体映射起止范围，并判断各段是否连续。
- 通过 PT_TLS 提取 TLS 模板信息，建立主程序 TLS 模块。
- 通过 PT_GNU_STACK 记录栈权限要求。
- 通过 PT_GNU_RELRO 记录 RELRO 区域，为后续只读保护做准备。
- 通过 PT_INTERP(如果存在) 把解释器路径与动态链接器自己的 link_map 关联起来，避免后续重复加载 ld.so。

- 通过 PT_NOTE 和 PT_GNU_PROPERTY 补充解析额外属性信息。

经过这一步，主程序才真正从“一个已经在内存里的 ELF 映像”变成“动态链接器可继续做依赖解析和重定位的 link_map 实例”。这是后面依赖装载和重定位的前置条件。

在函数 `rtld_setup_main_map` 中，处理主程序的各个segment的加载处理：

```c
/* 完成主可执行文件 link_map 的初始化。
   如果程序头中存在 PT_INTERP，则返回 true。 */
static bool
rtld_setup_main_map (struct link_map *main_map)
{
  /* 这些字段在 _dl_new_object 之后，或 _dl_map_object 中，
     已经提前填好了。这里直接取出主程序的 Program Header 表。 */
  const ElfW(Phdr) *phdr = main_map->l_phdr;
  ElfW(Word) phnum = main_map->l_phnum;

  /* 标记主程序是否带有解释器段 PT_INTERP。 */
  bool has_interp = false;

  /* 初始化主程序整体映射结束地址，后面扫描 PT_LOAD 时取最大值。 */
  main_map->l_map_end = 0;

  /* 初始化主程序整体映射起始地址，后面扫描 PT_LOAD 时取最小值。 */
  main_map->l_map_start = ~0;

  /* 主程序是被直接打开/直接引用的对象，因此直接引用计数加 1。 */
  ++main_map->l_direct_opencount;

  /* 先乐观地认为所有 PT_LOAD 段在内存中连续。 */
  main_map->l_contiguous = 1;

  /* 记录下一个 PT_LOAD 段“理论上应该开始”的地址。
     如果后续实际地址不匹配，则说明段之间不连续。 */
  ElfW(Addr) expected_load_address = 0;

  /* 第一次扫描 Program Header，提取主程序的关键信息。 */
  for (const ElfW(Phdr) *ph = phdr; ph < &phdr[phnum]; ++ph)
    switch (ph->p_type)
      {
      case PT_PHDR:
        /* 通过“程序头表在内存中的实际地址 - ELF 中记录的虚拟地址”
           计算主程序运行时装载基址。 */
        main_map->l_addr = (ElfW(Addr)) phdr - ph->p_vaddr;
        break;

      case PT_DYNAMIC:
        /* 定位主程序的 .dynamic 段，它包含动态链接所需的大量元信息。 */
        main_map->l_ld = (void *) main_map->l_addr + ph->p_vaddr;

        /* 记录 .dynamic 段是否位于只读区域。 */
        main_map->l_ld_readonly = (ph->p_flags & PF_W) == 0;
```

```
46          break;
47
48
49        case PT_INTERP:
50          /* PT_INTERP 段里保存的是程序解释器路径，也就是动态链接器路径。
51             把这个名字登记到动态链接器自己的 link_map 中，避免未来通过
52             dlopen 或 DT_NEEDED 再次把动态链接器自己重复加载。 */
53          _dl_rtld_libname.name = ((const char *) main_map->l_addr
54                                   + ph->p_vaddr);
55
56          /* next 默认为 NULL，这里不需要再赋值。 */
57          _dl_rtld_map.l_libname = &_dl_rtld_libname;
58
59          has_interp = true;
60          break;
61
62        case PT_LOAD:
63          {
64            ElfW(Addr) mapstart;
65            ElfW(Addr) allocend;
66
67            /* 计算该加载段按页向下对齐后的实际起始地址。 */
68            mapstart = (main_map->l_addr
69                        + (ph->p_vaddr & ~(GLRO(dl_pagesize) - 1)));
70
71            /* 维护整个主程序映射区间的最小起始地址。 */
72            if (main_map->l_map_start > mapstart)
73              main_map->l_map_start = mapstart;
74
75            /* 如果前一个段推导出的“期望起始地址”与当前段实际地址不同，
76               则说明各 PT_LOAD 段并非连续排列。 */
77            if (main_map->l_contiguous && expected_load_address != 0
78                && expected_load_address != mapstart)
79              main_map->l_contiguous = 0;
80
81            /* 计算该段在内存中的实际结束地址。使用 p_memsz，
82               因为段在内存中的大小可能大于文件中的大小。 */
83            allocend = main_map->l_addr + ph->p_vaddr + ph->p_memsz;
84
85            /* 维护整个主程序映射区间的最大结束地址。 */
86            if (main_map->l_map_end < allocend)
87              main_map->l_map_end = allocend;
88
89            /* 推导出“下一段如果连续，理论上应该开始”的页对齐地址。 */
90            expected_load_address = ((allocend + GLRO(dl_pagesize) - 1)
91                                     & ~(GLRO(dl_pagesize) - 1));
92          }
93          break;
```

```
      case PT_TLS:
        /* 如果主程序定义了 TLS（线程局部存储）模板，则提取相关信息。 */
        if (ph->p_memsz > 0)
          {
            /* TLS 内存块总大小。 */
            main_map->l_tls_blocksize = ph->p_memsz;

            /* TLS 对齐要求。 */
            main_map->l_tls_align = ph->p_align;

            /* 计算 TLS 首字节相对对齐边界的偏移。 */
            if (ph->p_align == 0)
              main_map->l_tls_firstbyte_offset = 0;
            else
              main_map->l_tls_firstbyte_offset = (ph->p_vaddr
                                                  & (ph->p_align - 1));

            /* TLS 初始化镜像在文件中的大小。 */
            main_map->l_tls_initimage_size = ph->p_filesz;

            /* 暂时记录为 ELF 虚拟地址，后面再加上 l_addr 修正。 */
            main_map->l_tls_initimage = (void *) ph->p_vaddr;

            /* 主程序 TLS 模块固定分配模块 ID 1。 */
            GL(dl_tls_max_dtv_idx) = main_map->l_tls_modid = 1;
          }
        break;

      case PT_GNU_STACK:
        /* 记录主程序要求的栈权限属性，例如是否可执行。 */
        GL(dl_stack_flags) = ph->p_flags;
        break;

      case PT_GNU_RELRO:
        /* 记录 RELRO 区域的起始虚拟地址和大小。
           重定位完成后，这块区域会被改回只读。 */
        main_map->l_relro_addr = ph->p_vaddr;
        main_map->l_relro_size = ph->p_memsz;
        break;
      }

  /* 第二次扫描 Program Header，这次倒序处理。
     这样如果同时存在 PT_GNU_PROPERTY 和 PT_NOTE，
     就可以先处理更专用的 GNU property。 */
  for (const ElfW(Phdr) *ph = &phdr[phnum]; ph != phdr; --ph)
    switch (ph[-1].p_type)
      {
      case PT_NOTE:
```

```
        _dl_process_pt_note (main_map, -1, &ph[-1]);
        break;

      case PT_GNU_PROPERTY:
        _dl_process_pt_gnu_property (main_map, -1, &ph[-1]);
        break;
      }

  /* 如果主程序有 TLS 初始化镜像，把它从 ELF 虚拟地址修正成运行时地址。
     这在主程序是 PIE（ET_DYN）时尤其重要。 */
  if (main_map->l_tls_initimage != NULL)
    main_map->l_tls_initimage
      = (char *) main_map->l_tls_initimage + main_map->l_addr;

  /* 如果根本没有 PT_LOAD，给 l_map_end 一个兜底值。 */
  if (! main_map->l_map_end)
    main_map->l_map_end = ~0;

  /* 如果前面没有通过 PT_INTERP 给动态链接器自己登记名字，
     但动态链接器本身已有名字，则用那个名字补上。 */
  if (! _dl_rtld_map.l_libname && _dl_rtld_map.l_name)
    {
      /* 这是动态链接器被直接执行时的兜底路径。 */
      _dl_rtld_libname.name = _dl_rtld_map.l_name;
      _dl_rtld_map.l_libname = &_dl_rtld_libname;
    }
  else
    /* 正常情况下，走到这里时动态链接器自己应当已经有名字了。 */
    assert (_dl_rtld_map.l_libname);

  /* 返回主程序是否存在 PT_INTERP。 */
  return has_interp;
}
```

2.2 dl_map_object

依赖库装载阶段的核心入口是 `dl_map_object` 。*在这之前，* `rtld_setup_main_map` 已经把主程序对应的 main_map 补全，包括基址、程序头、动态段以及 DT* 信息。只有主程序的 link_map 被正确建立后，动态链接器才能继续读取主程序动态段中的 DT_NEEDED，并开始装载依赖的共享库。

`map_doit` 本身只是一个参数封装层，它根据当前加载模式把目标识别为可执行文件或共享库，然后调用 `_dl_map_object` 。真正的装载逻辑发生在 `_dl_map_object` 中。

```c
static void
map_doit (void *a)
{
  /* 还原参数结构。 */
  struct map_args *args = (struct map_args *) a;

  /* 根据加载模式判断目标是“可执行文件”还是“共享库”。 */
  int type = (args->mode == __RTLD_OPENEXEC) ? lt_executable : lt_library;

  /* 将名字对应的 ELF 对象映射进内存，并把结果保存到 args->map。 */
  args->map = _dl_map_object (args->loader, args->str, type, 0,
                              args->mode, LM_ID_BASE);
}
```

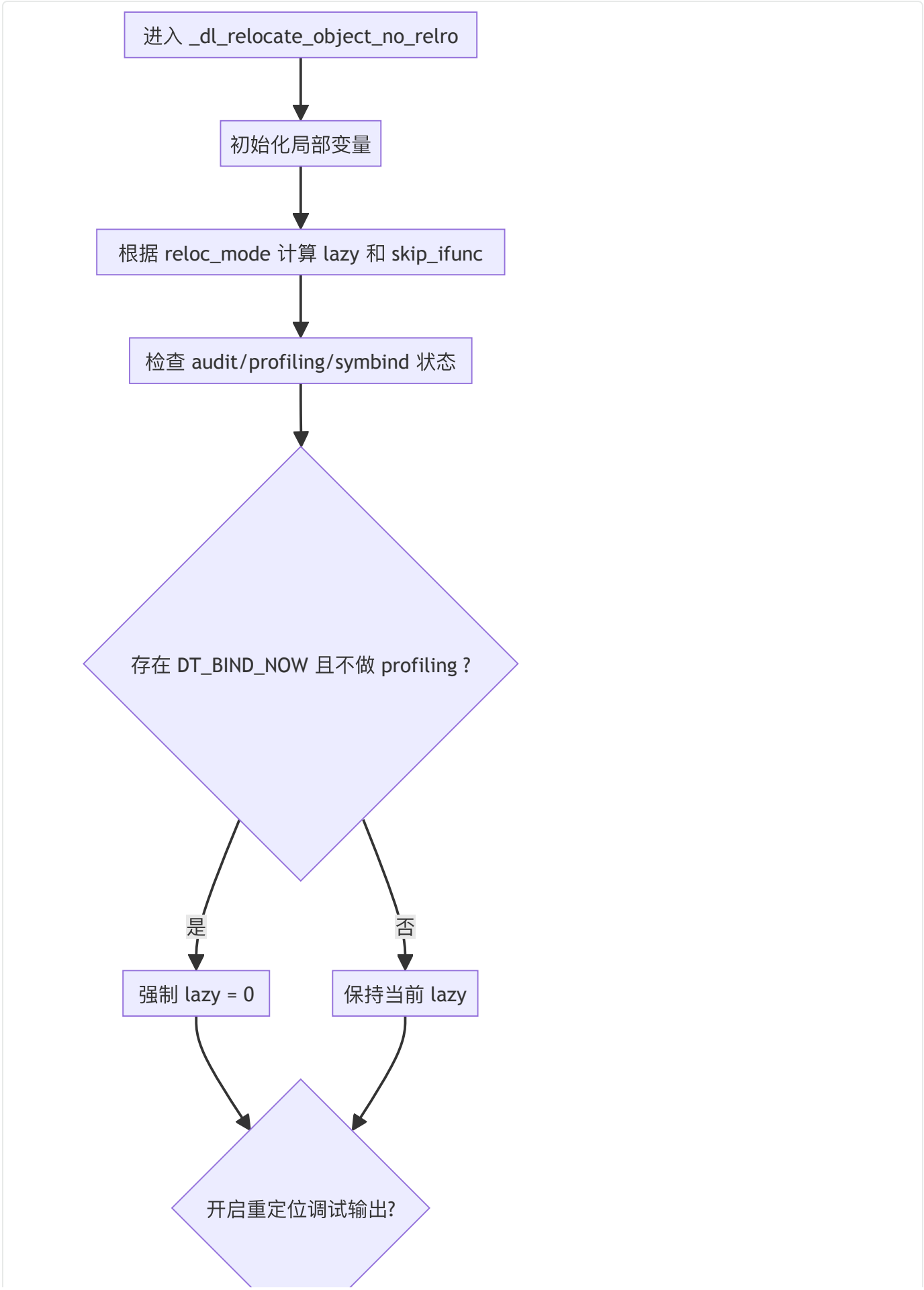
`_dl_map_object` 的第一步不是立刻打开文件，而是先在当前 namespace 的 `_ns_loaded` 链表中检查该对象是否已经加载。检查时不仅会比对现有名字列表，还会在必要时读取对象动态段中的 `DT_SONAME` 做匹配。如果找到匹配对象，就直接返回已有的 `link_map`，避免重复加载。同一个共享库即使通过不同路径名或库名被引用，只要最终识别为同一对象，都不会再次映射。

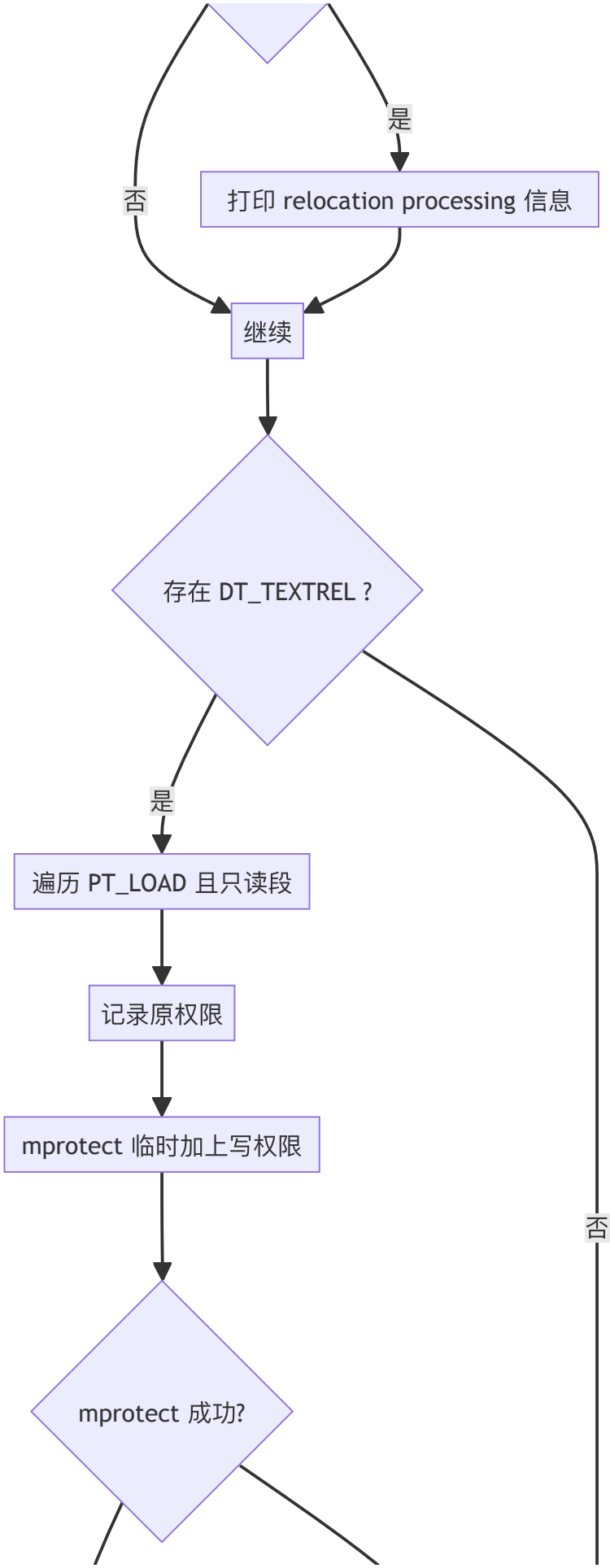
如果对象尚未加载，`_dl_map_object` 才会继续执行后续动作，包括查找库文件、打开 ELF、读取程序头、映射 PT_LOAD 段、创建新的 link_map，并把新对象挂入当前 namespace 的已加载对象链表中。

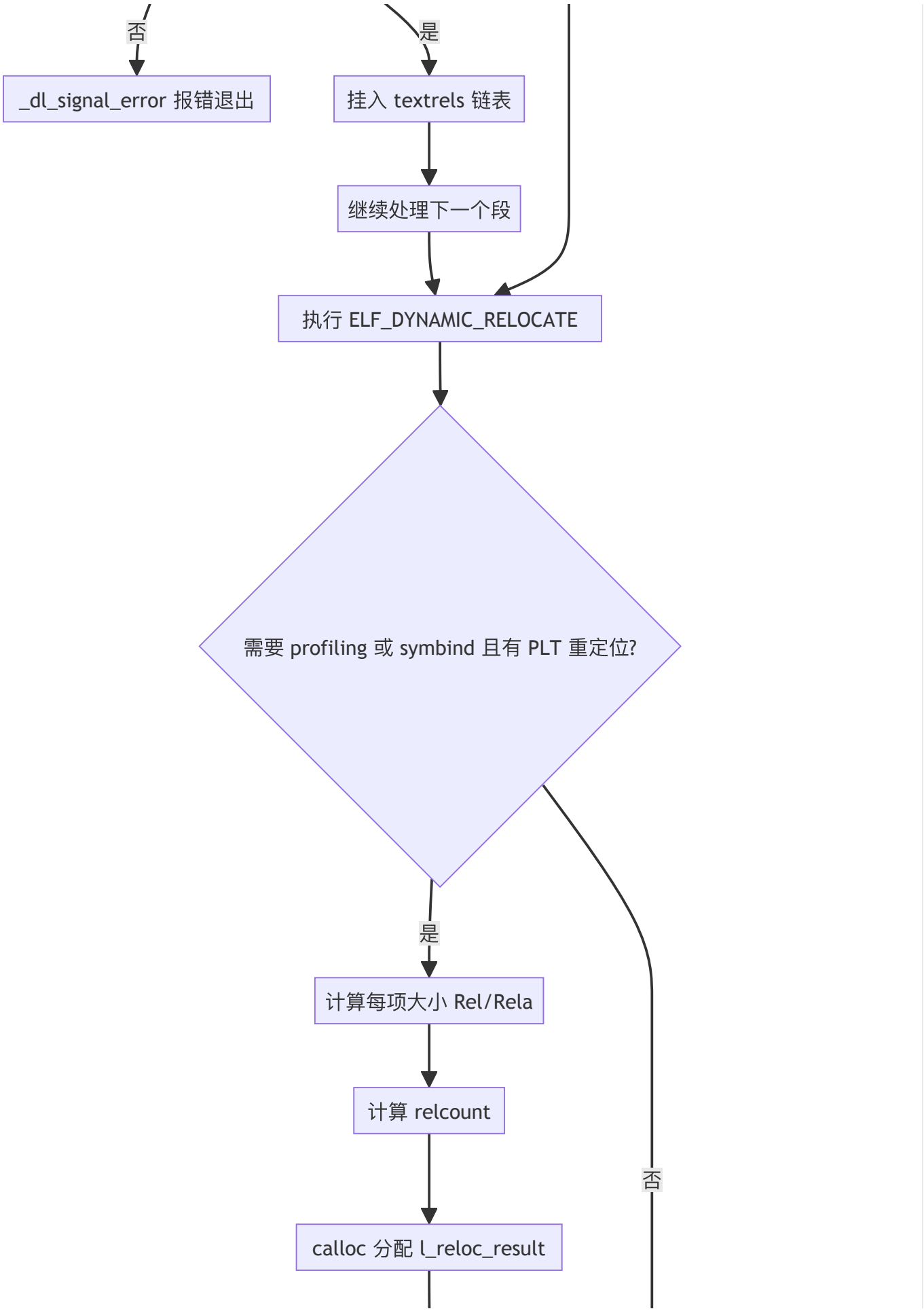
这里和主程序的 map 过程不同。主程序在 execve 时通常已经由内核完成基本映射，而共享库是进程运行期间由动态链接器主动装入的，因此必须显式把它的 LOAD segment 映射到内存中。因为动态段、字符串表、符号表、哈希表、重定位表等信息都位于这些被映射的段里，只有完成映射之后，后续的动态信息访问、符号解析和重定位才能进行。

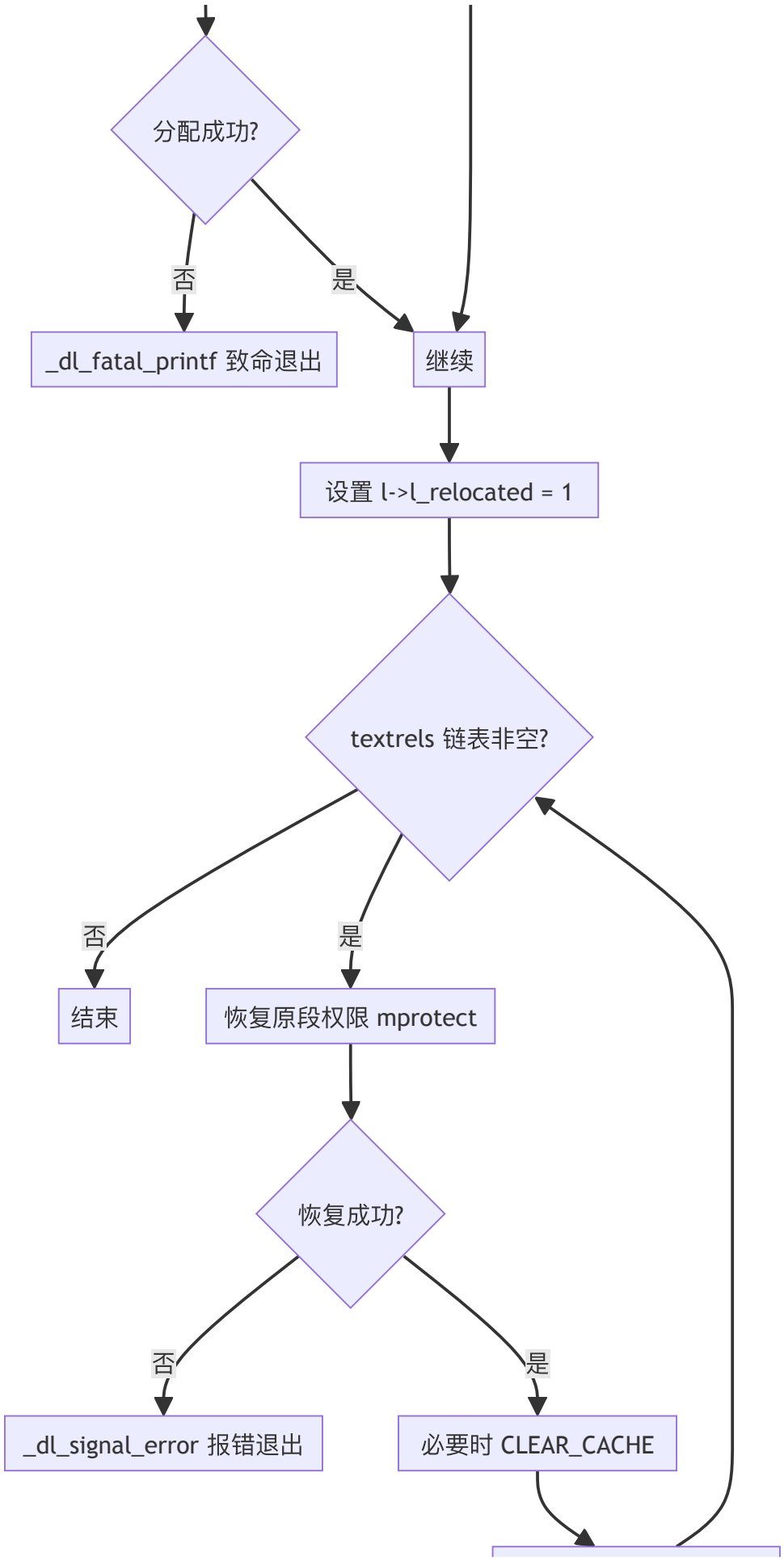
`_dl_map_object_deps` 则在 `_dl_map_object` 的基础上继续展开依赖图。它会遍历当前对象的 `DT_NEEDED` 项，把每个依赖库交给 `_dl_map_object` 处理；而新加载进来的共享库又可能继续依赖别的库，因此这个过程会不断迭代，直到再也没有新的依赖需要装入为止。最终，运行时链接器得到的是一组完整的 `link_map` 结构，每个已加载对象对应一个 `link_map`，它们被挂接到所属 namespace 的已加载链表和后续符号查找所需的搜索结构中，从而形成完整的运行时共享对象图。

```
23 /* The MODE argument to `dlopen' contains one of the following: */
24 #define RTLD_LAZY       0x00001 /* Lazy function call binding.  */
25 #define RTLD_NOW        0x00002 /* Immediate function call binding.
26 #define RTLD_BINDING_MASK   0x3 /* Mask of binding time value.  */
27 #define RTLD_NOLOAD     0x00004 /* Do not load the object.  */
28 #define RTLD_DEEPBIND   0x00008 /* Use deep binding.  */
```


进入 _dl_relocate_object_no_relro
初始化局部变量
根据 reloc_mode 计算 lazy 和 skip_ifunc
检查 audit/profiling/symbind 状态
存在 DT_BIND_NOW 且不做 profiling ?
是
否
强制 lazy = 0
保持当前 lazy
开启重定位调试输出?


是
否
打印 relocation processing 信息
继续
存在 DT_TEXTREL ?
是
否
遍历 PT_LOAD 且只读段
记录原权限
mprotect 临时加上写权限
mprotect 成功?


否
是
_dl_signal_error 报错退出
挂入 textrels 链表
继续处理下一个段
执行 ELF_DYNAMIC_RELOCATE
需要 profiling 或 symbind 且有 PLT 重定位?
是
否
计算每项大小 Rel/Rela
计算 relcount
calloc 分配 l_reloc_result


分配成功?
否
是
_dl_fatal_printf 致命退出
继续
设置 l->l_relocated = 1
textrels 链表非空?
否
是
结束
恢复原段权限 mprotect
恢复成功?
否
是
_dl_signal_error 报错退出
必要时 CLEAR_CACHE

处理下一个 textrel 节点

3.重定位 _dl_relocate_object

在 ELF 动态链接中：

- 可执行文件 / 共享库在编译时并不知道最终加载地址
- 所有外部符号（函数/变量）需要在运行时再“补上真实地址”

这个过程就叫 Relocation（重定位）

典型例子：

```
printf("hello");
```

编译后：

- 并不会直接写死 printf 地址
- 只会生成一个 重定位项 + GOT/PLT 入口

运行时由动态链接器 _ld.so 完成修复

函数入口：

```
void
_dl_relocate_object (struct link_map *l, struct r_scope_elem *scope[],
                     int reloc_mode, int consider_profiling)
{
  /* 如果该对象已经完成过重定位，就不再重复处理。 */
  if (l->l_relocated)
    return;

  /* 先执行真正的重定位逻辑，但此时还不恢复 RELRO 保护。 */
  _dl_relocate_object_no_relro (l, scope, reloc_mode, consider_profiling);

  /* 重定位完成后，把 RELRO 区域重新设为只读。 */
  _dl_protect_relro (l);
}

```

3.1关键控制参数：

```
1   void
2   _dl_relocate_object_no_relro (struct link_map *l, struct r_scope_elem *sco
    pe[],
3                                 int reloc_mode, int consider_profiling)
4   {
5     /* 记录那些因 DT_TEXTREL 而被临时改成可写的只读段，
6        后面重定位完成后要恢复其原始保护属性。 */
7     struct textrels
8     {
9       caddr_t start;          /* 段起始地址 */
10      size_t len;             /* 段长度 */
11      int prot;               /* 原始内存保护标志 */
12      struct textrels *next;  /* 链表指针 */
13    } *textrels = NULL;
14
15    /* 初始化错误字符串，避免编译器警告。 */
16    const char *errstring = NULL;
17
18    /* 是否启用懒绑定。 */
19    int lazy = reloc_mode & RTLD_LAZY;
20
21    /* 是否跳过 IFUNC 重定位。 */
22    int skip_ifunc = reloc_mode & __RTLD_NOIFUNC;
23
24    /* 是否需要考虑符号绑定审计。 */
25    bool consider_symbind = false;
26
27  #ifdef SHARED
28    /* 如果当前不是 audit 模式，则检查审计接口是否要求：
29       1. 对 PLT 调用做 profiling
30       2. 处理符号绑定回调 */
31    if ((reloc_mode & __RTLD_AUDIT) == 0)
32      {
33        struct audit_ifaces *afct = GLRO(dl_audit);
34        for (unsigned int cnt = 0; cnt < GLRO(dl_naudit); ++cnt)
35          {
36            /* 如果定义了 PLT 进入/退出钩子，则必须开启 profiling 相关支持。 */
37            if (afct->ARCH_LA_PLTENTER != NULL
38                || afct->ARCH_LA_PLTEXIT != NULL)
39              consider_profiling = 1;
40
41            /* 如果定义了 symbind 钩子，则记录下来。 */
42            if (afct->symbind != NULL)
43              consider_symbind = true;
44
45            afct = afct->next;
46          }
```

```
    }
#elif defined PROF
  /* gprof 模式下不使用动态链接器自己的 profiling。 */
  consider_profiling = 0;
#endif

```

参数	作用
lazy	是否启用懒绑定（PLT 延迟解析）
skip_ifunc	是否跳过 IFUNC
consider_profiling	是否启用 profiling
consider_symbind	是否需要 audit

IFUNC = Indirect Function（间接函数）：运行时根据 CPU 能力选最优实现。启动时决定“用哪个函数实现”（性能优化黑科技）

profiling = 函数调用统计/性能分析机制，运行时记录“函数怎么被调用”（性能分析工具）

audit:动态链接器（ld.so）的一种“审计/插桩”机制，用来跟踪和干预动态库加载及符号解析过程.

3.2两种绑定方式

```
/* 如果对象带有 DT_BIND_NOW，并且当前又不是 profiling，
    则强制关闭 lazy，改为立即绑定。 */
 if (!consider_profiling
     && __builtin_expect (l->l_info[DT_BIND_NOW] != NULL, 0))
   lazy = 0;
```

Lazy（默认）

- PLT 第一次调用才解析
- 启动快

Now（DT_BIND_NOW）

- 启动时全部解析
- 性能稳定、安全性更高（避免 GOT hijack：篡改 GOT（Global Offset Table）中的函数地址，让程序跳到攻击者指定的位置执行。）

3.3 判断代码段（text）是否有需要修改的重定位

代码段通常是 `R-X`（只读 + 可执行），但这里必须：`mprotect(..., PROT_WRITE)`，临时变成：`RWX`

比较危险，不推荐

实现步骤：

- 找所有只读 LOAD 段：`if (ph->p_type == PT_LOAD && !(ph->p_flags & PF_W))`
- 记录原始权限
- 临时改写权限
- 用链表保存

```
  /* 若开启了重定位调试输出，则打印当前对象的重定位信息。 */
  if (__glibc_unlikely (GLRO(dl_debug_mask) & DL_DEBUG_RELOC))
    _dl_debug_printf ("\nrelocation processing: %s%s\n",
                      DSO_FILENAME (l->l_name), lazy ? " (lazy)" : "");

  /* 如果存在 DT_TEXTREL，说明只读段中也需要被修补重定位。
     因此必须先把相关只读 LOAD 段临时改成可写。 */
  if (__glibc_unlikely (l->l_info[DT_TEXTREL] != NULL))
    {
      const ElfW(Phdr) *ph;
      for (ph = l->l_phdr; ph < &l->l_phdr[l->l_phnum]; ++ph)
        if (ph->p_type == PT_LOAD && (ph->p_flags & PF_W) == 0)
          {
            struct textrels *newp;

            /* 在栈上分配一个链表节点，记录该只读段的信息。 */
            newp = (struct textrels *) alloca (sizeof (*newp));

            /* 计算页对齐后的段长度。 */
            newp->len = ALIGN_UP (ph->p_vaddr + ph->p_memsz, GLRO(dl_pagesize))
                        - ALIGN_DOWN (ph->p_vaddr, GLRO(dl_pagesize));

            /* 计算该段在进程地址空间中的起始地址。 */
            newp->start = PTR_ALIGN_DOWN (ph->p_vaddr, GLRO(dl_pagesize))
                          + (caddr_t) l->l_addr;

            /* 保存原始权限。 */
            newp->prot = 0;
            if (ph->p_flags & PF_R)
              newp->prot |= PROT_READ;
            if (ph->p_flags & PF_W)
              newp->prot |= PROT_WRITE;
            if (ph->p_flags & PF_X)
              newp->prot |= PROT_EXEC;

            /* 临时增加写权限，方便执行重定位修补。 */
            if (__mprotect (newp->start, newp->len, newp->prot | PROT_WRITE) < 0)
              {
                errstring = N_("cannot make segment writable for relocation");
              call_error:
                _dl_signal_error (errno, l->l_name, NULL, errstring);
              }

            /* 头插法挂到链表里，后面统一恢复权限。 */
```

```
45                newp->next = textrels;
46                textrels = newp;
47              }
48          }
49
50
```

3.4 核心：真正的重定位执行

```
1  {
2      /* 真正执行对象的动态重定位：GOT、数据引用、PLT 等都在这里处理。 */
3      ELF_DYNAMIC_RELOCATE (l, scope, lazy, consider_profiling, skip_ifunc);
4  ......
5  }
```

展开：

```c
# define ELF_DYNAMIC_RELOCATE(map, scope, lazy, consider_profile, skip_ifunc) \
  do {
    \
    /* 先做机器相关的运行时准备。
 \
       对 x86-64 而言，主要是为 PLT lazy binding 设置 GOT/PLT 入口。 */
 \
    int edr_lazy = elf_machine_runtime_setup ((map), (scope), (lazy),
    \
                                              (consider_profile));
    \

     \
    /* 对非 rtld 自身对象，或 rtld bootstrap 阶段，处理 RELR 重定位。 */
\
    if (!is_rtld_link_map (map) || DO_RTLD_BOOTSTRAP)
    \
      ELF_DYNAMIC_DO_RELR (map);
    \

     \
    /* 处理 REL 类型重定位。 */
   \
    ELF_DYNAMIC_DO_REL ((map), (scope), edr_lazy, skip_ifunc);
    \

     \
    /* 处理 RELA 类型重定位。x86-64 常见主要使用 RELA。 */
\
    ELF_DYNAMIC_DO_RELA ((map), (scope), edr_lazy, skip_ifunc);
    \

     \
    /* 重定位结束后的收尾动作。 */
 \
    ELF_DYNA    MIC_AFTER_RELOC ((map), (edr_lazy));
        \
  } while (0)
```

ELF 动态链接时常见几类重定位表：

动态段项	含义

`DT_REL`	不带显式 addend 的重定位表
`DT_RELA`	带显式 addend 的重定位表
`DT_JMPREL`	PLT/GOT 函数调用相关重定位表
`DT_PLTREL`	表示 `DT_JMPREL` 使用的是 `REL` 还是 `RELA`
`DT_PLTRELSZ`	PLT 重定位表大小
`DT_RELR`	压缩格式的 relative relocation 表
`DT_RELRSZ`	`DT_RELR` 表大小

其中 `RELR` 是一种更紧凑的相对重定位编码方式，主要用于减少动态库中大量 `R_*_RELATIVE` 重定位项占用的空间。

DT_REL/DT_RELA的区别?

DT_REL：

Plain Text

```
typedef struct {
    Elf64_Addr r_offset;   // 要修改的位置
    Elf64_Xword r_info;    // 符号 + 类型
} Elf64_Rel;
```

addend 在内存里 `*(r_offset) = 原始值 = addend`

loader 逻辑：

result = S + *(r_offset)

DT_RELA：

```plain text
typedef struct {
    Elf64_Addr r_offset;   // 要修改的位置
    Elf64_Xword r_info;    // 符号 + 类型
    Elf64_Sxword r_addend; // 显式 addend
} Elf64_Rela;
```

addend 在表项里：`result = S + r_addend` 内存原始值可以忽略

主要工作：

1. 架构相关初始化（PLT/GOT） `elf_machine_runtime_setup`

2. 处理 RELR（压缩相对重定位） `ELF_DYNAMIC_DO_RELR`

3. 处理 REL（无 addend） `ELF_DYNAMIC_DO_REL`

4. 处理 RELA（含 addend，x86-64 主流） `ELF_DYNAMIC_DO_RELA`

5. 收尾处理 `MIC_AFTER_RELOC`

3.4.1 架构相关初始化（PLT/GOT）

为 PLT lazy binding 做准备

- 只有存在 PLT 重定位表（DT_JMPREL）
- 且启用了 lazy binding

才需要做初始化

关键操作：

1. 获取 .got.plt `got = (Elf64_Addr *) D_PTR (l, l_info[DT_PLTGOT]);`

```c
.got
 ├── [0] _DYNAMIC 地址    //.dynamic 段的运行时地址（动态符号表、字符串表、重定位表等动态链接信息）
 ├── [1] link_map
 ├── [2] _dl_runtime_resolve        //动态链接器的解析入口
 ├── [3...] 函数地址       //PLT 函数的槽位
```

2.写入 link_map `got[1] = l;`

当进入动态解析器时，可以知道是哪个.so 发起调用

3. 设置 runtime resolver

`got[2] = _dl_runtime_resolve`

或 profiling 版本：

_dl_runtime_profile_sse

_dl_runtime_profile_avx

_dl_runtime_profile_avx512

作用：第一次调用外部函数时，PLT 跳转到这里，完成 符号解析 + 回填 GOT

```c

static inline int __attribute__ ((unused, always_inline))
elf_machine_runtime_setup (struct link_map *l, struct r_scope_elem *scope[],
                           int lazy, int profile)
{
  Elf64_Addr *got;

  /* 带 profiling 的运行时解析入口，不同 CPU 特性用不同实现。 */
  extern void _dl_runtime_profile_sse (ElfW(Word)) attribute_hidden;
  extern void _dl_runtime_profile_avx (ElfW(Word)) attribute_hidden;
  extern void _dl_runtime_profile_avx512 (ElfW(Word)) attribute_hidden;

  /* 只有对象存在 PLT 重定位，并且启用了 lazy binding，
     才需要初始化 .got.plt 的运行时解析入口。 */
  if (l->l_info[DT_JMPREL] && lazy)
    {
      /* 取出 .got.plt 的地址。 */
      got = (Elf64_Addr *) D_PTR (l, l_info[DT_PLTGOT]);

      .......

      /* 把当前对象的 link_map 指针写入 got[1]。
         这样运行时解析器就知道是哪个对象发起的 PLT 绑定请求。 */
      *(ElfW(Addr) *) (got + 1) = (ElfW(Addr)) l;

#ifdef SHARED
      /* got[2] 保存“未解析 PLT 调用”的统一入口。
         第一次调用外部函数时，PLT 最终会跳到 got[2] 指向的函数。 */
      if (__glibc_unlikely (profile))
        {
          /* profiling 模式下，根据 CPU 能力选择不同的运行时入口。 */
          const struct cpu_features* cpu_features = __get_cpu_features ();

          if (X86_ISA_CPU_FEATURE_USABLE_P (cpu_features, AVX512F))
            *(ElfW(Addr) *) (got + 2) = (ElfW(Addr)) &_dl_runtime_profile_avx512;
          else if (X86_ISA_CPU_FEATURE_USABLE_P (cpu_features, AVX))
            *(ElfW(Addr) *) (got + 2) = (ElfW(Addr)) &_dl_runtime_profile_avx;
          else
            *(ElfW(Addr) *) (got + 2) = (ElfW(Addr)) &_dl_runtime_profile_sse;

          /* 如果当前对象就是用户指定的 profiling 目标，
```

```
                则把它记录下来。 */
              if (GLRO(dl_profile) != NULL
                  && _dl_name_match_p (GLRO(dl_profile), l))
                GL(dl_profile_map) = l;
            }
          else
#endif
            {
              /* 普通 lazy binding 模式下，
                 got[2] 指向标准的 x86-64 运行时解析器。 */
              *(ElfW(Addr) *) (got + 2)
                = (ElfW(Addr)) GLRO(dl_x86_64_runtime_resolve);
            }
        }

      /* 返回 lazy 标记，供外层重定位流程继续使用。 */
      return lazy;
    }
```

3.4.2 处理 RELR（压缩相对重定位）

DT_RELR 是 ELF 动态段中一种压缩表示 R_*_RELATIVE 重定位项的格式。它描述的是“哪些地址需要做 relative relocation”

Q1:什么是 relative relocation?

在 ELF 动态链接中，R_*_RELATIVE 重定位是一类非常常见的重定位。

典型来源包括：

- 全局指针数组；
- C++ 虚表；
- .init_array / .fini_array；
- 全局对象构造相关地址；
- GOT 中不需要符号查找的相对地址；
- PIE / shared object 中大量内部地址引用。

这些重定位不需要符号查找，只需要：`*where += load_base;` / `*where = load_base + addend;`，所以非常适合压缩。

其中：

- load_base / l_addr：对象实际装载地址与链接地址之间的偏移；
- where：需要被修正的内存位置；
- addend：原始地址值或重定位项里记录的附加值。

举例：

假设一个共享库在链接时认为自己从地址：`0x00000000` 开始，但运行时实际被 loader 映射到了 `0x7f0000000000`，那么库里某些指针、全局对象地址、虚表地址等，就需要加上这个装载基址：`运行时地址 = 链接时偏移 + 装载基址`，这就是 relative relocation 的作用。

传统 ELF 中，relative relocation 通常放在：

- DT_REL
- DT_RELA

对应的重定位表里。但是 R_*_RELATIVE 数量往往很多，尤其是大型 C++ 程序、动态库、PIE 可执行文件中，会有大量类似：“某个地址槽需要加上 load_base”如果每一个都用完整的 Elf_Rel 或 Elf_Rela 结构表示，会比较占空间。所以 DT_RELR 引入了一种压缩格式，专门压缩表示这些 R_*_RELATIVE 项。

```c
#define ELF_DYNAMIC_DO_RELR(map)                                                \
  do {                                                                          \
    /*                                                                          \
     * l_addr 是该共享对象/可执行文件的装载基址。                                \
     * 对 PIE / shared object 来说，运行时地址 = 链接时地址 + l_addr。          \
     */                                                                         \
    ElfW(Addr) l_addr = (map)->l_addr, *where = NULL;                           \
                                                                                \
    /*                                                                          \
     * r 指向 DT_RELR 表起始位置，end 指向表结束位置。                          \
     */                                                                         \
    const ElfW(Relr) *r, *end;                                                  \
                                                                                \
    /*                                                                          \
     * 如果该对象没有 DT_RELR 表，则无需处理，直接退出宏。                      \
     */                                                                         \
    if ((map)->l_info[DT_RELR] == NULL)                                         \
      break;                                                                    \
                                                                                \
    /*                                                                          \
     * 取得 DT_RELR 表的运行时地址。                                            \
     * D_PTR 会根据 map 和动态段项计算出实际可访问地址。                        \
     */                                                                         \
    r = (const ElfW(Relr) *)D_PTR((map), l_info[DT_RELR]);                      \
                                                                                \
```

```
26      /*                                                                  \
27       * DT_RELRSZ 表示 RELR 表的总字节数。                                \
28       * end = r + DT_RELRSZ。                                            \
29       */                                                                 \
30      end = (const ElfW(Relr) *)((const char *)r +                        \
31                                 (map)->l_info[DT_RELRSZ]->d_un.d_val);   \
32                                                                          \
33      /*                                                                  \
34       * 遍历 RELR 表中的每一个 entry。                                    \
35       */                                                                 \
36      for (; r < end; r++)                                                \
37        {                                                                 \
38          ElfW(Relr) entry = *r;                                          \
39                                                                          \
40          /*                                                              \
41           * 如果最低位为 0，entry 是一个地址偏移。                          \
42           * 该偏移相对于 l_addr。                                          \
43           */                                                             \
44          if ((entry & 1) == 0)                                           \
45            {                                                             \
46              /*                                                          \
47               * 计算需要重定位的位置。                                     \
48               */                                                         \
49              where = (ElfW(Addr) *)(l_addr + entry);                     \
50                                                                          \
```

```
51          \
52                  /*                                                      \
53                   * relative relocation:                                 \
54                   *   *where = *where + l_addr                           \
55                   *                                                      \
56                   * 然后 where 自增，供后续 bitmap entry 使用。          \
57                   */                                                     \
58                  *where++ += l_addr;                                     \
59                }                                                         \
60              else                                                        \
61                {                                                         \
62                  /*                                                      \
63                   * 如果最低位为 1，entry 是 bitmap。                    \
64                   *                                                      \
65                   * entry 的 bit0 只是标志位，不表示重定位位置，         \
66                   * 所以循环中先执行 entry >>= 1。                       \
67                   */                                                     \
68                  for (long int i = 0; (entry >>= 1) != 0; i++)           \
69                    {                                                     \
70                      /*                                                  \
71                       * 如果当前 bit 为 1，则 where[i] 需要做相对重定位。 \
72                       */                                                 \
73                      if ((entry & 1) != 0)                               \
74                        where[i] += l_addr;                               \
75                    }                                                     \
```

```
76       \
                /*
77      \
                 * 一个 RELR entry 有 sizeof(ElfW(Relr)) * CHAR_BIT 个 bit。
78      \
79               * 其中最低位用作 bitmap 标志位，因此 bitmap 覆盖:                \
                 *   bits_per_entry - 1
80       \
                 * 个地址槽。
81     \
                 *
82       \
83               * 无论 bitmap 中实际有多少个 1，where 都要向前推进这一段。       \
                 */
84       \
                where += CHAR_BIT * sizeof(ElfW(Relr)) - 1;
85       \
              }
86       \
            }
87       \
       } while (0)
```

核心逻辑：

RELR entry 编码规则：

1. entry 低位为 0：

entry 表示一个需要重定位的位置偏移。

2. entry 低位为 1：

entry 表示一个 bitmap。

bitmap 中的每一位表示从当前 where 开始的连续地址槽中，哪些 slot 需要做 relative relocation。

相当于用一个 64-bit 的位图，表示连续 63 个地址中哪些需要 relocation，从而把多个 relocation 压缩成一个 entry。极大的节省了空间。

3.4.3处理 REL / RELA

x86-64使用的是RELA

```
#ifdef DO_RELA
# define elf_dynamic_do_Rel             elf_dynamic_do_Rela
# define Rel                            Rela
# define elf_machine_rel                elf_machine_rela
# define elf_machine_rel_relative       elf_machine_rela_relative
#endif
```

普通 DT_REL / DT_RELA 重定位表:

- ranges[0].start = DT_RELOC
- ranges[0].size = DT_RELOCSZ

```c
/*
 * 统一处理 ELF 动态重定位表。
 *
 * RELOC:
 *   通常是 REL 或 RELA，用于拼接 DT_REL / DT_RELA / DT_RELSZ / DT_RELASZ 等。
 *
 * reloc:
 *   用于拼接实际调用函数名：
 *     elf_dynamic_do_##reloc
 *
 * map:
 *   当前对象的 link_map。
 *
 * scope:
 *   符号解析作用域。
 *
 * do_lazy:
 *   是否对 PLT 重定位启用 lazy binding。
 *
 * skip_ifunc:
 *   是否跳过 IFUNC 类型的重定位。
 *
 * test_rel:
 *   是否检查 DT_PLTREL 是否等于当前处理的 DT_REL 或 DT_RELA。
 */
#define _ELF_DYNAMIC_DO_RELOC(RELOC, reloc, map, scope, do_lazy, skip_ifunc, test_rel) \
  do {                                                                                  \
    /*                                                                                  \
     * ranges 最多保存两个重定位区间：                                                 \
     *                                                                                  \
     *   ranges[0]: 普通 DT_REL / DT_RELA 重定位表                                      \
     *   ranges[1]: PLT 重定位表，也就是 DT_JMPREL                                      \
     *                                                                                  \
     * start:     重定位表起始地址                                                     \

```

```
36          * size:      重定位表字节大小                                    \
37          * nrelative: 表开头 relative relocation 的数量                  \
38          * lazy:      是否按 lazy binding 方式处理                        \
39          */                                                              \
40         struct {                                                         \
41           ElfW(Addr) start, size;                                        \
42           __typeof (((ElfW(Dyn) *) 0)->d_un.d_val) nrelative;            \
43           int lazy;                                                      \
44         } ranges[2] = { { 0, 0, 0, 0 }, { 0, 0, 0, 0 } };                \
45                                                                          \
46         /*                                                               \
47          * 处理普通 DT_REL / DT_RELA 重定位表。                          \
48          *                                                              \
49          * 注意：有 DT_RELR 时，DT_REL / DT_RELA 的动态段项可能存在，    \
50          * 但是 d_un.d_ptr 可能为 0。                                    \
51          * 所以这里不仅检查 l_info[DT_RELOC] 是否存在，                  \
52          * 还检查 d_un.d_ptr 是否非 0。                                  \
53          */                                                              \
54         if ((map)->l_info[DT_##RELOC] != NULL                            \
55             && (map)->l_info[DT_##RELOC]->d_un.d_ptr != 0)               \
56           {                                                              \
57             /*                                                           \
58              * 取得普通重定位表的起始地址。                              \
59              * 对 REL 来说是 DT_REL；对 RELA 来说是 DT_RELA。            \
60              */                                                          \
```

```
61          ranges[0].start = D_PTR ((map), l_info[DT_##RELOC]);
        \
62
        \
63          /*
        \
64           * 取得普通重定位表大小。
    \
65           * 对 REL 来说是 DT_RELSZ；对 RELA 来说是 DT_RELASZ。
    \
66           */
        \
67          ranges[0].size = (map)->l_info[DT_##RELOC##SZ]->d_un.d_val;
        \
68
        \
69          /*
        \
70           * 如果存在 DT_RELCOUNT / DT_RELACOUNT，说明表开头有若干个            \
71           * R_*_RELATIVE 重定位。
    \
72           *
          \
73           * 这些 relative relocation 可以被快速处理，
    \
74           * 不需要做符号查找。
    \
75           */
        \
76          if (map->l_info[VERSYMIDX (DT_##RELOC##COUNT)] != NULL)
        \
77            ranges[0].nrelative
        \
78              = map->l_info[VERSYMIDX (DT_##RELOC##COUNT)]->d_un.d_val;
        \
79        }
        \
80
        \
```

支持优化：`DT_RELCOUNT / DT_RELACOUNT`

- 前 N 个 relocation 是 R_RELATIVE
- 可以走快速路径（无需符号查找）

PLT 重定位处理：

- start = DT_JMPREL
- size = DT_PLTRELSZ

```c
 /*                                                                      \
      * 处理 PLT 重定位表。                                              \
      *                                                                  \
      * DT_PLTREL 表示 DT_JMPREL 使用的是 REL 还是 RELA。                \
      * 如果 test_rel 为真，则只有当前 RELOC 类型匹配 DT_PLTREL 时才处理。      \
      */                                                                 \
     if ((map)->l_info[DT_PLTREL]                                        \
         && (!test_rel || (map)->l_info[DT_PLTREL]->d_un.d_val == DT_##RELOC)) \
       {                                                                 \
         /*                                                              \
          * DT_JMPREL 是 PLT 重定位表起始地址。                          \
          */                                                             \
         ElfW(Addr) start = D_PTR ((map), l_info[DT_JMPREL]);            \
                                                                         \
         /*                                                              \
          * DT_PLTRELSZ 是 PLT 重定位表大小。                            \
          */                                                             \
         ElfW(Addr) size = (map)->l_info[DT_PLTRELSZ]->d_un.d_val;       \
                                                                         \
         /*                                                              \
          * 如果当前还没有普通重定位表，则先把 ranges[0].start 设为 PLT 表。    \
          * 后续逻辑可能将 ranges[0] 用作 PLT 表，或者再拆到 ranges[1]。        \
          */                                                             \
         if (ranges[0].start == 0)                                       \
```

```
25            ranges[0].start = start;
26      \
27      \
28          /*
29      \
30           * 如果普通重定位表的末尾正好等于 PLT 表的末尾，                    \
31           * 说明 ranges[0] 可能包含了 PLT 部分。
32      \
33           *
34          \
35           * 这里把 PLT 部分从 ranges[0] 中减掉，避免重复处理。              \
36           */
37      \
38          if (ranges[0].start + ranges[0].size == (start + size))
39      \
40            ranges[0].size -= size;
41      \
42      \
43          /*
44      \
45           * 如果不是 lazy binding，并且普通重定位表刚好紧挨着 PLT 表之前，  \
46           * 则可以把两段合并成一个连续区间一起处理。                        \
47           *
48          \
49           * 条件：
50      \
51           *   ranges[0].start + ranges[0].size == start
52        \
53           *
54          \
55           * 表示 ranges[0] 的结束地址正好是 PLT 表起始地址。                \
56           */
57      \
58          if (!(do_lazy)
59      \
60              && (ranges[0].start + ranges[0].size) == start)
61      \
62            {
63      \
64              /*
65      \
66               * 非 lazy 情况下，可以合并处理普通重定位和 PLT 重定位。      \
67               */
68      \
69              ranges[0].size += size;
70      \
```

```
52              }
        \
53          else
        \
54              {
        \
55                /*
    \
56                 * 否则，把 PLT 重定位单独放到 ranges[1]。                                \
57                 * 如果 do_lazy 为真，则 ranges[1].lazy 也为真。
  \
58                 */
    \
59                ranges[1].start = start;
        \
60                ranges[1].size = size;
        \
61                ranges[1].lazy = (do_lazy);
        \
62              }
        \
63            }
        \
64
        \
```

是否合并处理?

1.非 lazy（立即绑定） `if (!do_lazy && contiguous)    merge`

- 普通 + PLT 一起处理
- 启动时完成所有重定位

2. lazy binding `ranges[1].lazy = 1`

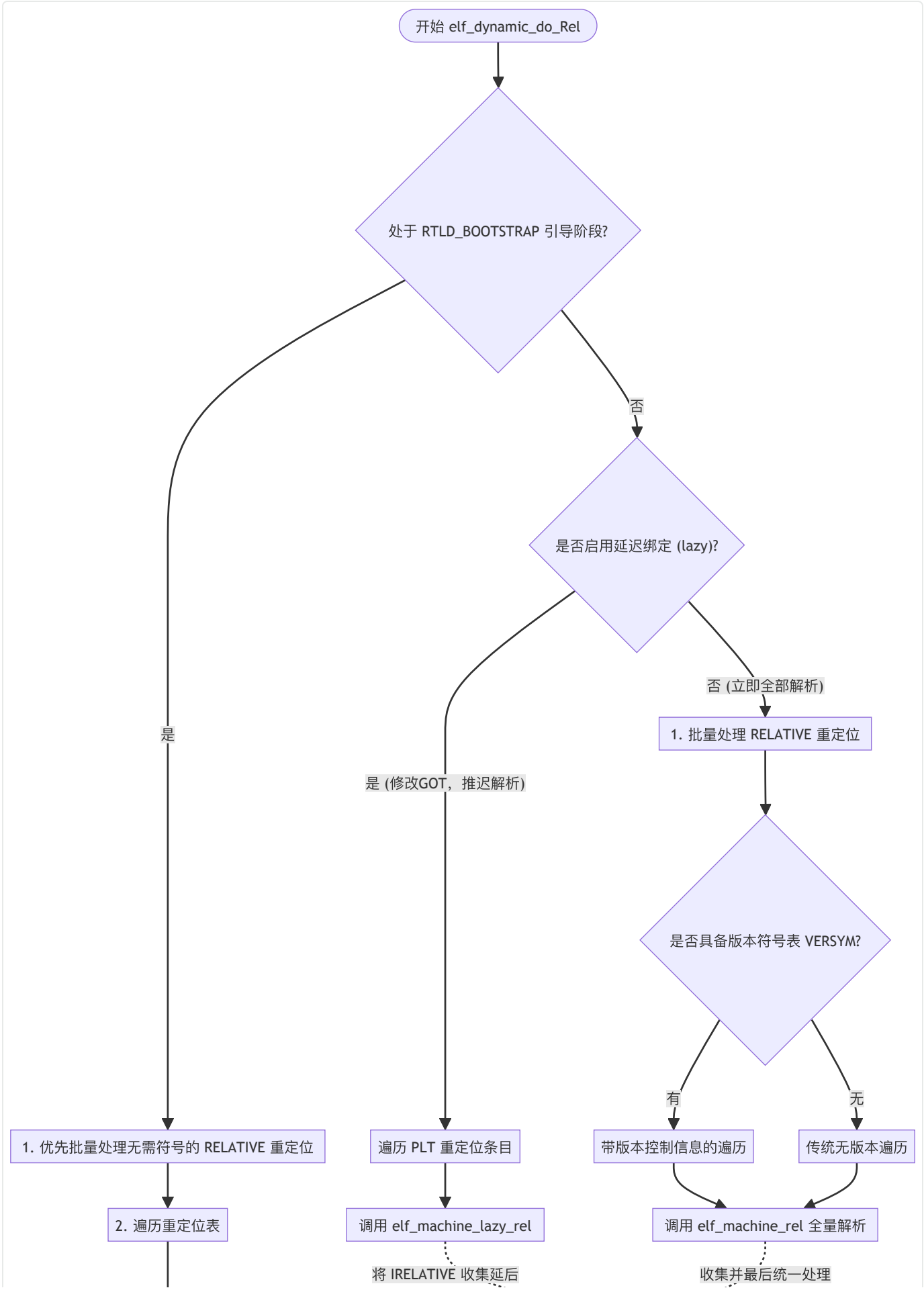
- PLT 重定位延迟执行
- 运行时按需解析

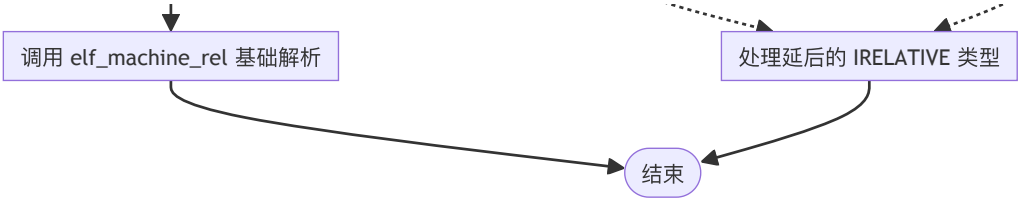
最终执行:

```c
  /*                                                                      \
   * 分别处理 ranges[0] 和 ranges[1]。                                    \
   *                                                                      \
   * 实际调用的函数由 reloc 参数拼接得到:                                  \
   *   elf_dynamic_do_##reloc                                             \
   *                                                                      \
   * 例如:                                                               \
   *   elf_dynamic_do_Rel(...)                                            \
   *   elf_dynamic_do_Rela(...)                                           \
   */                                                                     \
  for (int ranges_index = 0; ranges_index < 2; ++ranges_index)            \
    elf_dynamic_do_##reloc ((map), scope,                                 \
                            ranges[ranges_index].start,                   \
                            ranges[ranges_index].size,                    \
                            ranges[ranges_index].nrelative,               \
                            ranges[ranges_index].lazy,                    \
                            skip_ifunc);                                  \
  } while (0)

```

3.4.4 elf_dynamic_do_Rel


开始 elf_dynamic_do_Rel
处于 RTLD_BOOTSTRAP 引导阶段?
是
否
1. 优先批量处理无需符号的 RELATIVE 重定位
2. 遍历重定位表
是否启用延迟绑定 (lazy)?
是 (修改GOT，推迟解析)
否 (立即全部解析)
遍历 PLT 重定位条目
调用 elf_machine_lazy_rel
将 IRELATIVE 收集延后
1. 批量处理 RELATIVE 重定位
是否具备版本符号表 VERSYM?
有
无
带版本控制信息的遍历
传统无版本遍历
调用 elf_machine_rel 全量解析
收集并最后统一处理



变量声明：

```c
/*
 * 处理 ELF 动态重定位表中的 REL 类型重定位项。
 *
 * 注意：
 * 1. 这里处理的是 ElfW(Rel)，即 REL 重定位格式；
 * 2. REL 与 RELA 的区别在于 REL 没有显式 r_addend 字段；
 * 3. 该函数是 glibc 动态链接器 ld.so 中的核心重定位流程之一；
 * 4. 具体重定位动作由架构相关的 elf_machine_rel /
 *    elf_machine_lazy_rel / DO_ELF_MACHINE_REL_RELATIVE 完成。
 */
static inline void __attribute__ ((always_inline))
elf_dynamic_do_Rel (struct link_map *map, struct r_scope_elem *scope[],
                    ElfW(Addr) reladdr, ElfW(Addr) relsize,
                    __typeof (((ElfW(Dyn) *) 0)->d_un.d_val) nrelative,
                    int lazy, int skip_ifunc)
{
  /*
   * relative 指向重定位表起始位置。
   *
   * reladdr 是重定位表的运行时地址，一般来自动态段中的 DT_REL
   * 或其他架构相关动态项。
   */
  const ElfW(Rel) *relative = (const void *) reladdr;

  /*
   * r 指向第一个非 RELATIVE 的重定位项。
   *
   * glibc 会将 R_*_RELATIVE 类型的重定位项集中放在表的前面，
   * nrelative 表示这些 RELATIVE 项的数量。
   *
   * 因此：
   *   [relative, r) 是 RELATIVE 重定位区间；
   *   [r, end) 是普通重定位区间。
   */
  const ElfW(Rel) *r = relative + nrelative;

  /*
   * end 指向重定位表末尾。
   *
   * relsize 是整个重定位表的字节大小。
   */
  const ElfW(Rel) *end = (const void *) (reladdr + relsize);

  /*
   * 当前 ELF 对象的加载基地址。
```

```
     *
     * 对共享库或 PIE 来说，运行时加载地址通常不同于链接时地址，
     * l_addr 就是二者之间的偏移。
     */
    ElfW(Addr) l_addr = map->l_addr;

    /*
     * 获取动态符号表地址。
     *
     * 重定位项 r_info 中保存了符号索引，可通过该索引访问 symtab。
     */
    const ElfW(Sym) *const symtab
        = (const void *) D_PTR (map, l_info[DT_SYMTAB]);
```

2. RTLD_BOOTSTRAP 分支

这个分支用于动态链接器自身启动阶段。先处理 RELATIVE 重定位， 再处理普通重定位。

普通重定位的处理步骤：

- 从 r_info 中取符号索引
- 根据符号索引找到版本号
- 从符号表中取出符号项
- 计算被修正的目标地址
- 调用架构相关函数执行重定位

```c
#ifdef RTLD_BOOTSTRAP

  /*
   * RTLD_BOOTSTRAP 分支：
   *
   * 这是动态链接器 ld.so 自身启动阶段使用的代码路径。
   * 此时动态链接器还没有完全初始化，很多复杂机制不可用，
   * 所以处理流程较为简单。
   */

  /*
   * 第一阶段：批量处理 RELATIVE 重定位。
   *
   * RELATIVE 重定位不需要符号查找，只需要根据 l_addr 修正地址，
   * 是最简单、最高频的一类重定位。
   */
  for (; relative < r; ++relative)
    DO_ELF_MACHINE_REL_RELATIVE (map, l_addr, relative);

  /*
   * 获取符号版本表。
   *
   * VERSYMIDX(DT_VERSYM) 对应动态段中的符号版本信息。
   * 每个动态符号可以对应一个版本索引。
   */
  const ElfW (Half) *const version
      = (const void *) D_PTR (map, l_info[VERSYMIDX (DT_VERSYM)]);

  /*
   * 第二阶段：处理普通重定位项。
   */
  for (; r < end; ++r)
    {
      /*
       * 从 r_info 中取出符号索引，再访问 version 表。
       *
       * & 0x7fff 用于去掉高位标志位，仅保留版本索引。
       */
      ElfW (Half) ndx = version[ELFW (R_SYM) (r->r_info)] & 0x7fff;

      /*
       * 根据 r_info 中的符号索引，找到对应动态符号表项。
       */
      const ElfW (Sym) *sym = &symtab[ELFW (R_SYM) (r->r_info)];

```

```
        /*
         * 计算被重定位的位置。
         *
         * r_offset 是链接视角下的偏移，运行时需要加上加载基地址。
         */
        void *const r_addr_arg = (void *) (l_addr + r->r_offset);

        /*
         * 当前符号对应的版本约束。
         */
        const struct r_found_version *rversion = &map->l_versions[ndx];

        /*
         * 调用架构相关重定位处理函数。
         *
         * 具体如何处理 R_*_GLOB_DAT、R_*_JMP_SLOT、
         * R_*_RELATIVE、R_*_IRELATIVE 等类型，由各架构实现决定。
         */
        elf_machine_rel (map, scope, r, sym, rversion, r_addr_arg, skip_ifun
c);
    }
```

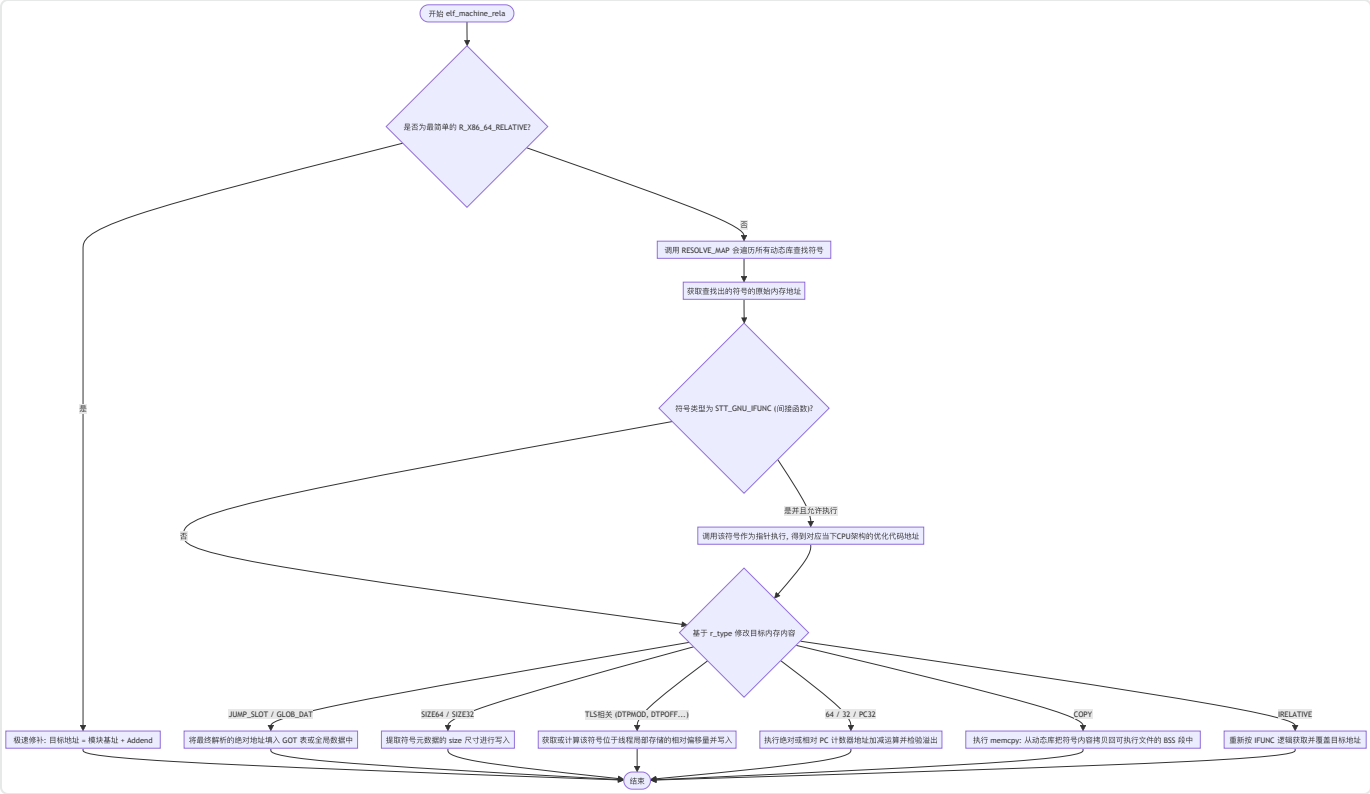
elf_machine_rel

根据一个重定位项 reloc 和已经解析完成的符号 sym，修改目标地址 reloc_addr 上的内容。它属于动态链接器的机器相关部分，专门处理 x86-64 ELF 重定位。

核心输入包括：

参数	含义
`map`	当前正在被重定位的共享对象，对应一个 `link_map`
`scope`	符号查找作用域
`reloc`	当前重定位项
`sym`	当前重定位项引用的符号

`version`	符号版本信息
`reloc_addr_arg`	需要被修正的目标地址
`skip_ifunc`	是否跳过 IFUNC resolver 调用



函数整体执行流程：

1. 获取重定位地址和重定位类型

2. 快速处理 RELATIVE / RELATIVE64 / NONE

```c
/*
 * 根据重定位项 RELOC 和已经解析完成的符号 SYM 执行重定位。
 *
 * MAP 表示当前正在被重定位的 ELF 对象，比如主程序或者某个共享库。
 *
 * 注意：
 * 这段代码来自 glibc 动态链接器的 x86-64 机器相关重定位逻辑，
 * 并不是 Linux 内核代码。内核负责装载 ELF 和启动动态链接器，
 * 真正的共享库符号解析和动态重定位由用户态的 ld.so 完成。
 */

static inline void __attribute__((always_inline))
elf_machine_rela (struct link_map *map, struct r_scope_elem *scope[],
                  const ElfW(Rela) *reloc, const ElfW(Sym) *sym,
                  const struct r_found_version *version,
                  void *const reloc_addr_arg, int skip_ifunc)
{
  /*
   * reloc_addr 是本次重定位需要修改的目标地址。
   *
   * reloc_addr_arg 通常由 relocation 的 r_offset 加上对象加载基址得到。
   * 对于不同的 relocation type，这里可能被当作 32 位、64 位、
   * TLS descriptor 等不同类型来解释。
   */
  ElfW(Addr) *const reloc_addr = reloc_addr_arg;

  /*
   * 从 r_info 字段中提取 relocation type。
   *
   * ELF64 Rela 的 r_info 同时编码了：
   *   - 符号表索引
   *   - relocation type
   *
   * ELFW(R_TYPE) 用来取出其中的重定位类型。
   */
  const unsigned long int r_type = ELFW(R_TYPE) (reloc->r_info);

# if !defined RTLD_BOOTSTRAP
  /*
   * R_X86_64_RELATIVE 是最常见、最快速的一类重定位。
   *
   * 它不需要做符号查找，计算公式是：
   *
   *   *reloc_addr = map->l_addr + reloc->r_addend
   *
```

```
     * 其中:
     *   map->l_addr        当前 ELF 对象的加载基址
     *   reloc->r_addend    relocation entry 中携带的加数
     *
     * 常见于 PIE 和共享库中的绝对地址修正。
     */
    if (__glibc_unlikely (r_type == R_X86_64_RELATIVE))
      *reloc_addr = map->l_addr + reloc->r_addend;
    else
# endif

# if !defined RTLD_BOOTSTRAP
    /*
     * R_X86_64_RELATIVE64 与 RELATIVE 类似,
     * 但它明确更新完整的 64-bit entry。
     *
     * 注释中提到:
     *   l_addr + r_addend may be > 0xffffffff
     *
     * 也就是说结果可能超过 32 位, 因此必须写入完整 64 位。
     */
    if (__glibc_unlikely (r_type == R_X86_64_RELATIVE64))
      *(Elf64_Addr *) reloc_addr = (Elf64_Addr) map->l_addr + reloc->r_addend;
    else
# endif

    /*
     * R_X86_64_NONE 表示空重定位, 不需要做任何操作。
     */
    if (__glibc_unlikely (r_type == R_X86_64_NONE))
      return;
```

3. 解析符号所在对象和符号运行时地址

- sym 表示符号表项;
- sym_map 表示该符号实际来自哪个 ELF 对象;
- value 是符号最终运行时地址。

```c
  else
    {
# ifndef RTLD_BOOTSTRAP
      /*
       * refsym 保存原始符号指针。
       *
       * 后面做错误打印、COPY relocation 尺寸比较时，
       * 需要用它从当前对象的字符串表中取符号名。
       */
      const ElfW(Sym) *const refsym = sym;
# endif

      /*
       * RESOLVE_MAP 负责根据当前对象、符号作用域、版本信息、
       * relocation type 等信息解析符号。
       *
       * 返回值 sym_map 表示符号最终定义所在的 ELF 对象。
       *
       * 注意：
       *   传入的是 &sym，说明 RESOLVE_MAP 可能会更新 sym 指针，
       *   使其指向最终解析到的符号表项。
       */
      struct link_map *sym_map = RESOLVE_MAP (map, scope, &sym, version,
                                              r_type);

      /*
       * 根据符号所在对象 sym_map 和符号表项 sym 计算符号运行时地址。
       *
       * 对于普通已定义符号，一般可以理解为：
       *
       *   value = sym_map->l_addr + sym->st_value
       *
       * 具体细节由 SYMBOL_ADDRESS 宏处理。
       */
      ElfW(Addr) value = SYMBOL_ADDRESS (sym_map, sym, true);
```

举一个函数符号的例子：

假设有：

```c
extern void foo(void);
```

main 引用了 foo，最终 `_dl_lookup_symbol_x()` 找到了 libfoo.so 里的定义。

假设：`libfoo.so` 加载到内存后的 `load bias = 0x7ffff7a00000`，`foo` 在 `libfoo.so` 中的 `st_value = 0x1140`

那么：`addr = SYMBOL_ADDRESS(map, ref, true);`

展开后相当于：

`addr = 0x7ffff7a00000 + 0x1140= 0x7ffff7a01140`

这个地址就是 foo() 的实际入口地址。

之后如果这是 R_X86_64_JUMP_SLOT 重定位，动态链接器会把这个地址写进 GOT/PLT 对应槽位。

> st_value?

> 在可重定位文件 .o、可执行文件、共享库里，st_value 的含义不完全一样。
>
> 对于动态链接阶段关注的共享库来说，通常可以这么理解：
>
> **st_value 是符号相对于该 ELF 映像基准地址的偏移或虚拟地址值**
>
> 动态库是位置无关的，加载到哪里不固定。
>
> 所以运行时必须加上：`map->l_addr` 这个加载偏移。
>
> 例如共享库链接时认为自己从：`0x0` 或者某个链接虚拟地址开始，但运行时实际被加载到：`0x7ffff7a00000` 因此符号真实地址要动态修正。

为什么需要符号查找?

在 ELF 动态链接过程中，程序或共享库中经常会出现这样的引用：

```c
extern int global_var;
extern void foo(void);
```

编译当前对象时，编译器和链接器可能并不知道 global_var 或 foo 的最终地址。

因此，在程序加载或运行时绑定阶段，动态链接器需要完成一件核心工作：把一个符号引用，解析到某个已经加载对象中的具体符号定义。

也就是说，动态链接器要回答：当前对象引用的 foo，最终应该绑定到哪个 .so 里的哪个 foo?

在 glibc 动态链接器中，这个过程的核心入口之一就是：`_dl_lookup_symbol_x()`

而在重定位路径上，通常会经过一个轻量封装：`resolve_map()`

符号解析大致流程：

```
重定位项
  |
  v
取出符号表项 ElfW(Sym)
  |
  v
resolve_map()
  |
  +--> 本地绑定？直接返回当前对象
  |
  +--> 查缓存？命中则复用上次结果
  |
  +--> 调用 _dl_lookup_symbol_x()
          |
          +--> 遍历作用域 scope
          |
          +--> 调用 do_lookup_x() 查找符号定义
          |
          +--> 处理弱符号、版本符号、protected 可见性
          |
          +--> 必要时建立依赖关系
          |
          +--> 返回符号定义所在对象 link_map
```

resolve_map()

这是重定位阶段调用 `_dl_lookup_symbol_x` 的入口封装，负责"决定一个符号引用最终绑定到哪个对象"。

数据结构 `ElfW(Sym)`

ELF 符号表项，描述一个符号的基本信息。

```c
typedef struct {
    ElfW(Word)    st_name;      //符号名在字符串表 .dynstr 中的偏移
    unsigned char st_info;      //符号绑定类型和符号类型
    unsigned char st_other;     //符号可见性
    ElfW(Section) st_shndx;     //符号所在 section，未定义符号通常为 SHN_UNDEF
    ElfW(Addr)    st_value;     //符号值，通常是相对模块基址的偏移
    ElfW(Xword)   st_size;
} ElfW(Sym);
```

情况1：本地绑定，直接返回当前对象

```c
static __always_inline lookup_t
resolve_map (lookup_t l, struct r_scope_elem *scope[], const ElfW(Sym) **ref,
             const struct r_found_version *version, unsigned long int r_type)
{
  /* 情况1：LOCAL 绑定或符号可见性要求本地绑定（如 STV_HIDDEN/STV_INTERNAL）。
     这类符号不需要跨对象查找，直接返回当前 link_map，不走后续流程。 */
  if (ELFW(ST_BIND) ((*ref)->st_info) == STB_LOCAL
      || __glibc_unlikely (dl_symbol_visibility_binds_local_p (*ref)))
    return l;
```

如果符号是：`STB_LOCAL` 或者可见性要求本地绑定，例如：`STV_HIDDEN` / `STV_INTERNAL`，那么该符号不需要跨共享库查找。因为这类符号语义上只能在当前对象内部解析，不能被其他对象抢占。

例如：`static int local_var;`

或者带 hidden visibility 的符号：

```c
__attribute__((visibility("hidden")))void hidden_func(void);
```

这类符号最终地址直接使用当前对象 l 计算即可。

情况2：查找缓存命中

```c
    /* 情况2：缓存命中——同一个符号 + 同一类型类别，之前已经查找过。
       bump_num_cache_relocations 统计缓存命中次数（调试/统计用）。
       直接复用 l_lookup_cache.ret 作为最终符号，跳过一次完整的符号查找。 */
    if (__glibc_unlikely (*ref == l->l_lookup_cache.sym)
        && elf_machine_type_class (r_type) == l->l_lookup_cache.type_class)
      {
        bump_num_cache_relocations ();
        *ref = l->l_lookup_cache.ret;    /* 用缓存结果覆盖 ref，不再查表 */
      }
```

这里使用了 l_lookup_cache。

缓存条件有两个：

- 当前符号表项 *ref 和上次查找符号相同；
- 当前重定位类型类别和上次相同。

同一个符号 + 同一类重定位，才可以复用上次查找结果。

为什么还要比较重定位类型类别?

因为函数调用类重定位和数据访问类重定位的符号绑定规则可能不同。

常见分类包括：

类型类别	含义
`ELF_RTYPE_CLASS_PLT`	PLT 调用类重定位，通常对应函数调用
非 PLT	数据、变量或普通地址重定位

尤其在处理 STV_PROTECTED、IFUNC 等特殊符号时，PLT 和非 PLT 的规则可能不同，因此不能只按符号名缓存。

情况3：缓存未命中，执行完整符号查找

```c
  else
    {
      /* 情况3：缓存未命中，需要做完整的跨对象符号查找。 */

      const int tc = elf_machine_type_class (r_type); /* 当前重定位类型类别
*/

      /* 将本次查询的 sym 和类型写进缓存，为下次可能的命中做准备。 */
      l->l_lookup_cache.type_class = tc;
      l->l_lookup_cache.sym = *ref;

      /* 从当前对象的字符串表里取出符号的名称字符串。
         DT_STRTAB 指向 .dynstr，st_name 是相对偏移。 */
      const char *undef_name
          = (const char *) D_PTR (l, l_info[DT_STRTAB]) + (*ref)->st_name;

      /* 只有 version->hash != 0 时才传版本约束，否则传 NULL 做无版本查找。
         hash == 0 通常表示版本信息无效或未指定。 */
      const struct r_found_version *v = NULL;
      if (version != NULL && version->hash != 0)
        v = version;

      /* 核心调用：在 scope 定义的作用域链里查找 undef_name 的定义。
         - DL_LOOKUP_ADD_DEPENDENCY：如果符号来自动态加载对象，建立依赖关系，
           防止该对象在当前对象还在用的情况下被 dlclose 卸载。
         - DL_LOOKUP_FOR_RELOCATE：告知查找器这是重定位阶段的调用，
           影响部分特殊符号（如 IFUNC）的处理方式。
         - skip_map = NULL：不跳过任何对象。
         返回值 lr 是"提供该符号定义的对象"的 lookup_t（本质是 link_map*）。 */
      lookup_t lr = _dl_lookup_symbol_x (
          undef_name, l, ref, scope, v, tc,
          DL_LOOKUP_ADD_DEPENDENCY | DL_LOOKUP_FOR_RELOCATE, NULL);

      /* 把查找结果写回缓存：ret 是最终命中的符号表项，value 是提供符号的对象。 */
      l->l_lookup_cache.ret   = *ref;    /* 命中的 Elf64_Sym 指针 */
      l->l_lookup_cache.value = lr;      /* 提供该符号的 link_map */
    }

  /* 返回"提供该符号定义的对象"，供调用方（elf_machine_rel 等）
     计算最终地址时使用（load_addr + st_value）。 */
  return l->l_lookup_cache.value;
}

```

传入的 flags：`DL_LOOKUP_ADD_DEPENDENCY | DL_LOOKUP_FOR_RELOCATE`

flag	含义
`DL_LOOKUP_ADD_DEPENDENCY`	如果符号来自动态加载对象，需要建立依赖关系，防止目标对象提前卸载
`DL_LOOKUP_FOR_RELOCATE`	表示这是重定位阶段的查找，会影响部分特殊处理逻辑

_dl_lookup_symbol_x

负责在作用域链中查找符号定义。

主要负责：

1. 计算符号哈希；
2. 遍历查找作用域；
3. 调用 do_lookup_x() 查找候选符号；
4. 处理找不到符号的情况；

```c
lookup_t
_dl_lookup_symbol_x (const char *undef_name, struct link_map *undef_map,
                     const ElfW(Sym) **ref,
                     struct r_scope_elem *symbol_scope[],
                     const struct r_found_version *version,
                     int type_class, int flags, struct link_map *skip_map)
{
  /* 计算 GNU 风格哈希值，加速符号查找。 */
  const unsigned int new_hash = _dl_new_hash (undef_name);

  /* 旧式 ELF hash，初始化为无效值，供 do_lookup_x 按需使用。 */
  unsigned long int old_hash = 0xffffffff;

  /* current_value 保存当前找到的“最佳匹配”：
     s = 符号表项
     m = 符号所在模块(link_map) */
  struct sym_val current_value = { NULL, NULL };

  /* scope 是作用域数组的遍历指针。 */
  struct r_scope_elem **scope = symbol_scope;

  /* 统计一次重定位/符号解析行为。 */
  bump_num_relocations ();

  /* 带版本的查找不允许同时要求“返回最新版本”。 */
  assert (version == NULL || !(flags & DL_LOOKUP_RETURN_NEWEST));

  size_t i = 0;

  if (__glibc_unlikely (skip_map != NULL))
    /* 如果给了 skip_map，则先在当前 scope 的对象列表里找到它的位置。
       后续查找从该位置开始或跳过它，用于控制搜索起点。 */
    while ((*scope)->r_list[i] != skip_map)
      ++i;

  /* 在相关已加载对象中查找符号定义。
     start 只在第一个 scope 中可能非 0，后面的 scope 都从 0 开始。 */
  for (size_t start = i; *scope != NULL; start = 0, ++scope)
    if (do_lookup_x (undef_name, new_hash, &old_hash, *ref,
                     &current_value, *scope, start, version, flags,
                     skip_map, type_class, undef_map) != 0)
      /* 找到了合适定义就停止搜索。 */
      break;

  /* 没找到任何定义。 */
```

```c
    if (__glibc_unlikely (current_value.s == NULL))
      {
        /* 如果原始引用不是弱符号，并且当前不是“仅调试未使用对象”的模式，
           那么这就是一个真正的未定义强符号错误。 */
        if ((*ref == NULL || ELFW(ST_BIND) ((*ref)->st_info) != STB_WEAK)
            && !(GLRO(dl_debug_mask) & DL_DEBUG_UNUSED))
          {
            /* 构造错误信息：undefined symbol: xxx[, version yyy] */
            const char *reference_name = undef_map ? undef_map->l_name : "";
            const char *versionstr = version ? ", version " : "";
            const char *versionname = (version && version->name
                                       ? version->name : "");
            struct dl_exception exception;

            /* 这里的错误信息不会做本地化翻译。 */
            _dl_exception_create_format
              (&exception, DSO_FILENAME (reference_name),
               "undefined symbol: %s%s%s",
               undef_name, versionstr, versionname);

            /* 抛出动态链接异常。 */
            _dl_signal_cexception (0, &exception, N_("symbol lookup error"
));
            _dl_exception_free (&exception);
          }

        /* 弱符号找不到时，允许返回空。 */
        *ref = NULL;
        return NULL;
      }
      ......
```

5. 处理protected符号；

```c
    /* 判断原始引用符号是否是 STV_PROTECTED 可见性。
       protected 符号的绑定规则与普通全局符号不同。 */
    int protected = (*ref
                     && ELFW(ST_VISIBILITY) ((*ref)->st_other) == STV_PROTECT
ED);
```

STV_PROTECTED 是 ELF 符号可见性中的一种。

它的语义比较特殊：符号对外可见，但当前模块内部对该符号的引用必须绑定到本模块自己的定义。

这不同于普通全局符号。

普通全局符号可能被其他共享库抢占，例如通过：

- 主程序定义同名符号；
- LD_PRELOAD；
- 更早加载的共享库。

但 protected 符号要求：模块内部引用优先绑定到模块自身定义

举例：

假设 libfoo.so 里有：

```c
void api(void) {
    helper();
}

void helper(void) {
    // libfoo 自己的 helper
}
```

如果 helper 是普通 STV_DEFAULT 全局符号，那么 libfoo.so 内部对 helper() 的调用，在动态链接语义上也可能被外部定义抢占。

例如主程序或者 LD_PRELOAD 里也定义了一个 helper：

```c
void helper(void) {
    // 外部的 helper
}
```

这时 libfoo.so 内部的 api() 调用 helper()，理论上可能绑定到外部的 helper()。

> 外部对象引用 helper：
>
> 可以绑定到 libfoo.so:helper
>
> libfoo.so 内部引用 helper：
>
> 必须绑定到 libfoo.so:helper
>
> 不能被 main、LD_PRELOAD 或其他 DSO 的 helper 抢占

分两种情况：

- **PLT 重定位场景**

```c
if (__glibc_unlikely (protected != 0))
   {
     /* protected 符号处理比较特殊。
        它要求“本模块内引用优先绑定到本模块定义”，不能像普通全局符号那样
        被别的模块轻易抢走。 */
     if (type_class == ELF_RTYPE_CLASS_PLT)
       {
         /* 对 PLT 调用类重定位：
            如果查找到的定义来自别的模块，而当前引用模块本身有 protected 定义，
            则改回绑定到本模块。 */
         if (current_value.s != NULL && current_value.m != undef_map)
           {
             current_value.s = *ref;
             current_value.m = undef_map;
           }
       }
```

如果当前是 PLT 类重定位，并且符号查找到别的模块中，但当前引用模块自身有 protected 定义，则强制改回本模块。

- **非 PLT 重定位场景**

非 PLT 场景下，代码会额外做一次按 PLT 规则的查找：

```c
      else
        {
          /* 对非 PLT 重定位，需要再做一次按 PLT 规则的查找，
             用来判断是否应该改回本模块定义。 */
          struct sym_val protected_value = { NULL, NULL };

          for (scope = symbol_scope; *scope != NULL; i = 0, ++scope)
            if (do_lookup_x (undef_name, new_hash, &old_hash, *ref,
                             &protected_value, *scope, i, version, flags,
                             skip_map, ELF_RTYPE_CLASS_PLT, NULL) != 0)
              break;

          /* 如果按 PLT 规则查到的是别的模块，而当前模块本身有 protected 定义，
             则最终仍应绑定回当前引用模块。 */
          if (protected_value.s != NULL && protected_value.m != undef_map)
            {
              current_value.s = *ref;
              current_value.m = undef_map;
            }
        }
    }
```

目的是判断：如果按照函数调用类规则查找，会不会命中别的模块?

如果会，并且当前模块有 protected 定义，则最终仍然回退到当前模块。

6. 输出调试绑定信息；

符号查找完成后，还有一个重要步骤：

```c
  /* 如果最终绑定到了一个 dlopen 动态加载的对象，并且这是隐式运行时查找，
     则要建立依赖关系，防止目标对象在当前引用者仍然存活时被卸载。 */
  if (__glibc_unlikely (current_value.m->l_type == lt_loaded)
      /* 显式查找（如 dlsym）不做这件事，只对运行时隐式绑定生效。 */
      && (flags & DL_LOOKUP_ADD_DEPENDENCY) != 0
      /* 建立 undef_map -> current_value.m 依赖。 */
      && add_dependency (undef_map, current_value.m, flags) < 0)
      /* 如果加依赖失败，说明目标对象可能刚被移除。
         递归重试一次符号查找，尝试找到其他可用定义。 */
      return _dl_lookup_symbol_x (undef_name, undef_map, ref,
                                  (flags & DL_LOOKUP_GSCOPE_LOCK)
                                  ? undef_map->l_scope : symbol_scope,
                                  version, type_class, flags, skip_map);
```

这里处理的是 dlopen() 加载对象的生命周期问题。

假设：`libA.so` 通过重定位绑定到了 `libB.so` 的符号，如果 libB.so 是运行时 `dlopen()` 加载的，那么必须保证：只要 `libA.so` 还在使用 `libB.so` 的符号，`libB.so` 就不能被 `dlclose()` 卸载

因此动态链接器需要建立依赖关系：

`undef_map -> current_value.m`

也就是：引用者 -> 提供者

如果建立依赖失败，说明目标对象可能正在卸载，于是会递归重试符号查找。

7. 标记对象已使用

```c
/* 标记目标对象已被使用。 */
  if (__glibc_unlikely (current_value.m->l_used == 0))
    current_value.m->l_used = 1;
```

这一步将符号提供者标记为已使用。

这个标记常用于：

- 调试输出；
- 未使用依赖分析；
- 动态链接器内部状态维护。

8. 调试输出：观察符号绑定

如果启用了绑定调试：

`LD_DEBUG=bindings`

动态链接器会打印类似信息：

```c
binding file ./main [0] to /lib/x86_64-linux-gnu/libc.so.6 normal symbol `printf'
```

这对应代码：

```c
    /* 如果启用了绑定调试输出，打印“谁绑定到了谁”。 */
  if (__glibc_unlikely (GLRO(dl_debug_mask) & DL_DEBUG_BINDINGS))
    {
      const char *reference_name = undef_map->l_name;

      _dl_debug_printf ("binding file %s [%lu] to %s [%lu]: %s symbol `%
s'",
                        DSO_FILENAME (reference_name),
                        undef_map->l_ns,
                        DSO_FILENAME (current_value.m->l_name),
                        current_value.m->l_ns,
                        protected ? "protected" : "normal", undef_name);
      if (version)
        _dl_debug_printf_c (" [%s]\n", version->name);
      else
        _dl_debug_printf_c ("\n");
    }
```

这对排查符号绑定问题非常有用。

例如：

```c
LD_DEBUG=bindings ./a.out
```

可以观察：

- printf 绑定到哪个 libc；
- 自定义函数是否被 LD_PRELOAD 抢占；
- 同名符号最终解析到了哪个共享库。

9. 最终返回结果

函数最后：

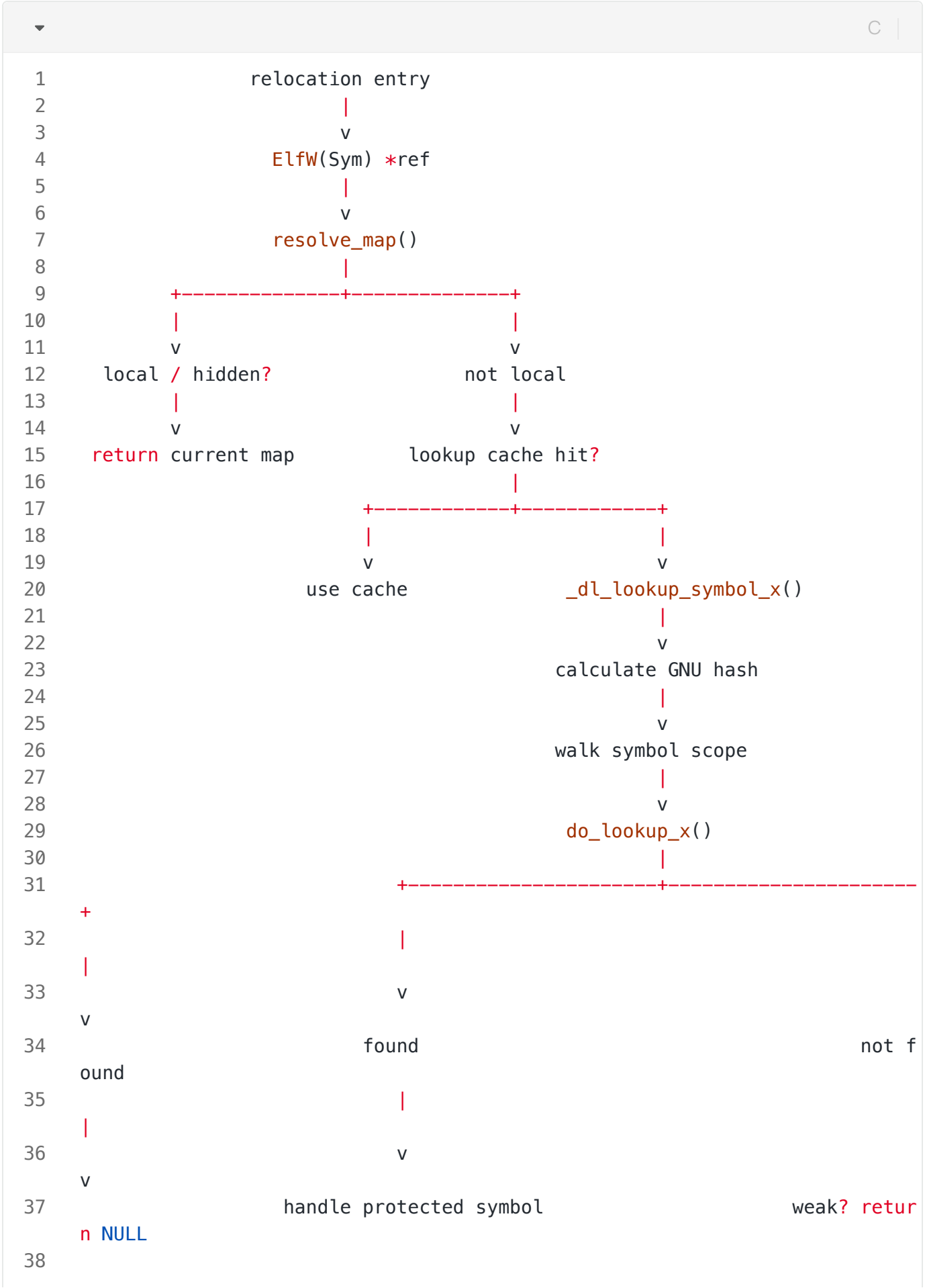
```c
    /* 输出最终命中的符号表项。 */
    *ref = current_value.s;

    /* 返回符号所在对象的 lookup 值。 */
    return LOOKUP_VALUE (current_value.m);
}
```

也就是说：

*ref 被更新为最终命中的符号表项；返回值是符号所在对象。

图示：


relocation entry
ElfW(Sym) *ref
resolve_map()
local / hidden?
return current map
not local
lookup cache hit?
use cache
_dl_lookup_symbol_x()
calculate GNU hash
walk symbol scope
do_lookup_x()
found
not found
handle protected symbol
weak? return NULL

```
                                |                                  strong? err
or
                                v
                      add dependency if needed
                                |
                                v
                        update *ref and return map
                                |
                                v
                      relocation computes address
```

do_lookup_x()

由 _dl_lookup_symbol_x 调用，用来在某一个查找作用域 scope 中逐个扫描 link_map，并尝试找到目标符号。

4. 处理 IFUNC

> 什么是 IFUNC?

IFUNC，全称是 indirect function，是 GNU 扩展机制。

一个符号不是直接对应一个函数地址，而是对应一个 resolver。运行时由 resolver 根据 CPU 特性、硬件能力或环境选择真正的实现。

典型例子：`memcpy`

在不同 CPU 上可能选择：

- 普通实现；
- SSE 实现；
- AVX 实现；
- AVX2 实现；
- AVX-512 实现。

代码中：

```c
        if (sym != NULL
            && __glibc_unlikely (ELFW(ST_TYPE) (sym->st_info) == STT_GNU_IFU
NC)
            && __glibc_likely (sym->st_shndx != SHN_UNDEF)
            && __glibc_likely (!skip_ifunc))
          {
            ......
            # endif
            /*
             * 对 IFUNC 来说，value 当前是 resolver 函数地址。
             *
             * 这里把 value 转成：
             *
             *   ElfW(Addr) (*)(void)
             *
             * 然后调用 resolver，返回真正的函数地址。
             */
            value = ((ElfW(Addr) (*) (void)) value) ();
          }
```

这里会把符号地址当作函数指针调用：

`value = resolver();`

resolver 返回真正要写入 GOT/PLT 的函数地址。

IFUNC 循环依赖检查

代码中还有一段：`if (sym_map != map && !sym_map->l_relocated)`

```c
        if (sym != NULL
            && __glibc_unlikely (ELFW(ST_TYPE) (sym->st_info) == STT_GNU_IFUNC)
            && __glibc_likely (sym->st_shndx != SHN_UNDEF)
            && __glibc_likely (!skip_ifunc))
          {
# ifndef RTLD_BOOTSTRAP
            if (sym_map != map
                && !sym_map->l_relocated)
              {
                /*
                 * 当前对象的字符串表，用于根据 refsym->st_name 找到符号名。
                 */
                const char *strtab
                  = (const char *) D_PTR (map, l_info[DT_STRTAB]);

                /*
                 * 如果 IFUNC 定义在可执行文件中并形成不可满足的循环依赖，
                 * 直接 fatal。
                 */
                if (sym_map->l_type == lt_executable)
                  _dl_fatal_printf ("\
%s: IFUNC symbol '%s' referenced in '%s' is defined in the executable \
and creates an unsatisfiable circular dependency.\n",
                                    RTLD_PROGNAME, strtab + refsym->st_name,
                                    map->l_name);
                else
                  /*
                   * 否则打印错误，提示用户重新链接，使依赖关系更合理。
                   */
                  _dl_error_printf ("\
%s: Relink `%s' with `%s' for IFUNC symbol `%s'\n",
                                    RTLD_PROGNAME, map->l_name,
                                    sym_map->l_name,
                                    strtab + refsym->st_name);
              }


```

意思是：如果 IFUNC 定义在另一个对象中，而那个对象还没有完成重定位，就可能产生循环依赖。

例如：libA 重定位时需要调用 libB 的 IFUNC resolver，libB 还没完成重定位，libB resolver 又依赖 libA

这种情况可能导致不可满足的依赖关系，所以 glibc 会打印错误或直接 fatal。

5.常见重定位类型分析

重定位类型	主要用途	计算逻辑	写入内容	写入位置	典型场景	备注
R_X86_64_RELATIVE	基于加载基址修正地址	B+A	修正后的运行时地址	relocation 指定的位置，常见于 .data、.data.rel.ro、.init_array、.fini_array、.got 等	PIE、共享库中的内部指针修正	不需要符号解析，性能最好，数量通常很多
R_X86_64_RELATIVE64	64 位版本的相对重定位	B+A	完整 64 位地址	relocation 指定的位置，通常是需要完整 64 位地址的表项或数据项	地址结果可能超过 32 位的场景	明确写入 64-bit entry
R_X86_64_NONE	空重定位	无	不写入	无	占位或无需处理的 relocation	直接返回
R_X86_64_JUMP_SLOT	函数调用地址绑定	S	函数真实地址	通常是 `.got.plt` 中的 PLT GOT 槽位	外部函数调用，例如 `printf@plt`	和 PLT/GOT、lazy binding 或 `-z now` 相关
R_X86_64_GLOB_DAT	全局符号地址绑定	S	全局变量地址或函数地址	通常是 `.got` 中的普通 GOT 表项	全局变量、函数指针、GOT 表项	和 `JUMP_SLOT` 在该代码中最终写入逻辑类似

R_X86_64_64	64 位绝对地址重定位	S+A	64 位绝对地址	relocation 指定的位置，常见于 `.data`、`.data.rel.ro`、部分指针常量区	指针、全局对象地址、函数地址	写入完整 64 位结果
R_X86_64_32	32 位绝对地址重定位	S+A	低 32 位值	relocation 指定的位置，目标字段是 32 位宽度	非 PIE 或特殊 ABI 场景	需要检查是否超过 `UINT_MAX`
R_X86_64_PC32	32 位 PC-relative 重定位	S+A-P	32 位相对偏移	relocation 指定的位置，常见于 `.text` 中的 call/jmp/访存指令立即数字段	相对跳转、相对调用、RIP-relative 引用	需要检查是否能表示为 32 位有符号数
R_X86_64_SIZE64	写入符号大小	Z+A	64 位大小值	relocation 指定的位置，目标字段通常是 64 位数据槽	使用符号大小的场景	写入的是 size，不是 address
R_X86_64_SIZE32	写入 32 位符号大小	Z+A	32 位大小值	relocation 指定的位置，目标字段通常是 32 位数据槽	使用符号大小且目标字段为 32 位	需要注意 32 位截断风险
R_X86_64_COPY	数据对象复制	从共享库符号地址拷贝数据	目标对象中的数据副本	通常是主程序的 `.bss` 或 `.data` 中对应变量位置	主程序引用共享库中的全局变量	ABI 复杂，现代代码中应尽量避免依赖
R_X86_64_IRELATIVE	间接相对重定位	resolver(B+A)	resolver 返回的地址	relocation 指定的位置，常见于 `.got`、`.data.rel.ro` 或函数指针槽位	IFUNC 风格的本地 resolver	不依赖普通符号查找，常用于运行时选择实现

其中：

符号	含义
B	当前 ELF 对象的加载基址，即 `map->l_addr`
S	符号运行时地址
A	relocation entry 中的 addend，即 `reloc->r_addend`
P	relocation 发生的位置地址，即 `reloc_addr`
Z	符号大小，即 `sym->st_size`

6.TLS 相关重定位总结

TLS，即 Thread Local Storage，线程局部存储。

例如：

```
__thread int tls_var;
```

每个线程都有自己独立的一份 tls_var。

因此 TLS relocation 不只是简单写入一个地址，还需要考虑：

- TLS 模块 ID；
- TLS 块偏移；
- 静态 TLS；
- 动态 TLS；

TLS 重定位类型	主要用途	计算逻辑	写入内容	写入位置	典型场景	备注
R_X86_64_DTPMOD64	获取 TLS 模块 ID	使用符号所在对象的 TLS module ID	TLS 模块编号	通常是 GOT 中 TLS 相关表项	动态 TLS 访问	写入 `sym_map->l_tls_modid`
R_X86_64_DTPOFF64	获取符号在 TLS block 内的偏移	TLS 符号值 + addend	TLS block 内部偏移	通常是 GOT 中 TLS 相关表项	General Dynamic / Local Dynamic TLS 模型	偏移相对于 TLS block 起始位置

R_X86_64_TPOFF64	获取相对于 thread pointer 的偏移	TLS 符号值 + addend − TLS 对象偏移	相对于线程指针的偏移	通常是 GOT 中 TLS 相关表项	Initial Exec / Local Exec TLS 模型	x86-64 下通常是负偏移
R_X86_64_TLSDESC	构造 TLS descriptor	根据是否可用 static TLS 选择不同路径	TLS descriptor 的 `arg` 和 `entry`	通常是 `.got.plt` 或 `.got` 中的 TLS descriptor 槽位，包含两个机器字	TLSDESC TLS 模型	更灵活，可支持 static TLS 和 dynamic TLS

普通重定位主要解决：

“这个符号或地址在运行时到底在哪里？”

TLS 重定位主要解决：

“当前线程访问这个线程局部变量时，应该如何定位到属于自己的那一份数据？”

下面进入非bootstrap 分支，这是正常动态链接过程使用的路径。相比 bootstrap，它需要处理更多情况：

- lazy binding
- IFUNC / IRELATIVE
- symbol versioning 可有可无
- audit hook
- ld.so 自身避免重复重定位

3.Lazy Binding 分支

lazy binding 是什么?

对于动态函数调用，ELF 通常使用 PLT/GOT 机制。

例如程序调用：`printf("hello\n");` 编译链接后，调用可能先进入 PLT stub。如果启用 lazy binding：

1. 程序启动时不立即解析 printf

2. 第一次调用 printf 时进入动态链接器
3. 动态链接器查找 printf
4. 把 GOT/PLT 表项改写为真实函数地址
5. 后续调用直接跳到真实函数

优点：

- 缩短程序启动时间
- 避免解析未使用函数

缺点：

- 首次调用有额外开销
- 安全性略差，因此很多场景使用 BIND_NOW 或 full RELRO

```c
#if !defined DO_RELA || !defined ELF_MACHINE_PLT_REL

  /*
   * lazy 分支:
   *
   * 在 ld.so bootstrap 阶段不会做 lazy binding。
   * 但编译器未必能通过函数调用关系推断出这一点，
   * 所以这里保留运行时判断。
   */
  if (lazy)
    {
      /*
       * 处理 lazy PLT 重定位。
       *
       * lazy PLT 重定位通常不需要完整符号解析，
       * 只需要设置好 PLT/GOT，让第一次函数调用时进入 resolver。
       */
      for (; r < end; ++r)
# ifdef ELF_MACHINE_IRELATIVE
        /*
         * 如果遇到 IRELATIVE，先不立即处理。
         *
         * 记录第一个 IRELATIVE 到 r2，
         * 记录最后一个 IRELATIVE 到 end2。
         */
        if (ELFW(R_TYPE) (r->r_info) == ELF_MACHINE_IRELATIVE)
          {
            if (r2 == NULL)
              r2 = r;
            end2 = r;
          }
        else
# endif
          /*
           * 架构相关的 lazy 重定位处理。
           */
          elf_machine_lazy_rel (map, scope, l_addr, r, skip_ifunc);
......
    }
```

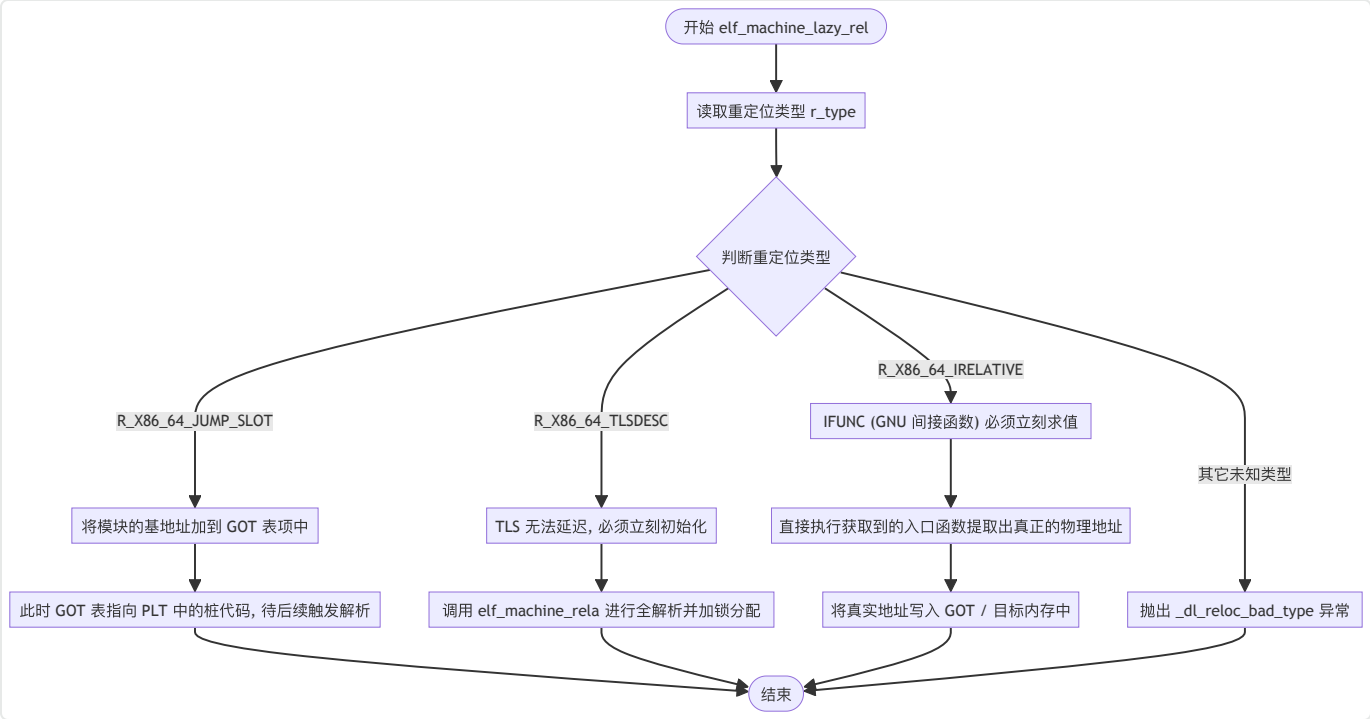
elf_machine_lazy_rel

作用：在 lazy binding 模式下，对延迟重定位项做初始化处理。

对于普通的 R_X86_64_JUMP_SLOT，它不会立即解析目标函数真实地址，而是把 GOT.PLT 表项修正到一个可以触发动态链接器解析流程的位置。

函数整体执行流程：

1. 根据 l_addr + reloc->r_offset 计算 relocation 目标地址
2. 取出 relocation type
3. 如果是 R_X86_64_JUMP_SLOT：
   初始化 GOT.PLT，使其第一次调用时进入 PLT/ld.so 解析流程
4. 如果是 R_X86_64_TLSDESC：
   不能真正 lazy，直接调用 elf_machine_rela 完整处理
5. 如果是 R_X86_64_IRELATIVE：
   立即执行 IFUNC resolver，写入最终地址
6. 其他类型：
   报错



在 lazy binding 模式下，程序启动阶段并不把 `GOTPLT[printf]` 直接写成 `&printf`，而是让它指向 PLT 中的一段跳板代码。

与elf_machine_rela区别：

elf_machine_rela 是“真正执行重定位”的通用处理函数；

elf_machine_lazy_rel 是“为 lazy binding 做初始化”的轻量处理函数。

对比项	elf_machine_rela	elf_machine_lazy_rel
主要用途	执行完整重定位	初始化 lazy relocation
典型处理对象	`.rela.dyn` 、立即绑定的 `.rela.plt`	lazy binding 下的 `.rela.plt`
是否解析普通符号	是	对 `JUMP_SLOT` 通常不解析
是否写入最终函数地址	是	`JUMP_SLOT` 下通常不写最终地址
`R_X86_64_JUMP_SLOT` 处理方式	把真实函数地址写入 GOT.PLT	把 GOT.PLT 修正到 PLT resolver 路径
是否调用 `RESOLVE_MAP`	通常会	普通 `JUMP_SLOT` 不会
是否调用 `SYMBOL_ADDRESS`	通常会	普通 `JUMP_SLOT` 不会
是否处理 `RELATIVE`	是	否
是否处理 `GLOB_DAT`	是	否
是否处理 TLS	处理多种 TLS relocation	只特殊处理 `TLSDESC`，并转交给 `elf_machine_rela`
是否处理 IFUNC	会处理 `STT_GNU_IFUNC`、`IRELATIVE`	只直接处理 `IRELATIVE`
复杂度	高	低
执行结果	relocation 位置被写入最终值	lazy `JUMP_SLOT` 位置被写入“触发解析”的跳板地址

之后还对IRELATIVE / IFUNC 进行了特殊处理

```c
# ifdef ELF_MACHINE_IRELATIVE
      /*
       * 所有普通 lazy 重定位处理完成后,
       * 再回过头处理 IRELATIVE 项。
       */
      if (r2 != NULL)
        for (; r2 <= end2; ++r2)
          if (ELFW(R_TYPE) (r2->r_info) == ELF_MACHINE_IRELATIVE)
            elf_machine_lazy_rel (map, scope, l_addr, r2, skip_ifunc);
# endif
```

Q1:什么是 IRELATIVE?

IRELATIVE 是 IFUNC 相关的一种重定位类型。

普通 RELATIVE 重定位通常是：`target = base + addend;`

而 IRELATIVE 类似：`target = resolver();` 或者更准确地说：`target = ((function_pointer)(base + addend))();` 也就是说，它需要执行一个 resolver 函数，resolver 返回真正的函数地址。

Q2:为什么 IRELATIVE 要延后处理?

代码中把 IRELATIVE 项先记录下来：

```c
#else /* !RTLD_BOOTSTRAP */

  /*
   * 非 RTLD_BOOTSTRAP 分支:
   *
   * 这是动态链接器正常运行阶段的重定位处理路径,
   * 需要支持 lazy binding、IFUNC、audit、符号版本等机制。
   */

# if defined ELF_MACHINE_IRELATIVE
  /*
   * r2/end2 用于记录 IRELATIVE 重定位项的范围。
   *
   * IRELATIVE 与 IFUNC 相关，处理它时可能需要调用 resolver 函数。
   * resolver 可能依赖其他重定位已经完成，因此这里先记录,
   * 后面延迟处理。
   */
  const ElfW(Rel) *r2 = NULL;
  const ElfW(Rel) *end2 = NULL;
# endif

```

然后在普通重定位处理完后，再处理：

```c
      if (r2 != NULL)
        for (; r2 <= end2; ++r2)
          if (ELFW(R_TYPE) (r2->r_info) == ELF_MACHINE_IRELATIVE)
            elf_machine_lazy_rel (map, scope, l_addr, r2, skip_ifunc);
```

原因是：

IRELATIVE resolver 函数执行时，可能依赖其他已经完成的重定位。

如果过早调用 resolver，可能导致：

- GOT 尚未完成
- 外部符号尚未解析
- 函数地址尚不可用
- resolver 内部访问全局变量失败

因此 glibc 选择先跳过 IRELATIVE，把它们延迟到其他重定位完成后处理。

4.非 lazy 分支

当 lazy == 0 时，进入立即绑定逻辑。这表示启动阶段就完成所有必要重定位。这通常发生在：

- LD_BIND_NOW=1
- ELF 设置了 DF_BIND_NOW
- 安全加固配置要求立即绑定
- 非 PLT 重定位
- 某些架构或重定位类型不支持 lazy

1.RELATIVE 重定位优化

```c
  else
#endif
    {
      /*
       * 非 lazy 分支:
       *
       * 启动阶段立即完成重定位。
       */

      /*
       * 如果当前 map 不是动态链接器自身，则处理 RELATIVE 重定位。
       *
       * ld.so 自身的重定位在更早的启动流程里已经完成，
       * 因此这里避免重复处理。
       */
      if (!is_rtld_link_map (map)) /* Already done in rtld itself.  */
# if !defined DO_RELA || defined ELF_MACHINE_REL_RELATIVE
        /*
         * 对 RELA 平台而言，addend 来自 r_addend，并且需要复制到
         * 被重定位的位置。
         *
         * 因此只有在 REL 重定位，或者某些 RELA 平台把重定位定义为
         * memory_loc += l_addr 的情况下，才能按这里的方式跳过或批处理。
         */
        if (l_addr != 0)
# endif
          /*
           * 批量处理 RELATIVE 重定位。
           */
          for (; relative < r; ++relative)
            DO_ELF_MACHINE_REL_RELATIVE (map, l_addr, relative);
```

`if (l_addr != 0)`

对于某些非 PIE 主程序，它的加载基地址可能是 0。如果 l_addr == 0，某些相对重定位可能不需要做额外修正。不过这里还有架构和 REL/RELA 差异相关的条件编译。

2.符号版本处理

- **有符号：**

如果 ELF 中存在 DT_VERSYM，说明该对象使用了 symbol versioning。

此时每个符号都有版本索引：

```c
        /*
         * 如果存在符号版本表，则处理带版本约束的普通重定位。
         */
        if (map->l_info[VERSYMIDX (DT_VERSYM)])
          {
            /*
             * 获取符号版本表。
             */
            const ElfW(Half) *const version =
              (const void *) D_PTR (map, l_info[VERSYMIDX (DT_VERSYM)]);

            /*
             * 遍历普通重定位项。
             */
            for (; r < end; ++r)
              {
                /*
                 * 获取符号版本索引。这里 & 0x7fff 是为了去掉最高位标志位。
                 */
                ElfW(Half) ndx = version[ELFW(R_SYM) (r->r_info)] & 0x7fff;

                /*
                 * 获取符号表项。
                 */
                const ElfW(Sym) *sym = &symtab[ELFW(R_SYM) (r->r_info)];

                /*
                 * 计算重定位目标地址。
                 */
                void *const r_addr_arg = (void *) (l_addr + r->r_offset);

                /*
                 * 获取符号版本信息。
                 */
                const struct r_found_version *rversion = &map->l_versions[nd
x];

#if defined ELF_MACHINE_IRELATIVE
                /*
                 * IRELATIVE 延后处理。
                 *
                 * 因为 IRELATIVE 可能调用 resolver，
                 * resolver 可能依赖其他符号和数据已经完成重定位。
                 */
                if (ELFW(R_TYPE) (r->r_info) == ELF_MACHINE_IRELATIVE)
```

```
                {
                  if (r2 == NULL)
                    r2 = r;
                  end2 = r;
                  continue;
                }
#endif

              /*
               * 执行普通重定位。
               */
              elf_machine_rel (map, scope, r, sym, rversion, r_addr_arg,
                               skip_ifunc);

.......
        }
```

这样符号解析时不仅匹配符号名，还会匹配符号版本。

例如：

```
memcpy@GLIBC_2.2.5
memcpy@@GLIBC_2.14
```

两者可能对应不同版本要求。

- 无符号版本

如果没有 DT_VERSYM，则版本信息传 NULL：

```
elf_machine_rel (map, scope, r, sym, NULL, r_addr_arg,
                                 skip_ifunc);
```

3.audit机制

代码中有：

```c
#if defined SHARED
                /*
                 * audit 机制:
                 *
                 * 如果当前重定位是 JMP_SLOT，并且启用了 audit 模块，
                 * 则触发符号绑定审计回调。
                 *
                 * JMP_SLOT 通常对应 PLT 函数调用项。
                 */
                if (ELFW(R_TYPE) (r->r_info) == ELF_MACHINE_JMP_SLOT
                    && GLRO(dl_naudit) > 0)
                  {
                    /*
                     * 查找该符号最终绑定到哪个 link_map。
                     */
                    struct link_map *sym_map
                      = RESOLVE_MAP (map, scope, &sym, rversion,
                                     ELF_MACHINE_JMP_SLOT);

                    /*
                     * 如果符号存在，则通知 audit 模块。
                     */
                    if (sym != NULL)
                      _dl_audit_symbind (map, NULL, r, sym, r_addr_arg, sym_
map,
                                         false);
                  }
#endif
              }
```

这是动态链接器的 audit 机制。

当启用 audit，例如通过：`LD_AUDIT=xxx.so`

动态链接器可以在符号绑定时通知 audit 模块。

这常用于：

- 调试
- 性能分析
- 安全监控
- 动态链接行为追踪

这里仅对：`ELF_MACHINE_JMP_SLOT`，即函数 PLT 重定位执行 audit 回调。

5.elf_dynamic_do_Rel整体执行流程总结：

```
输入：一个 ELF 对象的重定位表

1. 定位重定位表：
   - relative 指向起始
   - r 指向普通重定位开始
   - end 指向结束

2. 如果是 rtld bootstrap：
   2.1 先处理 RELATIVE
   2.2 再处理普通重定位
   2.3 返回

3. 如果不是 bootstrap：
   3.1 如果是 lazy 模式：
       - 遍历 PLT 重定位
       - 普通项调用 elf_machine_lazy_rel
       - IRELATIVE 延后处理

   3.2 如果不是 lazy 模式：
       - 先处理 RELATIVE
       - 如果存在符号版本表：
           - 带版本信息处理普通重定位
       - 否则：
           - 不带版本信息处理普通重定位
       - IRELATIVE 延后处理
       - JMP_SLOT 可触发 audit
```

3.5 PLT profiling / audit 支持

```c
/* 如果要做 profiling 或符号绑定审计，并且对象确实有 PLT 重定位项，
        则分配一个数组保存这些 PLT 重定位的解析结果。 */
     if ((consider_profiling || consider_symbind)
         && l->l_info[DT_PLTRELSZ] != NULL)
       {
         /* 根据 DT_PLTREL 判断每一项是 Rela 还是 Rel。 */
         size_t sizeofrel = l->l_info[DT_PLTREL]->d_un.d_val == DT_RELA
                            ? sizeof (ElfW(Rela))
                            : sizeof (ElfW(Rel));

         /* 计算 PLT 重定位项个数。 */
         size_t relcount = l->l_info[DT_PLTRELSZ]->d_un.d_val / sizeofrel;

         /* 为结果数组分配空间。 */
         l->l_reloc_result = calloc (sizeof (l->l_reloc_result[0]), relcount);

         if (l->l_reloc_result == NULL)
           {
             errstring = N_("\
%s: out of memory to store relocation results for %s\n");
             _dl_fatal_printf (errstring, RTLD_PROGNAME, l->l_name);
           }
       }
   }
```

当启用：gprof、audit（LD_AUDIT）

会保存：每个 PLT 绑定结果，用于分析 / hook

3.6 标记完成

防止重复执行

```c
    /* 标记该对象已经完成重定位。 */
    l->l_relocated = 1;
```

3.7 恢复 text 段权限

从RWX → RX

```
    /* 如果前面因为 DT_TEXTREL 修改过段权限，这里需要全部恢复。 */
    while (__builtin_expect (textrels != NULL, 0))
      {
        if (__mprotect (textrels->start, textrels->len, textrels->prot) < 0)
          {
            errstring = N_("cannot restore segment prot after reloc");
            goto call_error;
          }

  #ifdef CLEAR_CACHE
        /* 某些架构在修改可执行代码后，需要显式清理指令缓存。 */
        CLEAR_CACHE (textrels->start, textrels->start + textrels->len);
  #endif

        textrels = textrels->next;
      }
```

`CLEAR_CACHE` 必要原因：

代码被修改过，CPU 可能还在执行旧缓存

4._dl_init

```
136 #define RTLD_START asm ("\n\
137 .text\n\
138         .align 16\n\
139 .globl _start\n\
140 .globl _dl_start_user\n\
141 _start:\n\
142         mov %" RSP_LP ", %" RDI_LP "\n\
143         call _dl_start\n\
144 _dl_start_user:\n\
145         # Save the user entry point address in %r12.\n\
146         mov %" RAX_LP ", %" R12_LP "\n\
147         # Save %rsp value in %r13.\n\
148         mov %" RSP_LP ", % " R13_LP "\n\
149 "\
150         RTLD_START_ENABLE_X86_FEATURES \
151 "\
152         # Read the original argument count.\n\
153         mov (%rsp), %" RDX_LP "\n\
154         # Call _dl_init (struct link_map *main_map, int argc, char **argv, char **env)\n\
155         # argc -> rsi\n\
156         mov %" RDX_LP ", %" RSI_LP "\n\
157         # And align stack for the _dl_init call. \n\
158         and $-16, %" RSP_LP "\n\
159         # _dl_loaded -> rdi\n\
160         mov _rtld_local(%rip), %" RDI_LP "\n\
161         # env -> rcx\n\
162         lea 2*" LP_SIZE "(%r13,%rdx," LP_SIZE "), %" RCX_LP "\n\
163         # argv -> rdx\n\
164         lea " LP_SIZE "(%r13), %" RDX_LP "\n\
165         # Clear %rbp to mark outermost frame obviously even for constructors.\n\
166         xorl %ebp, %ebp\n\
167         # Call the function to run the initializers.\n\
168         call _dl_init\n\
169         # Pass our finalizer function to the user in %rdx, as per ELF ABI.\n\
170         lea _dl_fini(%rip), %" RDX_LP "\n\
171         # And make sure %rsp points to argc stored on the stack.\n\
172         mov %" R13_LP ", %" RSP_LP "\n\
173         # Jump to the user's entry point.\n\
174         jmp *%r12\n\
175 .previous\n\
176 ");
```

按照 x86-64 System V ABI，前四个参数分别放在：

参数	寄存器
第 1 个参数	`%rdi`
第 2 个参数	`%rsi`
第 3 个参数	`%rdx`
第 4 个参数	`%rcx`

所以这里要构造：

- rdi = main_map

- rsi = argc
- rdx = argv
- rcx = envp

所以等价于：`_dl_init(main_map, argc, argv, envp);`

_dl_init() 属于动态链接器 ld.so 的初始化流程。它主要负责在主程序正式运行前，执行动态链接阶段需要完成的初始化任务。

```c
void
_dl_init (struct link_map *main_map, int argc, char **argv, char **env)
{
  /*
   * main_map 表示主程序对应的 link_map。
   *
   * l_info 是动态段信息数组，里面保存了 ELF Dynamic Section 中
   * 各类动态标签对应的 ElfW(Dyn) 结构。
   *
   * DT_PREINIT_ARRAY:
   *   主程序的 .preinit_array 段地址。
   *
   * DT_PREINIT_ARRAYSZ:
   *   .preinit_array 段大小，单位是字节。
   *
   * .preinit_array 中的函数会在普通构造函数之前执行，
   * 并且只对主程序有意义，动态库一般不使用 preinit_array。
   */
  ElfW(Dyn) *preinit_array = main_map->l_info[DT_PREINIT_ARRAY];
  ElfW(Dyn) *preinit_array_size = main_map->l_info[DT_PREINIT_ARRAYSZ];

  unsigned int i;
```

它的主要工作可以分为三类:

4.1 调用 INITFIRST 动态库

```c
/*
 * GL(dl_initfirst) 用于处理带有 DF_1_INITFIRST 标志的共享库。
 *
 * 如果某个动态库被标记为 INITFIRST，
 * 那么它的初始化函数需要优先于其他库执行。
 */
if (__glibc_unlikely (GL(dl_initfirst) != NULL))
  {
    /*
     * 调用该对象的初始化函数。
     * 也就是执行它的 DT_INIT 和 DT_INIT_ARRAY。
     */
    call_init (GL(dl_initfirst), argc, argv, env);

    /*
     * 执行过之后清空，防止重复调用。
     */
    GL(dl_initfirst) = NULL;
  }
```

如果某个共享库带有特殊标志，要求它的初始化函数优先执行，那么动态链接器会把它记录到：`GL(dl_initfirst)`，然后在 _dl_init() 早期调用它的构造函数。

4.1.1 call_init

call_init() 用来初始化单个共享对象。它会处理这个对象的：

- DT_INIT
- DT_INIT_ARRAY

DT_INIT、DT_INIT_ARRAY是 ELF dynamic section 中的一个动态标签。

DT_INIT_ARRAY 是 .dynamic 中指向 .init_array 的描述项，.init_array 是实际存放函数指针的 section，告诉动态链接器或 libc：.init_array 在内存中的地址在哪里。

DT_INIT 指向 .init，保存老式初始化函数代码，现代程序更多依赖DT_INIT_ARRAY，但为了兼容，glibc 仍然会处理 DT_INIT。

1. 防止代理对象重复初始化

如果当前 link_map 不是实际对象，而只是代理对象，那么不执行构造函数。

```c
static void
call_init (struct link_map *l, int argc, char **argv, char **env)
{
  /*
   * 如果这是一个代理对象，则不执行构造函数。
   *
   * l_real 指向真实对象。
   * 如果 l != l->l_real，说明当前 link_map 不是实际要初始化的对象。
   */
  if (l != l->l_real)
    return;
```

2.确认对象已经重定位完成

共享库中的函数地址、全局变量地址等通常依赖重定位。如果还没有完成重定位就执行构造函数，那么函数指针可能是错误的。所以在调用构造函数前，需要确保对象已经完成 relocation。

```c
  /*
   * 如果对象还没有完成重定位，那么它里面的函数指针可能还不可用。
   *
   * 对共享库来说，必须先完成 relocation，才能调用构造函数。
   *
   * 可执行文件 lt_executable 是例外，因为主程序可能不需要同样的重定位处理。
   */
  assert (l->l_relocated || l->l_type == lt_executable);
```

3.防止重复执行构造函数

每个共享对象的初始化函数应该只执行一次。

这里先检查 `l->l_init_called` ，如果已经调用过，直接返回。如果没有调用过，则立即设置 `l->l_init_called = 1;` 这个标记是在真正调用构造函数之前设置的。这样可以处理循环依赖场景。例如：libA 初始化时触发 libB，libB 初始化时又触发 libA，如果不提前设置 l_init_called，就可能递归调用初始化函数，导致死循环。

```c
    /*
     * 如果这个对象的初始化函数已经被调用过，直接返回。
     *
     * 这可以避免重复执行构造函数。
     */
    if (l->l_init_called)
      return;

    /*
     * 先标记为已经初始化。
     *
     * 注意这里是在真正调用构造函数之前就设置。
     *
     * 这样做是为了处理循环依赖。
     * 如果 A 初始化时又间接触发 A 的初始化，就不会无限递归。
     */
    l->l_init_called = 1;
```

4.跳过主可执行文件

这段逻辑表示：

如果当前对象是主程序本身，则在这里直接返回。

原因是：_dl_init 主要负责动态链接器阶段的初始化，尤其是共享库的构造函数。主程序的初始化会在后续 `__libc_start_main` 路径中继续处理。因此，call_init() 这里主要针对共享库。

```c
/*
 * 检查是否是主程序本身。
 *
 * 对于主程序：
 *
 *   l->l_name[0] == '\0'
 *   l->l_type == lt_executable
 *
 * 这里直接 return，不在动态链接器里运行主程序的 init。
 *
 * 注意：
 *   主程序的某些初始化逻辑通常会由后续 __libc_start_main
 *   或启动代码路径处理。
 */
if (__builtin_expect (l->l_name[0], 'a') == '\0'
    && l->l_type == lt_executable)
  return;
```

5.调用 DT_INIT

DT_INIT 通常对应老式的 .init 段。它是一个函数入口地址。调用时需要加上对象的加载基址：

`运行时地址 = l->l_addr + d_un.d_ptr`，然后动态链接器调用该初始化函数。

```c
    /*
     * 如果开启了 LD_DEBUG=libs 或相关调试选项,
     * 打印当前正在调用哪个对象的 init。
     */
    if (__glibc_unlikely (GLRO(dl_debug_mask) & DL_DEBUG_IMPCALLS))
      _dl_debug_printf ("\ncalling init: %s\n\n",
                        DSO_FILENAME (l->l_name));

    /*
     * 执行传统的 DT_INIT 初始化函数。
     *
     * ELF 中有两种初始化机制:
     *
     *   1. DT_INIT
     *      通常对应老式的 .init 段。
     *
     *   2. DT_INIT_ARRAY
     *      通常对应 .init_array 段。
     *
     * ELF_INITFINI 用来判断当前平台/配置是否启用 DT_INIT/DT_FINI。
     */
    if (ELF_INITFINI && l->l_info[DT_INIT] != NULL)
      DL_CALL_DT_INIT(l,
                      l->l_addr + l->l_info[DT_INIT]->d_un.d_ptr,
                      argc,
                      argv,
                      env);
```

6.调用 DT_INIT_ARRAY

.init_array 是现代 ELF 中更常见的初始化机制。

例如:

```c
__attribute__((constructor))
static void init_func(void)
{
}
```

或者 C++ 全局对象构造函数，通常都会进入 .init_array。call_init() 会按照数组顺序依次调用这些函数。

```c
    /*
     * 下面处理 DT_INIT_ARRAY，也就是 .init_array。
     */
    ElfW(Dyn) *init_array = l->l_info[DT_INIT_ARRAY];

    if (init_array != NULL)
      {
        unsigned int j;
        unsigned int jm;
        ElfW(Addr) *addrs;

        /*
         * DT_INIT_ARRAYSZ 记录 .init_array 的总字节数。
         *
         * 每个元素是一个函数地址 ElfW(Addr)，
         * 因此元素个数为：
         *
         *   总字节数 / sizeof(ElfW(Addr))
         */
        jm = l->l_info[DT_INIT_ARRAYSZ]->d_un.d_val / sizeof (ElfW(Addr));

        /*
         * 计算 .init_array 在运行时的实际地址。
         *
         * init_array->d_un.d_ptr 是 ELF 中记录的地址，
         * l->l_addr 是该共享对象的加载基址。
         */
        addrs = (ElfW(Addr) *) (init_array->d_un.d_ptr + l->l_addr);

        /*
         * 按顺序调用 .init_array 中的每个初始化函数。
         *
         * C++ 全局对象构造函数、
         * GCC constructor 属性函数，
         * 通常都会被放入 .init_array。
         */
        for (j = 0; j < jm; ++j)
          ((dl_init_t) addrs[j]) (argc, argv, env);
      }
  }
```

4.2 调用主程序的 .preinit_array

.preinit_array 保存最早执行的一组函数指针。这些函数会在普通构造函数之前执行。

其执行顺序大致是：

```
main 的 .preinit_array
  |
  v
依赖库的 DT_INIT / DT_INIT_ARRAY
  |
  v
后续进入 _start / __libc_start_main
```

.preinit_array 通常只对主程序有意义，动态库一般不使用它。

```c
  /*
   * 如果主程序没有 .preinit_array，就不做任何事情。
   *
   * __builtin_expect(expr, expected) 是编译器分支预测提示。
   * 这里表示 preinit_array != NULL 通常不成立。
   */
  if (__builtin_expect (preinit_array != NULL, 0)
      && preinit_array_size != NULL
      && (i = preinit_array_size->d_un.d_val / sizeof (ElfW(Addr))) > 0)
    {
      ElfW(Addr) *addrs;
      unsigned int cnt;

      /*
       * 如果开启了动态链接器调试输出，则打印将要调用 preinit 的信息。
       */
      if (__glibc_unlikely (GLRO(dl_debug_mask) & DL_DEBUG_IMPCALLS))
        _dl_debug_printf ("\ncalling preinit: %s\n\n",
                          DSO_FILENAME (main_map->l_name));

      /*
       * preinit_array->d_un.d_ptr 是 ELF 中记录的虚拟地址或相对地址。
       *
       * main_map->l_addr 是该对象被加载到内存后的基址偏移。
       *
       * 二者相加得到运行时实际地址。
       */
      addrs = (ElfW(Addr) *) (preinit_array->d_un.d_ptr + main_map->l_addr
);

      /*
       * 依次调用 .preinit_array 中的每个函数。
       *
       * 函数原型类似：
       *
       *   void func(int argc, char **argv, char **envp);
       *
       * 这些函数在所有普通构造函数之前执行。
       */
      for (cnt = 0; cnt < i; ++cnt)
        ((dl_init_t) addrs[cnt]) (argc, argv, env);
    }
```

4.3 调用共享库构造函数

这里通过 main_map->l_initfini[] 遍历需要初始化的对象。

它会从后往前调用：`call_init(...)`，这样做是为了满足 ELF 初始化顺序要求：被依赖的库要先初始化，依赖它的库后初始化。

```c
/*
 * 下面开始调用各个共享对象的构造函数。
 *
 * ELF 规范要求：
 *
 *   一个对象的所有依赖库的构造函数，
 *   必须先于该对象自己的构造函数执行。
 *
 * 例如：
 *
 *   main 依赖 libA
 *   libA 依赖 libB
 *
 * 那么初始化顺序应当类似：
 *
 *   libB -> libA -> main
 *
 * 循环依赖的情况未严格规定。
 */

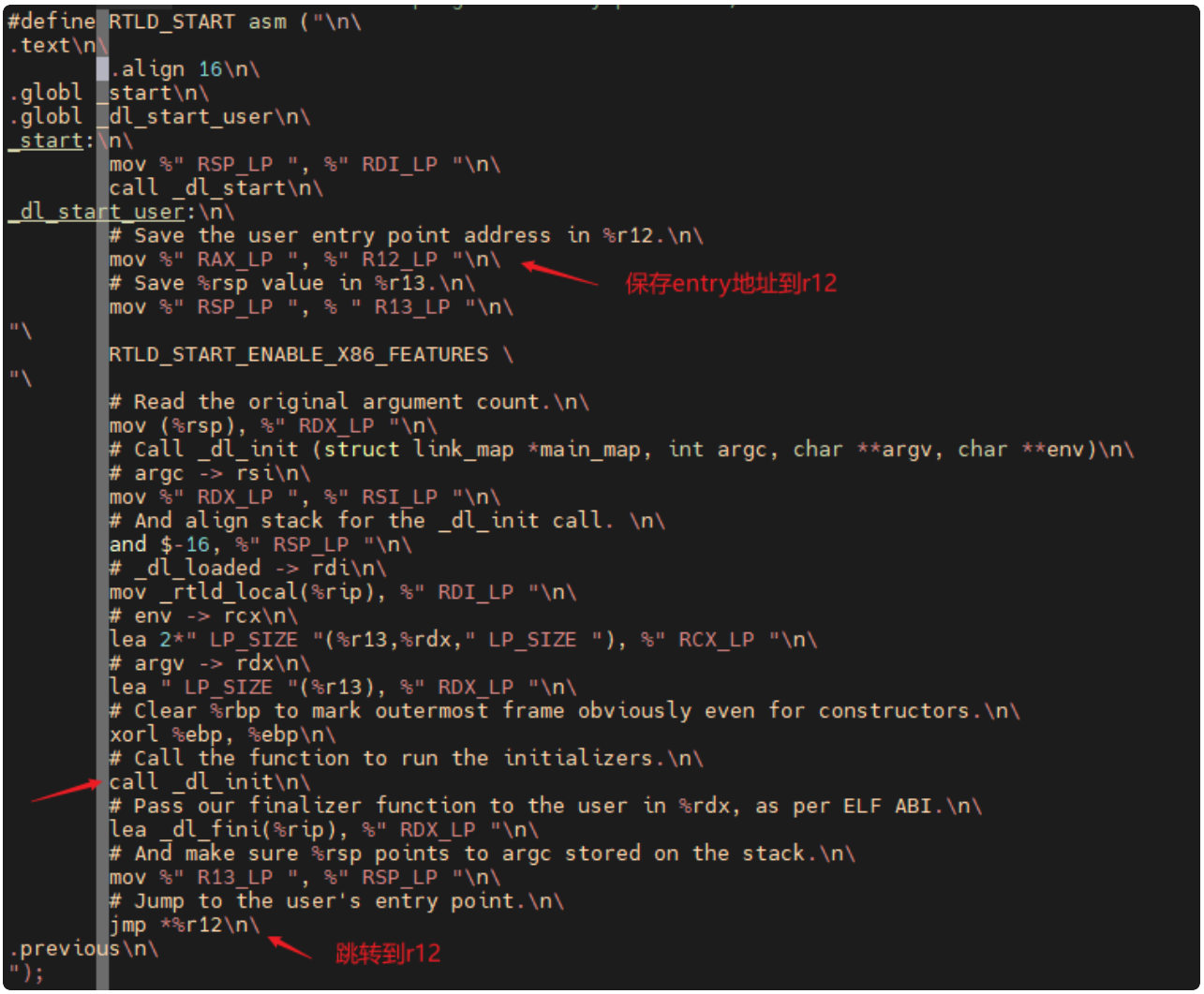
/*
 * main_map->l_searchlist 是主程序的依赖搜索列表。
 *
 * r_nlist 表示这个列表中有多少个对象。
 *
 * l_initfini[] 是经过依赖关系排序后的初始化/终结列表。
 */
i = main_map->l_searchlist.r_nlist;

/*
 * 从后往前调用 call_init。
 *
 * 这样可以保证依赖项先初始化，被依赖者后初始化。
 *
 * 例如列表中可能按某种搜索顺序保存：
 *
 *   main, libA, libB
 *
 * 反向调用后就是：
 *
 *   libB, libA, main
 */
while (i-- > 0)
  call_init (main_map->l_initfini[i], argc, argv, env);

```

```
#ifndef HAVE_INLINED_SYSCALLS
  /*
   * 表示动态链接器启动阶段结束。
   */
  _dl_starting_up = 0;
#endif
}
```

5._start



当动态链接器完成加载、重定位和 _dl_init() 后，它会跳转到主程序的 ELF 入口地址。这个入口地址通常是：`_start`

主要做的事情：

- 清空 rbp
- 取出 argc

- 计算 argv
- 保存 rtld_fini
- 对齐栈
- 设置 main 地址
- 调用 __libc_start_main

```c
#include <sysdep.h>

/*
 * ENTRY(_start)
 *
 * 定义程序入口符号 _start。
 *
 * 对于 ELF 可执行程序来说，真正的入口点不是 main，
 * 而是 ELF Header 中 e_entry 指向的地址。
 *
 * 在 glibc 动态链接程序中，这个入口通常就是 _start。
 */
ENTRY (_start)

        /*
         * Clearing frame pointer is insufficient, use CFI.
         *
         * CFI: Call Frame Information。
         *
         * cfi_undefined(rip) 表示在当前最外层栈帧中，
         * 返回地址寄存器 rip 是未定义的。
         *
         * 因为 _start 是用户程序执行的最外层入口，
         * 它不是被普通 call 调用进来的，
         * 所以不存在正常意义上的返回地址。
         */
        cfi_undefined (rip)

        /*
         * 清空帧指针 rbp。
         *
         * ABI 建议在最外层函数中将 frame pointer 清零，
         * 这样调试器或 unwind 逻辑可以明显识别出：
         *
         *   这里是调用栈的最外层。
         */
        xorl %ebp, %ebp

        /*
         * 下面开始从初始栈中取出内核传递给程序的参数，
         * 并按照 x86-64 System V ABI 的调用约定，
         * 准备传给 __libc_start_main 的参数。
         *
         * __libc_start_main 原型大致如下：
         *
```

```
         * int __libc_start_main(
         *     int (*main)(int, char **, char **),
         *     int argc,
         *     char **argv,
         *     void (*init)(void),
         *     void (*fini)(void),
         *     void (*rtld_fini)(void),
         *     void *stack_end
         * );
         *
         * x86-64 ABI 下，前 6 个参数通过寄存器传递：
         *
         *   main       -> %rdi
         *   argc       -> %rsi
         *   argv       -> %rdx
         *   init       -> %rcx     旧式主程序初始化函数,现代 x86-64 glibc 中通常由 _start 传 NULL。
         *   fini       -> %r8
         *   rtld_fini  -> %r9
         *
         * 第 7 个参数 stack_end 通过栈传递。     旧式终结函数,现代 x86-64 glibc 中通常由 _start 传 NULL。
         */

        /*
         * 程序进入 _start 时，%rdx 中保存的是动态链接器传来的
         * rtld_fini 函数地址。
         *
         * rtld_fini 用于程序退出时执行动态链接器相关的清理逻辑，
         * 例如调用共享库的析构函数等。
         *
         * 因为 __libc_start_main 的第 6 个参数通过 %r9 传递，
         * 所以这里把 %rdx 移到 %r9。
         */
        mov %RDX_LP, %R9_LP     /* Address of the shared library termination
                                   function.  */

#ifdef __ILP32__

        /*
         * x86-64 ILP32 ABI 特殊处理。
         *
         * ILP32 表示：
         *   int、long、pointer 都是 32 位，
         *   但运行在 x86-64 指令集上。
```

```
91            *
92            * 这里从栈顶读取 4 字节 argc 到 esi。
93            */
94           mov (%rsp), %esi        /* Simulate popping 4-byte argument coun
      t.  */
95
96           /*
97            * 栈指针向上移动 4 字节，
98            * 相当于弹出了 argc。
99            */
100          add $4, %esp
101
102  #else
103
104          /*
105           * 普通 x86-64 LP64 ABI。
106           *
107           * 初始栈顶保存 argc。
108           *
109           * popq %rsi 做两件事：
110           *
111           *   1. 将栈顶的 argc 弹出到 %rsi
112           *   2. %rsp 向上移动 8 字节
113           *
114           * 由于 __libc_start_main 的第 2 个参数 argc
115           * 通过 %rsi 传递，所以这里正好把 argc 放入 %rsi。
116           */
117          popq %rsi               /* Pop the argument count.  */
118
119  #endif
120
121          /*
122           * 此时 %rsp 指向 argv[0]。
123           *
124           * 初始栈布局大致如下：
125           *
126           *   argc
127           *   argv[0]
128           *   argv[1]
129           *   ...
130           *   argv[argc - 1]
131           *   NULL
132           *   envp[0]
133           *   envp[1]
134           *   ...
135           *   NULL
136           *   auxv[]
137           *
```

```
         * pop argc 后，当前栈顶就是 argv。
         *
         * __libc_start_main 的第 3 个参数 argv
         * 通过 %rdx 传递，
         * 所以这里将当前 %rsp 放入 %rdx。
         */
        mov %RSP_LP, %RDX_LP

        /*
         * 按照 x86-64 ABI 要求，
         * 函数调用时栈需要保持 16 字节对齐。
         *
         * 这里将 %rsp 向下对齐到 16 字节边界。
         */
        and  $~15, %RSP_LP

        /*
         * 压入一个无意义的值。
         *
         * 这么做是为了后续再 push 8 字节后，
         * 仍然满足函数调用时的栈对齐要求。
         *
         * 这里 push 的 %rax 内容本身不重要。
         */
        pushq %rax

        /*
         * 将当前栈顶地址压栈，作为 __libc_start_main 的第 7 个参数：
         *
         *   stack_end
         *
         * 因为 x86-64 前 6 个参数用寄存器传递，
         * 第 7 个参数需要通过栈传递。
         *
         * stack_end 表示用户程序可见的最高栈地址附近的位置，
         * libc 后续可能会保存它，用于栈相关检查或内部初始化。
         */
        pushq %rsp

        /*
         * 以前这里会传入 .fini 和 .init 的地址。
         *
         * 新版本 glibc 中，这两个参数通常置为 NULL。
         *
         * __libc_start_main 的第 5 个参数 fini 使用 %r8，
         * 第 4 个参数 init 使用 %rcx。
         *
         * 所以：
```

```
186             *
187             *   %r8  = 0
188             *   %rcx = 0
189             */
190            xorl %r8d, %r8d
191            xorl %ecx, %ecx
192
193    #ifdef PIC
194
195            /*
196             * 如果是位置无关代码 PIC/PIE,
197             * main 的地址需要通过 GOT 获取。
198             *
199             * main@GOTPCREL(%rip):
200             *   使用 RIP-relative addressing 从 GOT 中取出 main 的地址。
201             *
202             * __libc_start_main 的第 1 个参数 main
203             * 通过 %rdi 传递,
204             * 所以这里将 main 地址放入 %rdi。
205             */
206            mov main@GOTPCREL(%rip), %RDI_LP
207
208    #else
209
210            /*
211             * 非 PIC 情况下, main 的地址可以作为立即数直接放入 %rdi。
212             */
213            mov $main, %RDI_LP
214
215    #endif
216
217            /*
218             * 调用 __libc_start_main。
219             *
220             * 注意:
221             *   _start 并不直接调用 main。
222             *
223             * 而是把 main 函数地址、argc、argv、rtld_fini、stack_end 等
224             * 参数传给 __libc_start_main。
225             *
226             * __libc_start_main 会负责:
227             *
228             *   1. 初始化 libc
229             *   2. 初始化线程/TLS/栈保护等运行时环境
230             *   3. 调用主程序 init_array
231             *   4. 调用用户 main
232             *   5. main 返回后调用 exit
233             *
```

```
234             * 这里通过 GOT 间接调用 __libc_start_main，
235             * 避免走额外的 PLT 跳转。
236             *
237             * 对静态可执行文件，较新的链接器可能会把这个间接调用
238             * 优化成直接调用。
239             */
240            call *__libc_start_main@GOTPCREL(%rip)
241
242            /*
243             * 理论上 __libc_start_main 不应该返回。
244             *
245             * 因为 main 返回后，__libc_start_main 会调用 exit，
246             * exit 最终会终止进程。
247             *
248             * 如果由于某种异常原因 __libc_start_main 返回了，
249             * 执行 hlt 指令让程序崩溃。
250             */
251            hlt                     /* Crash if somehow `exit' does return.
    */
252
253    END (_start)
```

5.1 _start 如何准备 __libc_start_main 参数

x86-64 System V ABI 规定，函数前 6 个参数通过寄存器传递：

```
1   第 1 个参数 -> rdi
2   第 2 个参数 -> rsi
3   第 3 个参数 -> rdx
4   第 4 个参数 -> rcx
5   第 5 个参数 -> r8
6   第 6 个参数 -> r9
```

__libc_start_main 的参数对应关系是：

```
main       -> rdi
argc       -> rsi
argv       -> rdx
init       -> rcx
fini       -> r8
rtld_fini  -> r9
stack_end  -> 栈上传递
```

对应汇编逻辑如下：

```
 mov %RDX_LP, %R9_LP
 popq %rsi
 mov %RSP_LP, %RDX_LP
 and $~15, %RSP_LP
 pushq %rax
 pushq %rsp
 xorl %r8d, %r8d
 xorl %ecx, %ecx
 mov main@GOTPCREL(%rip), %RDI_LP
 call *__libc_start_main@GOTPCREL(%rip)
```

含义如下：

```
%r9  = rtld_fini
%rsi = argc
%rdx = argv
%r8  = NULL
%rcx = NULL
%rdi = main
stack = stack_end
```

所以 _start 的核心职责是：把内核和动态链接器传来的底层启动信息，整理成 __libc_start_main 能理解的参数格式。

6.__libc_start_main

__libc_start_main 负责完成 libc 运行时初始化。

6.1动态链接程序

动态链接程序已经由 ld-linux.so 做过很多早期工作：

- 加载 libc
- 解析 auxv
- 完成重定位
- 初始化 TLS
- 运行共享库构造函数
- 安全检查

所以在 SHARED 场景下，__libc_start_main 重点做：

- 注册 rtld_fini
- 初始化主程序自身
- 调用 main

6.2静态链接程序

静态链接程序没有独立的动态链接器帮忙。所以 __libc_start_main 需要自己完成更多工作：

- 设置 __environ
- 保存 __libc_stack_end
- 解析 auxv
- 初始化 tunables
- 初始化 CPU features
- static PIE 自重定位
- 初始化 TLS
- 设置 canary
- 初始化 pthread
- 注册 fini
- 安全检查标准 fd

6.1 静态链接路径详解：#ifndef SHARED

6.1.1 根据 argv 找到环境变量 envp

```c
/*
 * STATIC、LIBC_START_MAIN 都是 glibc 内部宏。
 *
 * LIBC_START_MAIN 在不同编译配置下可能展开为：
 *
 *   __libc_start_main_impl
 *
 * 最终对外导出的符号通常仍然是：
 *
 *   __libc_start_main
 *
 * _start 汇编代码中调用的：
 *
 *   call *__libc_start_main@GOTPCREL(%rip)
 *
 * 最终进入的就是这里。
 */
STATIC int
LIBC_START_MAIN (int (*main) (int, char **, char ** MAIN_AUXVEC_DECL),
                 int argc, char **argv,
#ifdef LIBC_START_MAIN_AUXVEC_ARG
                 ElfW(auxv_t) *auxvec,
#endif
                 __typeof (main) init,
                 void (*fini) (void),
                 void (*rtld_fini) (void), void *stack_end)
{
#ifndef SHARED

  /*
   * 非 SHARED 情况，主要对应静态链接 libc 的程序。
   *
   * 初始栈布局大致是：
   *
   *   argc
   *   argv[0]
   *   argv[1]
   *   ...
   *   argv[argc - 1]
   *   NULL
   *   envp[0]
   *   envp[1]
   *   ...
   *   NULL
   *   auxv[]
```

```
     *
     * argv[argc] 是 NULL,
     * 所以 &argv[argc + 1] 就是 envp，也就是环境变量数组起始地址。
     */
    char **ev = &argv[argc + 1];

    /*
     * 设置 libc 内部的全局环境变量指针。
     *
     * __environ 指向 envp。
     *
     * 后续 main 的第三个参数 envp,
     * 以及 getenv 等 libc 函数，都会依赖这个环境变量指针。
     */
    __environ = ev;
```

6.1.2 保存栈边界地址

```c
/*
  * 保存最低或边界栈地址。
  *
  * 对静态链接程序，这里由 __libc_start_main 保存。
  *
  * 对动态链接程序，这件事通常已经在动态链接器 ld.so 中完成。
  */
 __libc_stack_end = stack_end;
```

stack_end 是 _start 传进来的初始栈相关地址。

libc 保存它用于后续运行时管理。

动态链接程序中，这件事通常由动态链接器完成。

6.1.3 解析辅助向量 auxv

```c
# ifdef HAVE_AUX_VECTOR

  /*
   * 处理辅助向量 auxv。
   *
   * auxv 是内核传给用户程序的一组键值对
   *
   * libc 和动态链接器会依赖 auxv 获取：
   *
   *   - 程序头表位置
   *   - TLS 信息
   *   - 随机数种子
   *   - CPU/平台信息
   *   - page size
   *   - secure execution 标志
   */

#  ifndef LIBC_START_MAIN_AUXVEC_ARG

  /*
   * 如果 auxvec 不是作为参数直接传进来的，
   * 那么需要从初始栈中手动找到 auxv。
   */
  ElfW(auxv_t) *auxvec;
  {
    char **evp = ev;

    /*
     * envp 是以 NULL 结尾的字符串指针数组。
     *
     * 从 envp 开始向后扫描，直到遇到 NULL。
     */
    while (*evp++ != NULL)
      ;

    /*
     * envp 结束后的下一个位置就是 auxv。
     */
    auxvec = (ElfW(auxv_t) *) evp;
  }
#  endif

  /*
   * 初始化动态链接器/运行时所需的 auxv 信息。
   *
```

```
     * 例如解析 AT_PHDR、AT_PHNUM、AT_RANDOM 等。
     */
    _dl_aux_init (auxvec);
# endif
```

auxv 是内核传给用户态的重要信息。

典型内容包括：

```
AT_PHDR       程序头表地址
AT_PHNUM      程序头数量
AT_ENTRY      程序入口地址
AT_RANDOM     随机数地址
AT_PAGESZ     页大小
AT_UID        用户 ID
AT_EUID       有效用户 ID
AT_SECURE     secure execution 标志
AT_PLATFORM   平台字符串
AT_HWCAP      CPU 能力位
```

_dl_aux_init(auxvec) 会解析这些信息，为后续初始化做准备。

其中非常重要的是 `AT_RANDOM`，它会用于生成 `stack canary`、`pointer guard`

6.1.4 初始化 glibc tunables

```
    /*
     * 初始化 glibc tunables。
     */
    __tunables_init (__environ);
```

tunables 是 glibc 的运行时调优机制。

它可以通过环境变量控制某些 glibc 行为，例如 malloc、cpu 特性相关策略等。

这一步需要尽早执行，因为后续 CPU feature、IFUNC resolver 可能依赖这些配置。

6.1.5 初始化 CPU features

```c
  /*
    * 架构相关的 CPU feature 初始化。
    *
    * 例如 x86 上可能会检测 AVX、SSE、ERMS、BMI 等能力。
    *
    * 后续 IFUNC resolver 可能依赖这些信息选择最优实现。
    */
   ARCH_INIT_CPU_FEATURES ();
```

这是架构相关初始化。

以 x86 为例，glibc 可能检查：

```c
SSE
AVX
AVX2
AVX512
ERMS
BMI
FMA
CPU vendor
CPU model
```

这些信息会影响 glibc 中某些函数的具体实现选择。

例如：

```c
memcpy
memmove
strlen
strcmp
```

glibc 可能会根据 CPU 能力选择更优版本。

6.1.6 static PIE 自重定位

```c
  /*
    * 对 static PIE 做自重定位。
    *
    * static PIE 是静态链接但位置无关的可执行文件。
    *
    * 在执行更复杂的 libc 代码前，需要先完成自身重定位。
    *
    * 注释中提到：
    *
    *   这一步要在 tunables 和 cpu features 初始化之后，
    *   因为 IFUNC resolver 可能依赖这些信息。
    *
    * 在此之前应尽量避免访问需要重定位的内容。
    */
    _dl_relocate_static_pie ();
```

static PIE 是：静态链接 + 位置无关可执行。

它没有独立动态链接器帮它重定位，所以 libc 启动代码需要自己完成重定位。

6.1.7 IFUNC / IREL 重定位

IFUNC 是 glibc 中非常重要的机制。

它允许一个函数在运行时选择具体实现。

例如：

```c
memcpy
  |
  |-- 普通版本
  |-- SSE 版本
  |-- AVX 版本
  |-- AVX512 版本
```

程序启动时通过 resolver 判断当前 CPU 支持什么，然后把函数地址绑定到最佳实现。IREL / IRELA 就与这种间接函数重定位有关。

```c
    /*
     * 执行 IREL / IRELA 重定位。
     *
     * IREL 通常与 IFUNC 机制有关。
     *
     * IFUNC 可以根据 CPU 能力在运行时选择函数实现，
     * 比如 memcpy 选择 AVX 版本还是普通版本。
     */
    ARCH_SETUP_IREL ();

    /*
     * 初始化 TLS。
     *
     * TLS: Thread Local Storage，线程局部存储。
     *
     * 栈保护 canary 等信息可能要存入 TCB，
     * 所以 TLS 需要尽早初始化。
     */
    ARCH_SETUP_TLS ();

    /*
     * 某些架构希望在 TLS 初始化后再执行 IREL/IRELA。
     *
     * 因为 IFUNC resolver 可能需要访问 TCB 中的信息。
     *
     * 例如 powerpc 的 hwcap、platform 字段可能放在 TCB 里。
     */
    ARCH_APPLY_IREL ();
```

6.1.8 设置 stack canary

tack canary 用于栈溢出保护。

当编译器开启：

```c
  -fstack-protector
  -fstack-protector-strong
```

后，函数栈帧中会放置 canary。

函数返回前检查 canary 是否被破坏。

如果被破坏，说明可能发生了栈溢出攻击，程序会终止。

canary 的随机性通常来自：`AT_RANDOM -> _dl_random`

```c
  /*
   * 设置栈保护 canary。
   *
   * _dl_random 通常来自 auxv 中的 AT_RANDOM。
   *
   * 编译器开启 stack protector 后，
   * 函数会检查栈上的 canary 是否被破坏，
   * 用于检测栈溢出攻击。
   */
  uintptr_t stack_chk_guard = _dl_setup_stack_chk_guard (_dl_random);

# ifdef THREAD_SET_STACK_GUARD

  /*
   * 如果架构/线程库支持把 stack guard 放进 TCB，
   * 就写入线程控制块。
   */
  THREAD_SET_STACK_GUARD (stack_chk_guard);

# else

  /*
   * 否则写入全局变量 __stack_chk_guard。
   */
  __stack_chk_guard = stack_chk_guard;

# endif
```

6.1.9 初始化 pthread 最小环境 + 设置 pointer guard

```c
/*
   * 如果静态链接进了 libpthread，
   * 这里执行最小 pthread 初始化。
   *
   * 这是为了在线程相关功能可用前，先完成必要的运行时准备。
   */
  if (__pthread_initialize_minimal != NULL)
    __pthread_initialize_minimal ();

  /*
   * 设置 pointer guard。
   *
   * pointer guard 用于指针混淆/保护，
   * 例如保护某些函数指针、longjmp buffer 等。
   *
   * 它通常也依赖 AT_RANDOM。
   */
  uintptr_t pointer_chk_guard = _dl_setup_pointer_guard (_dl_random,
                                                         stack_chk_guard);

# ifdef THREAD_SET_POINTER_GUARD

  /*
   * 如果支持线程级 pointer guard，
   * 则写入 TCB。
   */
  THREAD_SET_POINTER_GUARD (pointer_chk_guard);

# else

  /*
   * 否则保存到本地全局变量。
   */
  __pointer_chk_guard_local = pointer_chk_guard;

# endif
```

6.2 注册动态链接器析构函数 rtld_fini

静态和动态链接都会执行这一段：

```c
#endif /* !SHARED  */
  /*
   * 如果动态链接器传入了 rtld_fini,
   * 则把它注册到 __cxa_atexit。
   *
   * rtld_fini 是动态链接器的退出清理函数。
   *
   * 程序退出时，需要执行动态链接器相关清理,
   * 例如调用共享库析构函数、释放部分运行时状态等。
   *
   * 对应 _start 中:
   *
   *   mov %RDX_LP, %R9_LP
   *
   * 也就是动态链接器把 rtld_fini 放在 rdx,
   * _start 再转移到 r9 作为 __libc_start_main 的第 6 个参数。
   */
  if (__glibc_likely (rtld_fini != NULL))
    __cxa_atexit ((void (*) (void *)) rtld_fini, NULL, NULL);
```

rtld_fini 是谁传来的?

在 _start 中有：`mov %RDX_LP, %R9_LP`

进入 _start 时，动态链接器把自己的清理函数地址放在 %rdx。

而 __libc_start_main 的第 6 个参数通过 %r9 传递，所以 _start 将：`rdx -> r9`，也就是说`rtld_fini`来自动态链接器。

`__libc_start_main`将它注册到`__cxa_atexit`这样当程序退出时，可以调用动态链接器的清理函数。

退出时会处理：共享库析构函数动态链接器清理逻辑

6.3 静态链接下的 libc early init

这部分只在静态链接场景执行。

```c
#ifndef SHARED

  /*
   * 静态链接场景下执行 libc early init。
   *
   * 动态链接场景中，这通常已经由动态链接器更早调用。
   */
  __libc_early_init (true);

  /*
   * 调用 libc 的 first init。
   *
   * 静态链接时，没有动态链接器的 _dl_start_user
   * 帮忙运行相关构造逻辑，所以这里需要补上。
   */
  __libc_init_first (argc, argv, __environ);

  /*
   * 注册静态链接程序的析构函数。
   *
   * main 返回或 exit 时会调用.fini_array、析构函数、其他清理函数
   */
  __cxa_atexit (call_fini, NULL, NULL);

  /*
   * 安全检查。
   *
   * 对 SUID/SGID 等 secure execution 场景，
   * 需要确保标准文件描述符 0、1、2 已经打开。
   *
   * 动态链接程序中，这件事通常由动态链接器完成。
   * 静态链接程序需要 libc 在这里处理。
   */
  if (__builtin_expect (__libc_enable_secure, 0))
    __libc_check_standard_fds ();
```

- 0 stdin
- 1 stdout
- 2 stderr

6.4 动态链接场景下初始化主程序

这是动态链接程序进入 main 前的最后一段初始化逻辑。前面的 _dl_init/call_init 已经处理过共享库初始化。这里处理的是主程序自身的初始化逻辑。

```c
#ifdef SHARED

  /*
   * 动态链接场景。
   *
   * 如果开启了 LD_DEBUG 相关调试选项，
   * 打印即将初始化主程序的信息。
   */
  if (__builtin_expect (GLRO(dl_debug_mask) & DL_DEBUG_IMPCALLS, 0))
    GLRO(dl_debug_printf) ("\ninitialize program: %s\n\n", argv[0]);

  /*
   * 如果 init != NULL，说明这是 legacy program，
   * 由启动代码传入了一个自定义 init 函数。现代 x86-64 glibc 程序通常传入NULL，因此一般不会走这条 legacy init 路径。
   */
  if (init != NULL)
    /*
     * 调用 legacy init。
     *
     * 参数包括 argc、argv、environ，
     * 以及在某些配置下传入的 auxv。
     */
    (*init) (argc, argv, __environ MAIN_AUXVEC_PARAM);
  else
    /*
     * 当前程序通常走这里。
     *
     * 通过主程序的动态段查找构造函数，
     * 并调用主程序的 .init / .init_array。
     *
     * 注意：
     *   这里的 call_init 与动态链接器 dl-init.c 中
     *   初始化共享库的 call_init 名字相同/类似，
     *   但语境不同。
     *
     *   这里主要用于初始化“主程序自身”。
     */
    call_init (argc, argv, __environ);

  /*
   * audit 检查点。
   *
   * 动态链接器 auditing 机制可以在这里观察到：
   * 主程序初始化即将完成，控制流即将交给用户程序。
```

```
   */
  _dl_audit_preinit (GL(dl_ns)[LM_ID_BASE]._ns_loaded);

  /*
   * 如果开启调试输出，打印即将把控制权转交给主程序。
   */
  if (__glibc_unlikely (GLRO(dl_debug_mask) & DL_DEBUG_IMPCALLS))
    GLRO(dl_debug_printf) ("\ntransferring control: %s\n\n", argv[0]);

```

ld.so 中的 call_init:

初始化共享库的 DT_INIT / DT_INIT_ARRAY

libc-start.c 中的 call_init:

初始化主程序自己的 DT_INIT / DT_INIT_ARRAY

因此完整初始化顺序是：

```
_dl_init
  |
  |-- 主程序 .preinit_array
  |-- 共享库 .init
  |-- 共享库 .init_array
  |
  v
_start
  |
  v
__libc_start_main
  |
  |-- 主程序 .init
  |-- 主程序 .init_array
  |
  v
main
```

这解释了为什么动态链接器中的 call_init 要跳过主程序：

```
if (l->l_name[0] == '\0' && l->l_type == lt_executable)
  return;
```

因为主程序的普通构造函数在这里处理。

如果是静态链接，直接调用主程序初始化逻辑。

```c
#else /* !SHARED */

  /*
   * 静态链接场景下，直接调用主程序初始化逻辑。
   */
  call_init (argc, argv, __environ);

  /*
   * 初始化调试相关结构。
   */
  _dl_debug_initialize (0, LM_ID_BASE);

#endif
```

6.5 最终调用 main

```c
  /*
   * 最终调用 main。
   *
   * 注意这里不会直接写：
   *
   *   return main(argc, argv, __environ);
   *
   * 而是进入 __libc_start_call_main。
   *
   * __libc_start_call_main 会调用 main，
   * 并把 main 的返回值传给 exit。
   *
   * 因此 main 返回后，不会正常回到 _start，
   * 而是进入 exit 退出流程。
   */
  __libc_start_call_main (main, argc, argv MAIN_AUXVEC_PARAM);
}
```

```c
/*
 * __libc_start_call_main 是最终调用用户 main 的地方。
 *
 * _Noreturn 表示这个函数不会返回。
 *
 * __always_inline 表示尽量强制内联。
 */
_Noreturn static __always_inline void
__libc_start_call_main (int (*main) (int, char **, char ** MAIN_AUXVEC_DECL),
                        int argc, char **argv MAIN_AUXVEC_DECL)
{
  /*
   * 调用用户定义的 main 函数。
   *
   * 传入：
   *
   *   argc
   *   argv
   *   __environ，也就是 envp
   *
   * 在某些配置下，还可能额外传入 auxv。
   *
   * main 返回后，将返回值作为参数传给 exit。
   *
   * 所以等价于：
   *
   *   int ret = main(argc, argv, envp);
   *   exit(ret);
   *
   * exit 会执行：
   *
   *   - atexit/__cxa_atexit 注册的函数
   *   - C++ 全局对象析构
   *   - libc 清理
   *   - 动态库析构函数
   *   - 最终执行 _exit 系统调用
   */
  exit (main (argc, argv, __environ MAIN_AUXVEC_PARAM));
}
```

这就是最终调用用户 main 的地方。

可以简化理解为：

```c
int ret = main(argc, argv, __environ);
exit(ret);
```

所以：main 返回值不会直接返回给 _start。main 返回值会传给 exit。

为什么要用 `exit(main(...))`？

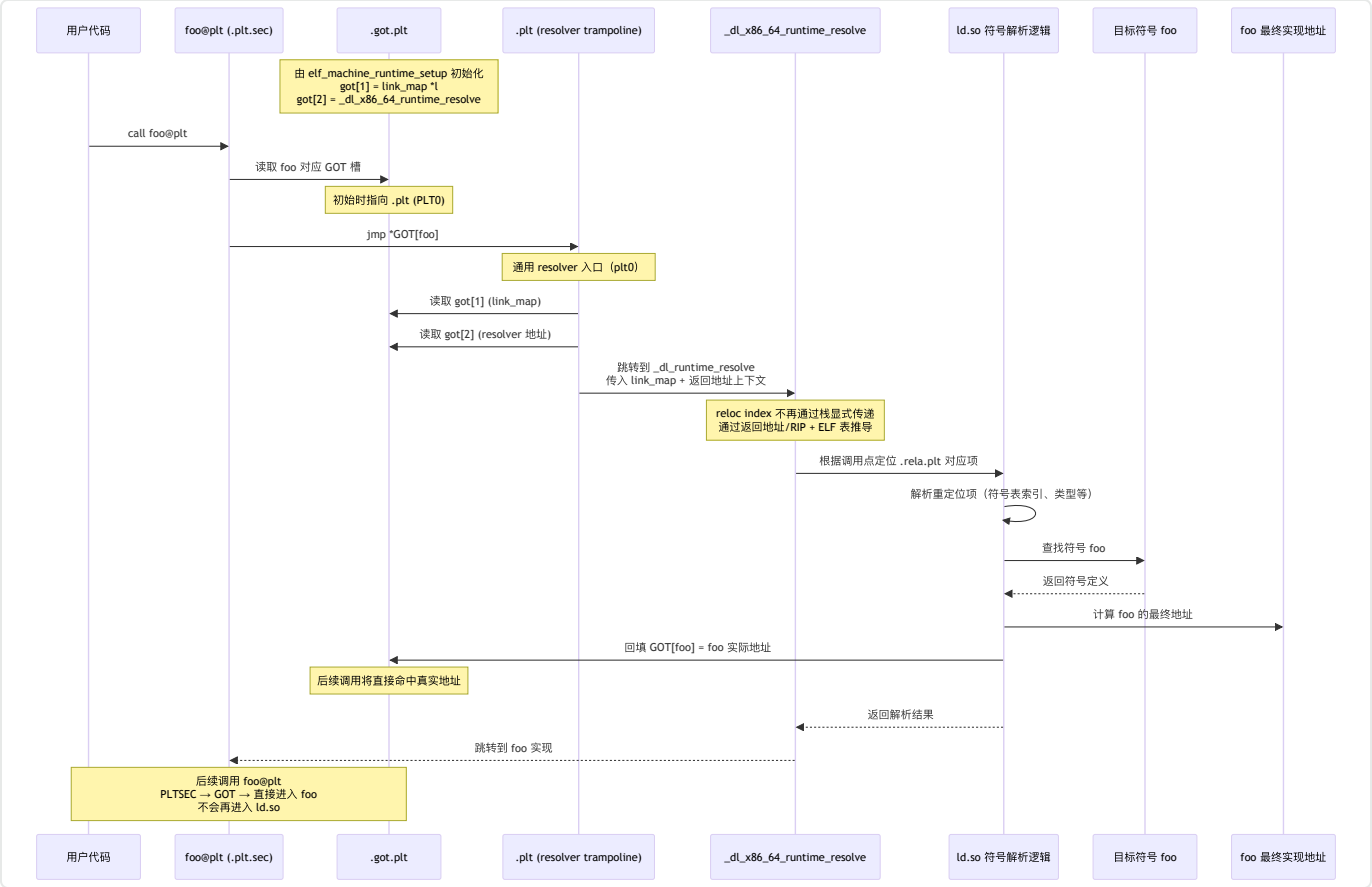
如果只是：`main(argc, argv, envp);` 那么 main 返回后很多清理工作不会执行。

而使用 `exit(main(...));` 可以保证程序退出时执行完整清理流程

所以 `return 0;` 本质上等价于 `exit(0);`

7.延迟绑定

时序图：



例子：

```c
// lazy.c
#include <stdio.h>

int main() {
    printf("hello\n");
    printf("hello world\n");
    return 0;
}
```

```Bash
liujx@c1fcdb233db4:/mnt/home/liujx/run/7.2$ objdump -d lazy
.......
Disassembly of section .plt:

0000000000401020 <.plt>:
  401020:       ff 35 ca 2f 00 00       push   0x2fca(%rip)        # 403ff
0 <_GLOBAL_OFFSET_TABLE_+0x8>
  401026:       ff 25 cc 2f 00 00       jmp    *0x2fcc(%rip)        # 403f
f8 <_GLOBAL_OFFSET_TABLE_+0x10>
  40102c:       0f 1f 40 00             nopl   0x0(%rax)
  401030:       f3 0f 1e fa             endbr64
  401034:       68 00 00 00 00          push   $0x0
  401039:       e9 e2 ff ff ff          jmp    401020 <_init+0x20>
  40103e:       66 90                   xchg   %ax,%ax

Disassembly of section .plt.sec:

0000000000401040 <puts@plt>:
  401040:       f3 0f 1e fa             endbr64
  401044:       ff 25 b6 2f 00 00       jmp    *0x2fb6(%rip)        # 4040
00 <puts@GLIBC_2.2.5>
  40104a:       66 0f 1f 44 00 00       nopw   0x0(%rax,%rax,1)
.......

liujx@c1fcdb233db4:/mnt/home/liujx/run/7.2$ readelf -r ./lazy | grep puts
000000404000  000200000007 R_X86_64_JUMP_SLO 0000000000000000 puts@GLIBC_
2.2.5 + 0
```

```Bash
# main 函数
b main

# puts@plt
b puts@plt

# PLT入口 (plt0)
b *0x401020     # 用 objdump 查 plt0 地址
```

这里 puts 的 GOT 槽是 `0x404000`，所以：

```Bash
pwndbg> x/gx 0x404000
0x404000 <puts@got.plt>:        0x0000000000401030
```

这说明：`GOT[puts] -> 401030`，也就是跳去：

```Bash
   401030:       f3 0f 1e fa             endbr64
   401034:       68 00 00 00 00          push   $0x0
   401039:       e9 e2 ff ff ff          jmp    401020 <_init+0x20>
```

反汇编里：

```C
401020: push 0x403ff0   ; GOT+0x8
401026: jmp *0x403ff8   ; GOT+0x10
```

所以：

```C
pwndbg> x/gx 0x403ff0
0x403ff0:       0x00007f0c3104c2e0
pwndbg> x/gx 0x403ff8
0x403ff8:       0x00007f0c310282f0
```

```C
0x403ff0 = GOT[1] = link_map
0x403ff8 = GOT[2] = resolver 地址
```

```
pwndbg> set $resolver = *(void **)0x403ff8
pwndbg> p/x $resolver
$1 = 0x7f9ce2ff92f0
pwndbg> b *$resolver
Breakpoint 3 at 0x7f9ce2ff92f0: file ../sysdeps/x86_64/dl-trampoline.h, line 71.
```

```
—
pwndbg> info symbol $pc
_dl_runtime_resolve_xsavec in section .text of /lib64/ld-linux-x86-64.so.2
pwndbg> x/8gx $rsp
0x7ffdc19165c8: 0x00007f0c3104c2e0      0x0000000000000000
0x7ffdc19165d8: 0x000000000040114d      0x00007ffdc1916680
0x7ffdc19165e8: 0x00007f0c30e231ca      0x00007ffdc1916630
0x7ffdc19165f8: 0x00007ffdc1916708      0x0000000100400040
pwndbg> x/gx 0x404000
0x404000 <puts@got.plt>:        0x0000000000401030
```

```
0x7ffc8d82be38: 0x00007f9ce301d2e0      0x0000000000000000
                ↑                       ↑
                link_map                reloc_arg

0x7ffc8d82be48: 0x000000000040114d      0x00007ffdc1916680
                ↑
                返回到 main
```

```
+----------------+
| link_map       |
+----------------+
| reloc_arg=0    |
+----------------+
| return addr    |
+----------------+
```

```
pwndbg> disassemble
Dump of assembler code for function _dl_runtime_resolve_xsavec:
   0x00007f0c310282f0 <+0>:     endbr64
=> 0x00007f0c310282f4 <+4>:     push   rbx
   0x00007f0c310282f5 <+5>:     mov    rbx,rsp
   0x00007f0c310282f8 <+8>:     and    rsp,0xffffffffffffffc0
   0x00007f0c310282fc <+12>:    sub    rsp,QWORD PTR [rip+0x2296d]
# 0x7f0c3104ac70 <_rtld_global_ro+464>
   0x00007f0c31028303 <+19>:    mov    QWORD PTR [rsp],rax
   0x00007f0c31028307 <+23>:    mov    QWORD PTR [rsp+0x8],rcx
   0x00007f0c3102830c <+28>:    mov    QWORD PTR [rsp+0x10],rdx
   0x00007f0c31028311 <+33>:    mov    QWORD PTR [rsp+0x18],rsi
   0x00007f0c31028316 <+38>:    mov    QWORD PTR [rsp+0x20],rdi
   0x00007f0c3102831b <+43>:    mov    QWORD PTR [rsp+0x28],r8
   0x00007f0c31028320 <+48>:    mov    QWORD PTR [rsp+0x30],r9
   0x00007f0c31028325 <+53>:    mov    eax,0xee
   0x00007f0c3102832a <+58>:    xor    edx,edx
   0x00007f0c3102832c <+60>:    mov    QWORD PTR [rsp+0x250],rdx
   0x00007f0c31028334 <+68>:    mov    QWORD PTR [rsp+0x258],rdx
   0x00007f0c3102833c <+76>:    mov    QWORD PTR [rsp+0x260],rdx
   0x00007f0c31028344 <+84>:    mov    QWORD PTR [rsp+0x268],rdx
   0x00007f0c3102834c <+92>:    mov    QWORD PTR [rsp+0x270],rdx
   0x00007f0c31028354 <+100>:   mov    QWORD PTR [rsp+0x278],rdx
   0x00007f0c3102835c <+108>:   xsavec [rsp+0x40]
   0x00007f0c31028361 <+113>:   mov    rsi,QWORD PTR [rbx+0x10]
   0x00007f0c31028365 <+117>:   mov    rdi,QWORD PTR [rbx+0x8]
   0x00007f0c31028369 <+121>:   call   0x7f0c31025af0 <_dl_fixup>
   0x00007f0c3102836e <+126>:   mov    r11,rax
   0x00007f0c31028371 <+129>:   mov    eax,0xee
   0x00007f0c31028376 <+134>:   xor    edx,edx
   0x00007f0c31028378 <+136>:   xrstor [rsp+0x40]
   0x00007f0c3102837d <+141>:   mov    r9,QWORD PTR [rsp+0x30]
   0x00007f0c31028382 <+146>:   mov    r8,QWORD PTR [rsp+0x28]
   0x00007f0c31028387 <+151>:   mov    rdi,QWORD PTR [rsp+0x20]
   0x00007f0c3102838c <+156>:   mov    rsi,QWORD PTR [rsp+0x18]
   0x00007f0c31028391 <+161>:   mov    rdx,QWORD PTR [rsp+0x10]
   0x00007f0c31028396 <+166>:   mov    rcx,QWORD PTR [rsp+0x8]
   0x00007f0c3102839b <+171>:   mov    rax,QWORD PTR [rsp]
   0x00007f0c3102839f <+175>:   mov    rsp,rbx
   0x00007f0c310283a2 <+178>:   mov    rbx,QWORD PTR [rsp]
   0x00007f0c310283a6 <+182>:   add    rsp,0x18
   0x00007f0c310283aa <+186>:   jmp    r11
End of assembler dump.
```

```
_dl_fixup(link_map, reloc_arg);
```

```
_
pwndbg> bt
#0  _dl_fixup (l=0x7f0c3104c2e0, reloc_arg=0) at ./elf/dl-runtime.c:46
#1  0x00007f0c3102836e in _dl_runtime_resolve_xsavec () at ../sysdeps/x86_64/dl-trampoline.h:130
#2  0x000000000040114d in main () at lazy.c:5
#3  0x00007f0c30e231ca in __libc_start_call_main (main=main@entry=0x401136 <main>, argc=argc@entry=1, argv=argv@entry=0x7ffdc1916708) at ../sysdeps/nptl/libc_start_call_main.h:58
#4  0x00007f0c30e2328b in __libc_start_main_impl (main=0x401136 <main>, argc=1, argv=0x7ffdc1916708, init=<optimized out>, fini=<optimized out>, rtld_fini=<optimized out>, stack_end=0x7ffdc19166f8) at ../csu/libc-start.c:360
#5  0x0000000000401075 in _start ()
```

```
liujx@c1fcdb233db4:/mnt/home/liujx/run/7.2$ readelf -Wr lazy

Relocation section '.rela.dyn' at offset 0x4b0 contains 2 entries:
    Offset             Info             Type               Symbol's Value  Symbol's Name + Addend
0000000000403fd8  0000000100000006 R_X86_64_GLOB_DAT      0000000000000000 __libc_start_main@GLIBC_2.34 + 0
0000000000403fe0  0000000300000006 R_X86_64_GLOB_DAT      0000000000000000 __gmon_start__ + 0

Relocation section '.rela.plt' at offset 0x4e0 contains 1 entry:
    Offset             Info             Type               Symbol's Value  Symbol's Name + Addend
0000000000404000  0000000200000007 R_X86_64_JUMP_SLOT     0000000000000000 puts@GLIBC_2.2.5 + 0
```

重定位表 第0项 = puts

```
pwndbg> x/gx 0x404000
0x404000 <puts@got.plt>:        0x0000000000401030
```

说明:

```
GOT[puts]
        ↓
lazy stub
```

```
finish
pwndbg> x/gx 0x404000
0x404000 <puts@got.plt>:        0x00007f9a5cd7bbe0
```

libc 地址。即：

```
before:
puts@got.plt = 0x401030

after:
puts@got.plt = 0x00007f9a5cd7bbe0
```

这一步就是：

```
_dl_fixup
  ↓
_dl_lookup_symbol_x
  ↓
找到 libc::puts
  ↓
写回 GOT
```

第二次：

```
─
 ► 0x401040       <puts@plt>      endbr64
   0x401044       <puts@plt+4>    jmp    qword ptr [rip + 0x2fb6]    <puts>

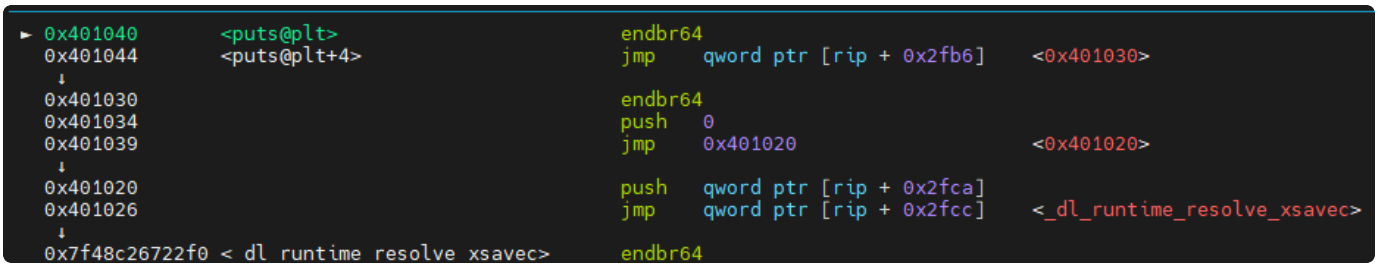
si
─
pwndbg> info symbol $pc
puts in section .text of /lib/x86_64-linux-gnu/libc.so.6
```

发现直接跳进 `libc.so.6:puts`



而不会再经过：

```
1  401030
2  401020
3  _dl_runtime_resolve_xsavec
```



第一次调用时，大致流程是：

```
main
 ↓
call puts@plt
 ↓
.plt.sec
 ↓
0x401030 (push reloc_arg=0)
 ↓
PLT0 (push link_map)
 ↓
_dl_runtime_resolve_xsavec
 ↓
_dl_fixup(link_map, 0)
 ↓
_dl_lookup_symbol_x
 ↓
找到 libc puts
 ↓
写 GOT[puts]
 ↓
返回 puts 地址
 ↓
jmp puts
 ↓
puts 执行
```

后续再次调用：

```
call printf@plt
        |
        v
GOTPLT[printf]
        |
        v
libc 中真正的 printf
```

这时就不再进入动态链接器。

```
pwndbg> x/50i $pc
=> 0x7f48c26722f0 <_dl_runtime_resolve_xsavec>: endbr64
   0x7f48c26722f4 <_dl_runtime_resolve_xsavec+4>:       push   rbx
   0x7f48c26722f5 <_dl_runtime_resolve_xsavec+5>:       mov    rbx,rsp
   0x7f48c26722f8 <_dl_runtime_resolve_xsavec+8>:       and    rsp,0xfffffffffffffc0
   0x7f48c26722fc <_dl_runtime_resolve_xsavec+12>:      sub    rsp,QWORD PTR [rip+0x2296d]        # 0x7
   0x7f48c2672303 <_dl_runtime_resolve_xsavec+19>:      mov    QWORD PTR [rsp],rax
   0x7f48c2672307 <_dl_runtime_resolve_xsavec+23>:      mov    QWORD PTR [rsp+0x8],rcx
   0x7f48c267230c <_dl_runtime_resolve_xsavec+28>:      mov    QWORD PTR [rsp+0x10],rdx
   0x7f48c2672311 <_dl_runtime_resolve_xsavec+33>:      mov    QWORD PTR [rsp+0x18],rsi
   0x7f48c2672316 <_dl_runtime_resolve_xsavec+38>:      mov    QWORD PTR [rsp+0x20],rdi
   0x7f48c267231b <_dl_runtime_resolve_xsavec+43>:      mov    QWORD PTR [rsp+0x28],r8
   0x7f48c2672320 <_dl_runtime_resolve_xsavec+48>:      mov    QWORD PTR [rsp+0x30],r9
   0x7f48c2672325 <_dl_runtime_resolve_xsavec+53>:      mov    eax,0xee
   0x7f48c267232a <_dl_runtime_resolve_xsavec+58>:      xor    edx,edx
   0x7f48c267232c <_dl_runtime_resolve_xsavec+60>:      mov    QWORD PTR [rsp+0x250],rdx
   0x7f48c2672334 <_dl_runtime_resolve_xsavec+68>:      mov    QWORD PTR [rsp+0x258],rdx
   0x7f48c267233c <_dl_runtime_resolve_xsavec+76>:      mov    QWORD PTR [rsp+0x260],rdx
   0x7f48c2672344 <_dl_runtime_resolve_xsavec+84>:      mov    QWORD PTR [rsp+0x268],rdx
   0x7f48c267234c <_dl_runtime_resolve_xsavec+92>:      mov    QWORD PTR [rsp+0x270],rdx
   0x7f48c2672354 <_dl_runtime_resolve_xsavec+100>:     mov    QWORD PTR [rsp+0x278],rdx
   0x7f48c267235c <_dl_runtime_resolve_xsavec+108>:     xsavec [rsp+0x40]
   0x7f48c2672361 <_dl_runtime_resolve_xsavec+113>:     mov    rsi,QWORD PTR [rbx+0x10]
   0x7f48c2672365 <_dl_runtime_resolve_xsavec+117>:     mov    rdi,QWORD PTR [rbx+0x8]
   0x7f48c2672369 <_dl_runtime_resolve_xsavec+121>:     call   0x7f48c266faf0 <_dl_fixup>
   0x7f48c267236e <_dl_runtime_resolve_xsavec+126>:     mov    r11,rax
   0x7f48c2672371 <_dl_runtime_resolve_xsavec+129>:     mov    eax,0xee
   0x7f48c2672376 <_dl_runtime_resolve_xsavec+134>:     xor    edx,edx
   0x7f48c2672378 <_dl_runtime_resolve_xsavec+136>:     xrstor [rsp+0x40]
   0x7f48c267237d <_dl_runtime_resolve_xsavec+141>:     mov    r9,QWORD PTR [rsp+0x30]
   0x7f48c2672382 <_dl_runtime_resolve_xsavec+146>:     mov    r8,QWORD PTR [rsp+0x28]
   0x7f48c2672387 <_dl_runtime_resolve_xsavec+151>:     mov    rdi,QWORD PTR [rsp+0x20]
   0x7f48c267238c <_dl_runtime_resolve_xsavec+156>:     mov    rsi,QWORD PTR [rsp+0x18]
   0x7f48c2672391 <_dl_runtime_resolve_xsavec+161>:     mov    rdx,QWORD PTR [rsp+0x10]
   0x7f48c2672396 <_dl_runtime_resolve_xsavec+166>:     mov    rcx,QWORD PTR [rsp+0x8]
   0x7f48c267239b <_dl_runtime_resolve_xsavec+171>:     mov    rax,QWORD PTR [rsp]
   0x7f48c267239f <_dl_runtime_resolve_xsavec+175>:     mov    rsp,rbx
   0x7f48c26723a2 <_dl_runtime_resolve_xsavec+178>:     mov    rbx,QWORD PTR [rsp]
   0x7f48c26723a6 <_dl_runtime_resolve_xsavec+182>:     add    rsp,0x18
   0x7f48c26723aa <_dl_runtime_resolve_xsavec+186>:     jmp    r11
   0x7f48c26723ad:      nop    DWORD PTR [rax]
   0x7f48c26723b0 <do_tunable_update_val>:      mov    rax,rdi
   0x7f48c26723b3 <do_tunable_update_val+3>:    mov    edi,DWORD PTR [rdi+0x30]
   0x7f48c26723b6 <do_tunable_update_val+6>:    cmp    edi,0x2
   0x7f48c26723b9 <do_tunable_update_val+9>:    ja     0x7f48c2672480 <do_tunable_update_val+208>
   0x7f48c26723bf <do_tunable_update_val+15>:   test   edi,edi
   0x7f48c26723c1 <do_tunable_update_val+17>:   je     0x7f48c2672470 <do_tunable_update_val+192>
   0x7f48c26723c7 <do_tunable_update_val+23>:   mov    r9,QWORD PTR [rsi]
   0x7f48c26723ca <do_tunable_update_val+26>:   mov    r10d,0x1
   0x7f48c26723d0 <do_tunable_update_val+32>:   mov    rdi,QWORD PTR [rax+0x38]
   0x7f48c26723d4 <do_tunable_update_val+36>:   test   rdx,rdx
pwndbg> quit
```

```
1   call _dl_fixup
2   mov  r11, rax
3   ...
4   jmp r11
```

也就是：

```
1   addr = _dl_fixup(link_map, reloc_arg);
2   jump(addr);
```

因此：

```
_dl_fixup
    ↓
找到 puts
    ↓
写 GOT
    ↓
返回 puts真实地址
    ↓
jmp puts
```

不会返回到 PLT。

```
1. CPU 执行 call printf@plt
2. PLT 发现 GOT 中还不是 printf 真实地址
3. 跳到动态链接器 resolver
4. _dl_fixup 读取 .rela.plt / .dynsym / .dynstr
5. 得到要解析的符号名 "printf"
6. 调用 _dl_lookup_symbol_x
7. _dl_lookup_symbol_x 遍历符号查找作用域
8. 对每个 scope 调用 do_lookup_x
9. do_lookup_x 在 libc.so.6 的 GNU hash 中找到 printf
10. check_match 确认名称、类型、版本都匹配
11. result 返回 printf 所在 link_map 和 ElfW(Sym)
12. _dl_fixup 计算最终地址
13. GOT 被回填
14. 后续 printf 调用直接跳到 libc 中的 printf
```

8.do_lookup_x

在给定的 ELF 查找作用域 scope 中，从第 i 个对象开始，逐个遍历已加载的共享对象，通过 GNU hash 或 SysV hash 快速筛选候选符号，再通过 check_match 做精确匹配，最终返回找到的符号及其所在 link_map。

8.1关键数据结构

8.1.1scope

```c
struct r_scope_elem *scope
```

表示一次符号查找的作用域。可以理解为：

```c
scope = [main executable, libc.so, libpthread.so, libfoo.so, ...]
```

其中：

```c
scope->r_list
scope->r_nlist
```

分别表示：

- r_list：link_map * 数组；
- r_nlist：数组元素数量。

8.1.2 link_map

每个 link_map 表示一个已加载 ELF 对象，例如：

```c
a.out
libc.so.6
libm.so.6
libfoo.so
ld-linux-x86-64.so.2
```

do_lookup_x 通过 link_map 访问：

```c
map->l_info[DT_SYMTAB]   // 动态符号表
map->l_info[DT_STRTAB]   // 动态字符串表
map->l_gnu_bitmask       // GNU hash bloom filter
map->l_gnu_buckets       // GNU hash buckets
map->l_buckets           // SysV hash buckets
map->l_chain             // SysV hash chain
map->l_versyms           // 符号版本信息
```

8.1.3 result

```c
struct sym_val *result
```

查找成功后写入：

```c
result->s = sym;  // 符号表项
result->m = map;  // 符号所在对象
```

后续动态链接器会根据：

```c
map->l_addr + sym->st_value
```

计算最终运行时地址。

流程图：

```
do_lookup_x
  |
  |-- 读取 scope->r_nlist
  |-- 获取 scope->r_list
  |
  for each link_map in scope:
      |
      |-- 跳过 skip 对象
      |-- copy relocation 时跳过 executable
      |-- 跳过已 removed 对象
      |-- 如果无 hash bucket, 跳过
      |
      |-- 获取 symtab / strtab
      |
      |-- 如果存在 GNU hash:
      |       |
      |       |-- Bloom filter 快速判断
      |       |-- bucket 定位候选链
      |       |-- 遍历 GNU hash chain
      |       |-- check_match 精确匹配
      |
      |-- 否则使用 SysV hash:
      |       |
      |       |-- 计算 old_hash
      |       |-- bucket 定位候选链
      |       |-- 遍历 chain
      |       |-- check_match 精确匹配
      |
      |-- 处理版本符号兼容逻辑
      |-- 检查 visibility
      |-- 检查 binding
            |
            |-- STB_GLOBAL      -> 成功返回
            |-- STB_WEAK        -> 可能暂存, 继续找强符号
            |-- STB_GNU_UNIQUE  -> 走 unique 处理
            |-- 其他            -> 忽略
```

8.2读取查找作用域长度

```c
/*
 * 符号查找函数的内部实现。
 *
 * 返回值语义：
 *   > 0 ：找到了目标符号；
 *   = 0 ：没有找到符号；
 *   < 0 ：查找过程中发生错误。
 */
static int
__attribute_noinline__
do_lookup_x (const char *undef_name, unsigned int new_hash,
             unsigned long int *old_hash, const ElfW(Sym) *ref,
             struct sym_val *result, struct r_scope_elem *scope, size_t i,
             const struct r_found_version *const version, int flags,
             struct link_map *skip, int type_class, struct link_map *undef
_map)
{
  /*
   * scope 表示当前符号查找作用域。
   * r_nlist 表示作用域中 link_map 的数量。
   */
  size_t n = scope->r_nlist;

  /*
   * 这里使用空的内联汇编作为编译器屏障。
   *
   * 目的：
   *   确保在继续执行之前，先读取 scope->r_nlist 的值。
   *
   * 背景：
   *   在 dl-open.c 的某些路径中，scope->r_list 可能被 resize。
   *   如果没有顺序约束，可能出现：
   *
   *     - r_list 仍然指向旧的数组；
   *     - r_nlist 却已经是 resize 后的新值。
   *
   *   这样后续遍历就可能越界或访问错误对象。
   *
   * 这里没有使用真正的读内存屏障，因为符号查找是高频路径，
   * 完整屏障的开销可能过大。
   */
  __asm volatile ("" : "+r" (n), "+m" (scope->r_list));

  /*
   * 获取当前作用域中的 link_map 列表。
```

```
45      * 每个 link_map 对应一个已经加载的 ELF 对象，例如主程序或共享库。
46      */
47     struct link_map **list = scope->r_list;
```

8.3 遍历当前 scope 中的每个对象

有几类跳过逻辑：

```c
  /*
   * 从传入的索引 i 开始，遍历作用域中的每一个对象。
   */
  do
    {
      /*
       * l_real 指向真实的 link_map。
       * 某些情况下 link_map 可能是代理对象，这里取其真实对象。
       */
      const struct link_map *map = list[i]->l_real;

      /*
       * _dl_lookup_symbol_skip 场景下需要额外跳过某个对象。
       * 如果当前对象正好是 skip 指定的对象，则不在该对象中查找。
       */
      if (map == skip)
        continue;

      /*
       * 处理 copy relocation 的特殊情况。
       *
       * 当解析 copy reloc 时，不应该搜索可执行文件本身。
       * copy reloc 是 ELF 动态链接中针对数据符号的一种特殊重定位方式。
       */
      if ((type_class & ELF_RTYPE_CLASS_COPY) && map->l_type == lt_executable)
        continue;

      /*
       * 不要在即将被移除的对象中查找符号。
       *
       * 例外情况：
       *   如果发起引用的对象 undef_map 自身也正在被移除，
       *   那么允许在同样即将移除的对象中解析符号。
       *
       * 背景：
       *   某些对象在析构函数运行期间可能触发 lazy binding。
       *   此时另一个对象的析构函数可能已经执行完毕，
       *   dlclose 已经将其标记为 l_removed。
       *
       */
      if (map->l_removed && !(undef_map != NULL && undef_map->l_removed))
        continue;

      /*
```

```
         * 如果打开了动态链接器的符号调试开关，则打印当前查找信息。
         *
         * 输出内容包括：
         *   - 正在查找的符号名；
         *   - 当前搜索的文件名；
         *   - 命名空间编号。
         */
        if (__glibc_unlikely (GLRO(dl_debug_mask) & DL_DEBUG_SYMBOLS))
          _dl_debug_printf ("symbol=%s;  lookup in file=%s [%lu]\n",
                            undef_name, DSO_FILENAME (map->l_name),
                            map->l_ns);

        /*
         * 如果当前对象的 hash bucket 数量为 0，说明没有可用于查找的 hash 表，
         * 直接跳过该对象。
         */
        if (map->l_nbuckets == 0)
          continue;
```

引用者 undef_map	被搜索对象 map	是否允许
正常	正常	✓
正常	l_removed	✗
l_removed	l_removed	✓
l_removed	正常	✓

8.4 获取符号表和字符串表

```c
        /*
         * symidx 用于保存当前候选符号在符号表中的索引。
         */
        Elf_Symndx symidx;

        /*
         * num_versions 用于记录查找过程中遇到的版本化符号数量。
         */
        int num_versions = 0;

        /*
         * versioned_sym 用于保存某个版本化符号候选。
         *
         * 在查找未版本化符号时，如果只遇到一个版本化符号，
         * 后续可能会接受这个符号。
         */
        const ElfW(Sym) *versioned_sym = NULL;

        /*
         * 获取当前 ELF 对象的符号表。
         *
         * DT_SYMTAB 是 dynamic section 中指向符号表的条目。
         * D_PTR 会根据对象加载地址等信息得到运行期地址。
         */
        const ElfW(Sym) *symtab = (const void *) D_PTR (map, l_info[DT_SYMTAB]);

        /*
         * 获取当前 ELF 对象的字符串表。
         *
         * 符号表项中的 st_name 是字符串表中的偏移，
         * 需要结合 strtab 才能得到符号名。
         */
        const char *strtab = (const void *) D_PTR (map, l_info[DT_STRTAB]);

```

ELF 动态符号解析主要依赖两个表：

```c
.dynsym  -> 符号表
.dynstr  -> 字符串表
```

符号表项中的 st_name 不是字符串指针，而是 .dynstr 中的偏移。所以符号名通常这样得到：

```c
strtab + sym->st_name
```

8.5 GNU hash 查找路径

现代 Linux ELF 通常使用 GNU hash。

```c
        /*
         * sym 用于保存 check_match 返回的匹配符号。
         */
        const ElfW(Sym) *sym;

        /*
         * l_gnu_bitmask 非空表示该对象使用 GNU hash 表。
         *
         * GNU hash 是 SysV hash 的改进版本，通常查找性能更好。
         */
        const ElfW(Addr) *bitmask = map->l_gnu_bitmask;

        if (__glibc_likely (bitmask != NULL))
          {

```

8.5.1 Bloom filter 快速过滤

```c
/*
           * GNU hash 的第一步：Bloom filter 检查。
           *
           * 根据 new_hash 选择 bitmask 中的一个 word。
           */
          ElfW(Addr) bitmask_word
            = bitmask[(new_hash / __ELF_NATIVE_CLASS)
                      & map->l_gnu_bitmask_idxbits];

          /*
           * 计算 Bloom filter 中需要检查的第一个 bit。
           */
          unsigned int hashbit1 = new_hash & (__ELF_NATIVE_CLASS - 1);

          /*
           * 计算 Bloom filter 中需要检查的第二个 bit。
           *
           * 第二个 bit 由 hash 右移 l_gnu_shift 后得到。
           */
          unsigned int hashbit2 = ((new_hash >> map->l_gnu_shift)
                                   & (__ELF_NATIVE_CLASS - 1));

          /*
           * 如果 Bloom filter 的两个 bit 都命中，
           * 说明该对象中可能存在目标符号。
           *
           * 注意：
           *   Bloom filter 可能误报，但不会漏报。
           *   如果这里不命中，就可以确定该对象没有目标符号。
           */
```

Bloom filter 的作用是：

如果 bloom filter 判断“不可能存在”，则一定不存在；如果判断“可能存在”，再继续查 bucket 和 chain。

对于大量共享库、大量导出符号的进程，Bloom filter 可以显著减少字符串比较和符号表遍历次数。

8.5.2 bucket 定位

bucket = 哈希桶数组

```c
            if (__glibc_unlikely ((bitmask_word >> hashbit1)
                                  & (bitmask_word >> hashbit2) & 1))
              {
                /*
                 * 根据 GNU hash 值定位 bucket。
                 */
                Elf32_Word bucket = map->l_gnu_buckets[new_hash
                                                       % map->l_nbuckets];
```

如果 `bucket == 0`，表示没有候选符号链，直接跳过。

8.5.3 遍历 GNU hash chain

chain = 冲突链表

```c
                /*
                 * bucket 为 0 表示没有对应的符号链。
                 */
                if (bucket != 0)
                  {
                    /*
                     * 获取 GNU hash chain 的起始位置。
                     *
                     * l_gnu_chain_zero 是为了便于通过符号索引直接定位 chain。
                     */
                    const Elf32_Word *hasharr = &map->l_gnu_chain_zero[bucket];

                    /*
                     * 遍历 GNU hash chain。
                     */
                    do
                      /*
                       * 比较 hash 值。
                       *
                       * GNU hash chain 元素的最低位用于标记链是否结束，
                       * 因此比较时右移一位，忽略最低位（最低位用于表示链尾）。
                       */
                      if (((*hasharr ^ new_hash) >> 1) == 0)
                        {
                          /*
                           * 根据 hasharr 的位置计算符号表索引。
                           */
                          symidx = ELF_MACHINE_HASH_SYMIDX (map, hasharr);

                          /*
                           * 对候选符号做精确匹配。
                           *
                           * check_match 会检查：
                           *   - 符号名是否一致；
                           *   - 符号是否定义；
                           *   - 符号类型是否满足要求；
                           *   - 符号版本是否匹配；
                           *   - 当前 flags 与 type_class 是否允许该符号。
                           */
                          sym = check_match (undef_name, ref, version, flags,
                                             type_class, &symtab[symidx], symidx,
                                             strtab, map, &versioned_sym,
```

```
                                        &num_versions);

                    /*
                     * 如果 check_match 返回非空，说明找到可用符号。
                     */
                    if (sym != NULL)
                      goto found_it;
                  }

                /*
                 * GNU hash chain 的最低位为 1 表示链结束。
                 * 每次循环后 hasharr++，继续检查下一个 chain 元素。
                 */
                while ((*hasharr++ & 1u) == 0);
              }
            }

          /*
           * 走到这里表示 GNU hash 查找没有找到符号。
           */
          symidx = SHN_UNDEF;
        }
```

8.6 SysV hash 查找路径

如果对象没有 GNU hash，则走传统 SysV hash：

```c
        else
          {
            /*
             * 如果当前对象没有 GNU hash，则使用传统 SysV ELF hash。
             *
             * old_hash 初始值为 0xffffffff，表示尚未计算。
             * 只有真正需要 SysV hash 时才计算，避免额外开销。
             */
            if (*old_hash == 0xffffffff)
              *old_hash = _dl_elf_hash (undef_name);

            /*
             * 使用 SysV 风格 hash 表查找。
             *
             * 查找流程：
             *   1. 根据 old_hash 定位 bucket；
             *   2. 从 bucket 得到第一个符号索引；
             *   3. 沿着 chain 遍历候选符号；
             *   4. 对每个候选符号调用 check_match。
             */
            for (symidx = map->l_buckets[*old_hash % map->l_nbuckets];
                 symidx != STN_UNDEF;
                 symidx = map->l_chain[symidx])
              {
                /*
                 * 对当前候选符号做精确匹配。
                 */
                sym = check_match (undef_name, ref, version, flags,
                                   type_class, &symtab[symidx], symidx,
                                   strtab, map, &versioned_sym,
                                   &num_versions);

                /*
                 * 找到匹配符号后跳转到统一处理逻辑。
                 */
                if (sym != NULL)
                  goto found_it;
              }
          }
```

SysV Hash

- bucket[i] = 链表头符号索引

- chain[j] = 下一个符号索引

GNU Hash

- bucket[i] = 链表起始符号索引
- chain[j] = 符号 hash 值（带结束标志）
- 通过连续符号 + hash 比较代替真正链表

这也是 GNU Hash 比 SysV Hash 更快、更省内存的关键原因。

举例：

下面举例说明 SysV Hash 和 GNU Hash 是如何匹配到符号的。

假设动态库里有如下符号表：

```
.symtab

Index   Symbol
-----   ------
0       UND
1       printf
2       malloc
3       free
4       puts
```

用户要查找：`malloc`

1.SysV Hash匹配示例

1. 计算Hash

SysV Hash算法得到：

```
elf_hash("malloc")
= 0x7383353
```

假设 nbucket = 4，则：

```
bucket_id = 0x7383353 % 4
          = 1
```

2. Hash表内容

bucket：bucket[i] 保存的是这个哈希桶中的第一个符号索引。

```
bucket[0] = 0   //empty
bucket[1] = 1   //printf
bucket[2] = 4   //puts
bucket[3] = 0
```

表示bucket[1]指向第一个符号

```
bucket[1]
    ↓
symbol #1
```

即 `printf`

现在有冲突，printf、malloc、free都落到了 bucket1。bucket只能保存第一个元素，那么剩下的由chain保存。

chain

```
chain[1] = 2
chain[2] = 3
chain[3] = 0
chain[4] = 0
```

即symbol#1 –> symbol#2 –> symbol#3 –> END　对应：printf –> malloc –> free –> 结束

```
bucket[1]
    |
    ▼
+-------+
|   1   | printf
+-------+
    |
chain[1]=2
    ▼

+-------+
|   2   | malloc
+-------+
    |
chain[2]=3
    ▼

+-------+
|   3   | free
+-------+
    |
chain[3]=0
    ▼

END
```

3. 查找过程

第一次比较

```
bucket[1]
 ↓
symbol #1

printf
```

比较：`printf == malloc ?` 结果：NO

跳到chain

```
chain[1]
   ↓
   2
```

第二次比较 `malloc == malloc ?` 结果：YES

找到符号。

整个路径

```
hash("malloc")
      |
      ▼
bucket[1]
      |
      ▼
printf
      |
      ▼
chain[1]
      |
      ▼
malloc
      |
      ▼
MATCH
```

2.GNU Hash匹配示例

GNU Hash觉得chain里面全是next指针这种方式太浪费，因为printf、malloc、free在符号表里本来就是连续的，没有必要存1 –> 2，2 –> 3，于是GNU Hash做了更改:

bucket还是一样 `bucket[1]=1` 表示从symbol #1开始查；而chain里保存的不是next索引，而是直接保存hash值，例如：

```
symbol 1 printf
hash=0x12345678

symbol 2 malloc
hash=0x87654320

symbol 3 free
hash=0x99999999
```

chain:

```
chain[1]=0x12345678
chain[2]=0x87654320
chain[3]=0x99999999|1
```

最后一个元素最低位设置为1,表示链表结束。

GNU查找malloc

1.计算GNU Hash

假设：`gnu_hash("malloc") = 0xd39ad3d`

2.Bloom Filter检查

假设`bloom[0] = 0b101001011110...`，计算两个bit：

```
bit1 = hash % 64
bit2 = (hash >> shift2) % 64
```

得到：bit1 = 13、bit2 = 37，去查bloom[0]：

```
bloom[0] bit13 == 1
bloom[0] bit37 == 1
```

说明malloc 可能存在，如果其中某个位是0就直接失败，连bucket都不会访问。

3.Bucket定位

假设 `nbuckets = 2`，计算bucket_id = 0xd39ad3c % 2 = 1，读取bucket[1] = 1，从symbol #1开始搜索。

查看 `chain[1]`，对应 `printf hash`，比较hash不一致的话就 `idx++`,以此类推，发现hash一致，再做 `check_match`，确认符合，找到符号。

数组扫描+hash预过滤。

```
hash("malloc")
        |
        ▼
 Bloom Filter
        |
        ▼
 通过
        |
        ▼
 bucket[1]
        |
        ▼
 symbol #1
        |
        ▼
 chain hash compare
        |
        ▼
 HASH MATCH
        |
        ▼
 check_match
        |
        ▼
 malloc
        |
        ▼
 MATCH
```

8.7 版本符号兼容逻辑

```c
        sym = num_versions == 1 ? versioned_sym : NULL;
```

当查找的是未版本化符号，但过程中只遇到一个版本化候选符号时，glibc 可能接受它。因为只有一个候选版本，不存在二义性。如果有多个版本，则不能随便选。

8.8 找到符号后的处理逻辑

进入 `found_it:` 之后还不能立刻返回，要继续检查符号可见性和绑定属性。

8.8.1hidden/internal 符号不可被外部绑定

```c
  if (sym != NULL)
          {
          found_it:
            /*
             * hidden 和 internal 可见性的符号是本地绑定符号，
             * 不应该被其他对象解析到，因此直接忽略。
             */
            if (__glibc_unlikely (dl_symbol_visibility_binds_local_p (sym)))
              goto skip;
```

STV_HIDDEN / STV_INTERNAL 这类符号只应该在本对象内部使用。即使名字和 hash 都匹配，也不能让外部对象解析到它。

8.8.2 protected 符号检查

```c
            /*
             * 如果符号是 STV_PROTECTED，需要做额外检查。
             *
             * protected 符号对外可见，
             * 但在定义该符号的对象内部引用时不允许被外部对象抢占。
             */
            if (ELFW(ST_VISIBILITY) (sym->st_other) == STV_PROTECTED)
              _dl_check_protected_symbol (undef_name, undef_map, ref, map,
                                          type_class);
```

8.8.3 根据 binding 决定返回策略

```c
          /*
           * 根据符号绑定类型决定如何处理。
           */
          switch (ELFW(ST_BIND) (sym->st_info))
            {
            case STB_WEAK:
              /*
               * 弱符号定义。
               *
               * 如果启用了 dl_dynamic_weak：
               *   - 第一次遇到弱符号时先保存；
               *   - 不立即返回；
               *   - 继续向后查找是否存在强符号定义。
               */
              if (__glibc_unlikely (GLRO(dl_dynamic_weak)))
                {
                  /*
                   * 如果当前还没有记录任何结果，则保存这个弱符号。
                   */
                  if (! result->s)
                    {
                      result->s = sym;
                      result->m = (struct link_map *) map;
                    }

                  /*
                   * 继续查找后续对象。
                   */
                  break;
                }

              /*
               * 如果没有启用 dl_dynamic_weak，则弱符号按照普通全局符号处理。
               * 这里故意 fallthrough 到 STB_GLOBAL。
               */
              /* FALLTHROUGH */

            case STB_GLOBAL:
              /*
               * 全局符号定义。（找到全局强符号，直接返回）
               *
               * 这是最常见、最直接的成功路径：
               *   - 保存符号表项；
               *   - 保存符号所在对象；
               *   - 返回 1 表示找到符号。
```

```
                 */
                result->s = sym;
                result->m = (struct link_map *) map;
                return 1;

              case STB_GNU_UNIQUE:;
                /*
                 * GNU unique 符号。
                 *
                 * 这类符号通常用于 C++ 场景，
                 * 需要保证进程范围内只有唯一实例。
                 *
                 * 因此交给 do_lookup_unique 做专门处理。
                 */
                do_lookup_unique (undef_name, new_hash, (struct link_map *)
map,
                                  result, type_class, sym, strtab, ref,
                                  undef_map, flags);

                /*
                 * unique 符号处理完成后返回成功。
                 */
                return 1;

              default:
                /*
                 * 其他绑定类型，例如本地符号，直接忽略。
                 */
                break;
              }
          }

skip:
        /*
         * 当前对象没有找到可用符号，或者找到的符号不可用。
         * 继续搜索作用域中的下一个对象。
         */
        ;
      }
    while (++i < n);

    /*
     * 遍历完整个作用域后仍然没有找到符号。
     */
    return 0;
}
```